Titan-Board-Mini SDK

开发文档

Titan Board Mini 开发板是 RT-Thread 推出基于瑞萨 Cortex-M85 与 Cortex-M33 双核架构 R7KA8P1 芯片，为工程师们提供了一个灵活、全面的开发平台，助力开发者在嵌入式物联网领域获得更深层次的体验。

Version 1.2.0 | 中文 | 2026年10月22日

www.rt-thread.org

Copyright (c) 2006-2025, RT-Thread Development Team

目录

- 1. 快速上手
  - 1.1. 模板工程
- 2. 基础篇
  - 2.1. LED 闪烁示例
- 3. 驱动篇
  - 3.1. Titan Mini Driver All 示例说明
  - 3.2. Ethernet 示例说明
- 4. 多媒体显示篇
  - 4.1. MIPI CSI 摄像头示例说明
  - 4.2. LVGL 示例说明
  - 4.3. MIPI NPU 人脸检测
- 5. 多核通信篇
  - 5.1. Titan_Mini_rpmsg_master 核间通信示例（M85）
  - 5.2. Titan_Mini_rpmsg_remote 核间通信示例（M33）
- 6. 外设扩展篇
  - 6.1. PDM 麦克风实时回环示例说明
  - 6.2. IMU 传感器示例说明
  - 6.3. RGB LCD 示例说明
  - 6.4. USB-PCDC 驱动示例说明
  - 6.5. WAV 音频播放器示例说明

1. 快速上手

1.1. 模板工程

中文 | **English**

简介

这是 **Titan Board Mini** 的模板工程。它实现了基本的 LED 闪烁功能，可以作为二次开发的起点。

特性

本模板提供：

- RT-Thread 完整系统配置
- 基础 LED 驱动（GPIO）
- 串口调试输出（UART）
- 二次开发项目结构

项目结构

```
project/
├── board/                # 板级配置
├── libraries/            # RA8P1 HAL 库
├── rt-thread/            # RT-Thread 内核
├── src/                  # 应用源代码
├── Kconfig               # Menuconfig 配置
├── SConstruct            # 构建脚本
└── ...
```

快速开始

1. 在 RT-Thread Studio 中打开工程
2. 如需要，使用 menuconfig 配置工程
3. 编译工程
4. 下载并运行

二次开发

本模板可作为以下开发的基础：

- 添加新的外设
- 实现自定义应用
- 开发驱动程序
- 构建完整应用

进阶阅读

- RT-Thread 文档

2. 基础篇

2.1. LED 闪烁示例

中文 | English

简介

本示例是 SDK 中最简单、最基础的示例，类似于程序员接触的第一个程序 Hello World 一样简洁。它的主要功能是让板载的 RGB LED 进行周期性闪烁。

GPIO 简介

1. 概述

GPIO（通用输入输出） 是 MCU 最常用的外设接口之一，能够在软件控制下配置为 **输入模式** 或 **输出模式**：

- **输入模式**：用于读取外部电平状态，例如按键输入。
- **输出模式**：用于控制外设电平，例如点亮 LED、驱动蜂鸣器。

2. RA8 系列 GPIO 特点

- **丰富的引脚资源**：每个端口可独立配置为输入/输出/复用功能。
- **多功能复用**：除 GPIO 功能外，同一引脚还可能作为 UART、SPI、I2C 等外设的信号线。
- **电平控制**：输出时可驱动高电平或低电平，部分引脚支持开漏模式。
- **方向控制**：输入/输出可动态切换。

3. RT-Thread PIN 框架

RT-Thread 提供了 **PIN 设备驱动框架**，通过统一的接口屏蔽底层硬件差异：

- `rt_pin_mode()` - 设置引脚工作模式（输入/输出/上拉/下拉等）

- `rt_pin_write()` - 输出电平（高/低）
- `rt_pin_read()` - 读取输入电平

这样开发者不需要直接操作寄存器，而是通过 RT-Thread 的 API 即可完成 GPIO 控制。

在本示例中，LED 引脚被配置为 **输出模式**，软件循环输出高低电平，从而实现 LED 闪烁。

硬件说明

Titan Board Mini 提供三个用户 LED：LED_R（红色）、LED_B（蓝色）、LED_G（绿色）。LED 对应 MCU 上的特定引脚。MCU 引脚输出低电平即可点亮 LED，输出高电平则会熄灭 LED。

软件说明

本例程的源码位于 `src/hal_entry.c`。RGB LED 对应的单片机引脚定义及 LED 控制逻辑可以查阅源码文件。

```
/* 配置 LED 灯引脚 */
rt_pin_mode(LED_PIN, PIN_MODE_OUTPUT);

/* 控制 LED */
rt_pin_write(LED_PIN, LED_ON);  // 点亮
rt_pin_write(LED_PIN, LED_OFF); // 熄灭
```

编译下载

1. 在 RT-Thread Studio 中打开工程
2. 点击构建按钮进行编译
3. 连接调试器到开发板
4. 下载固件到开发板

运行效果

按下复位按键重启开发板，观察 LED 闪烁效果。RGB LED 会周期性变化。

进阶阅读

- RT-Thread PIN 设备

3. 驱动篇

3.1. Titan Mini Driver All 示例说明

English | 中文

简介

`Titan_Mini_driver_all` 是 Titan Mini Board 的外设驱动示例集合工程，用于集中验证常用外设、存储和网络功能。工程通过 MSH 命令行提供各外设的单项示例入口，方便单独调试。

当前工程内已集成的示例包括：

- RTC
- GPIO / PWM / 按键中断
- SD 卡
- Flash 文件系统
- SDRAM
- ADC
- SPI
- CANFD
- I2C
- Watchdog

FSP 环境配置

本工程基于 **Renesas FSP (Flexible Software Package) 6.4.0** 和 **RT-Thread Studio** 进行硬件抽象层配置与代码生成。开箱即用无需重新配置，但若需要修改外设引脚或新增 FSP Stack，请遵循以下流程。

FSP 下载链接

- setup_fsp_v6_4_0_rasc_v2025-12.exe

版本要求

项目	版本	说明
FSP 版本	6.4.0	`configuration.xml` 中 `#FSPVersion#` 已锁定
MCU	R7KA8P1KFLCAC (RA8P1)	Cortex-M85 + Cortex-M33 双核
IDE	RT-Thread Studio	内置 FSP 配置插件

> ⚠️ *请务必使用 FSP 6.4.0。其他版本可能导致 `ra/`、`ra_gen/`、`ra_cfg/` 重新生成代码与驱动层不兼容。*

工程目录结构

```
Titan_Mini_driver_all/
├── configuration.xml       # FSP 配置文件（引脚/时钟/Stack 定义）
├── ra/                     # FSP 生成的驱动源码（r_gpt/r_spi/r_can
├── ra_cfg/                 # FSP 配置头文件（fsp_cfg/）
├── ra_gen/                 # FSP 自动生成的硬件抽象数据（hal_data.c
├── board/                  # BSP 板级初始化
├── src/                    # 应用层示例代码（rt_example_*.c）
└── rtconfig.h              # RT-Thread 内核配置
```

修改 FSP 配置步骤

1. **打开配置**：在 RT-Thread Studio 中右键工程 → `Renesas FSP Configuration`（或双击 `configuration.xml`）。
2. **调整 Stack**：在 `Stacks` 视图中新增/移除驱动 Stack（如 `r_gpt`、`r_adc`、`r_canfd`），每个 Stack 的属性需与外设章节描述的通道/引脚一致。

3. **配置引脚**：在 `Pins` 视图中分配引脚模式（GPIO/SPI/I2C/CAN 等），以太网相关引脚需将驱动能力改为 `H`。
4. **生成代码**：点击 `Generate Project Content`，FSP 会重新生成 `ra_gen/hal_data.c`、`ra_gen/pin_data.c` 和 `ra_cfg/` 下的配置头。
5. **同步 RT-Thread Settings**：在 `RT-Thread Settings` 中勾选对应驱动框架（PWM / ADC / CAN / SPI / I2C / WDT 等），使 RT-Thread 设备框架注册对应设备节点（如 `pwm6`、`adc0`、`canfd0`）。
6. **重新编译**：FSP 生成代码后必须 clean 再 build，避免旧 `.o` 缓存导致链接错误。

常见问题

- **找不到 `pwm6` / `adc0` 等设备**：通常是 RT-Thread Settings 未勾选对应驱动框架，或 FSP Stack 未添加。
- **`hal_data.h` 报错**：FSP 版本不一致，请用 6.4.0 重新生成。
- **ETH 引脚通信异常**：确认 ETH 所有引脚驱动能力已设为 `H`（参考 Ethernet 章节）。
- **修改配置后链接错误**：执行 `Project → Clean` 后再编译。

编译与运行

1. 使用 RT-Thread Studio 打开 `project/Titan_Mini_driver_all` 工程。
2. 编译并下载固件到 Titan Mini 开发板。
3. 通过串口终端或 USB PCDC 终端进入 MSH 命令行。
4. 输入 `help` 查看可用命令，各外设示例命令详见后续章节。

上电后终端会打印如下启动信息：

```
============================================================
           Titan Mini Board Factory Test System
============================================================
System initialized successfully!
```

RTC 示例

介绍

RTC 示例基于 **RT-Thread RTC 设备驱动框架**，验证 RA8P1 内置实时时钟外设能否正常工作，支持日期/时间设置与读取，以及在启用 `RT_USING_ALARM` 组件时验证闹钟功能。RTC 由 32.768kHz 晶振驱动，主电源断开后可通过 VBATT 电池备份继续走时。

硬件参数

参数	规格	说明
时钟源	32.768kHz 晶振	副时钟振荡器，精度 ±5ppm
日历范围	2000-2099	自动闰年补偿
时间格式	24 小时制	时:分:秒
工作电压	1.62-3.6V	宽电压范围
备份电源	VBATT	主电源断开后保持时间信息
闹钟触发	秒/分/时/日/周/月/年	多种可编程触发方式

使用步骤

1. 输入 `rtc_sample`，设置 RTC 日期为 2025-8-1、时间为 15:00:00，3 秒后读取并打印当前时间。
2. 如果工程启用了 `RT_USING_ALARM`，可输入 `alarm_sample` 创建一个秒级闹钟。

关键代码

示例源码位于 `src/rt_example_rtc.c`，核心逻辑如下。

时间设置与读取

```
void rtc_sample(void)
{
    rt_device_t device = rt_device_find(RTC_NAME);     // RTC_N
    if (device == RT_NULL)
    {
        rt_kprintf("find %s failed!\n", RTC_NAME);
        return;
    }

    if (rt_device_open(device, 0) != RT_EOK)
    {
        rt_kprintf("open %s failed!\n", RTC_NAME);
        return;
    }

    set_date(2025, 8, 1);                              // 设置日
    set_time(15, 0, 0);                                // 设置时

    rt_thread_mdelay(3000);
    time_t now = 0;
    get_timestamp(&now);
    rt_kprintf("now: %.*s", 25, ctime(&now));
}
MSH_CMD_EXPORT(rtc_sample, rtc sample);
```

闹钟创建（依赖 `RT_USING_ALARM`）

```
static void user_alarm_callback(rt_alarm_t alarm, time_t timest
{
    rt_kprintf("user alarm callback function.\n");
}

void alarm_sample(void)
{
    struct rt_alarm_setup setup;
    static rt_alarm_t alarm = RT_NULL;
    time_t now = get_timestamp(RT_NULL) + 1;            // 下一秒
    struct tm p_tm;
    gmtime_r(&now, &p_tm);

    setup.flag = RT_ALARM_SECOND;                       // 秒级闹
    setup.wktime = *(struct tm *)&p_tm;                 // 触发时

    alarm = rt_alarm_create(user_alarm_callback, &setup);
```

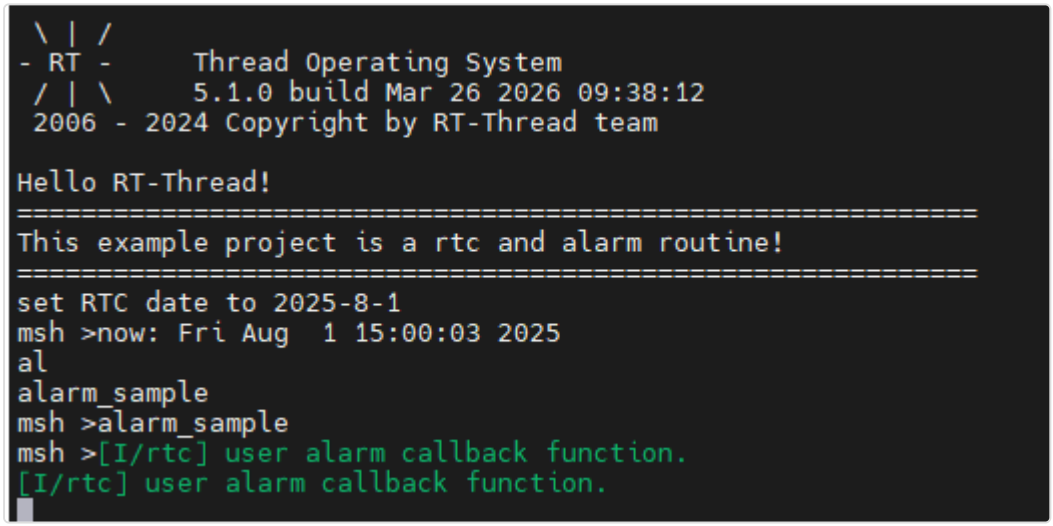
```
    rt_alarm_start(alarm);
}
```

> *若未启用 `RT_USING_ALARM`，`alarm_sample` 会打印 `alarm sample is unavailable: RT_USING_ALARM is not enabled.`。*

效果

- `rtc_sample` 会设置时间后打印当前 RTC 时间。
- `alarm_sample` 触发时会打印 `user alarm callback function.`。

运行 `alarm_sample` 后的终端效果：



GPIO / PWM / 按键中断示例

介绍

这部分示例基于 **RT-Thread PWM 设备驱动框架** 和 **PIN 设备中断接口**，验证 RA8P1 的 GPT 通用 PWM 定时器与 GPIO 外部中断功能，包含两类入口：

- `pwm_sample`：手动配置 `pwm6` 的周期和占空比，用于示波器观测
- `key_irq_sample`：配置按键下降沿中断，触发后切换绿色 LED 状态

硬件参数

参数	规格	说明
PWM 设备	`pwm6` (GPT6)	32 位通用 PWM 定时器
PWM 计数器	32 位	范围 0 ~ 4294967295
PWM 输出引脚	P601 (GTIOC6A)	示波器观测点
LED 引脚	`BSP_IO_PORT_01_PIN_08`	绿色 LED
按键引脚	`BSP_IO_PORT_02_PIN_01`	内部上拉，下降沿触发
GPT 工作模式	周期 / 单次 / PWM	方波 / 锯齿波 / 三角波

使用步骤

1. 输入 `pwm_sample <period> <pulse>`，例如 `pwm_sample 500000 250000`（单位 ns，对应 2kHz / 50% 占空比），将示波器接到 P601 观测波形。
2. 输入 `key_irq_sample`，然后按下按键观察绿色 LED 切换。

关键代码

PWM 示例源码位于 `src/rt_example_gpt.c`，按键中断源码位于 `src/rt_example_key_irq.c`。

PWM 配置（手动设置周期与脉宽）

```
#define PWM_DEV_NAME        "pwm6"
#define PWM_DEV_CHANNEL     0

static int pwm_sample(int argc, char *argv[])
{
    rt_uint32_t period = (rt_uint32_t) atoi(argv[1]);
    rt_uint32_t pulse  = (rt_uint32_t) atoi(argv[2]);

    if ((period == 0) || (pulse > period))
    {
        rt_kprintf("invalid parameters, ensure period > 0 and p
        return -RT_ERROR;
```

```
    }

    struct rt_device_pwm *pwm_dev = (struct rt_device_pwm *) rt
    rt_pwm_set(pwm_dev, PWM_DEV_CHANNEL, period, pulse);
    rt_pwm_enable(pwm_dev, PWM_DEV_CHANNEL);

    rt_kprintf("pwm started on %s channel %d, period=%u pulse=%
               PWM_DEV_NAME, PWM_DEV_CHANNEL, period, pulse);
    return RT_EOK;
}
MSH_CMD_EXPORT(pwm_sample, configure and start pwm output: pwm_
```

按键中断（下降沿切换 LED）

```
#define LED_PIN_G   BSP_IO_PORT_01_PIN_08
#define KEY_PIN     BSP_IO_PORT_02_PIN_01

static volatile rt_bool_t g_key_flag = RT_FALSE;

static void key_irq_callback(void *args)
{
    rt_pin_write(LED_PIN_G, g_key_flag ? PIN_HIGH : PIN_LOW);
    g_key_flag = g_key_flag ? RT_FALSE : RT_TRUE;
}

void key_irq_sample(void)
{
    rt_pin_mode(LED_PIN_G, PIN_MODE_OUTPUT);
    rt_pin_mode(KEY_PIN, PIN_MODE_INPUT_PULLUP);
    rt_pin_attach_irq(KEY_PIN, PIN_IRQ_MODE_FALLING, key_irq_ca
    rt_pin_irq_enable(KEY_PIN, PIN_IRQ_ENABLE);
}
MSH_CMD_EXPORT(key_irq_sample, key interrupt sample);
```

参数换算说明

`pwm_sample` 参数单位为**纳秒**：

```
频率 = 1e9 / period
占空比 = pulse / period × 100%

示例 pwm_sample 500000 250000:
  频率 = 1e9 / 500000 = 2000 Hz (2 kHz)
  占空比 = 250000 / 500000 = 50%
```

效果

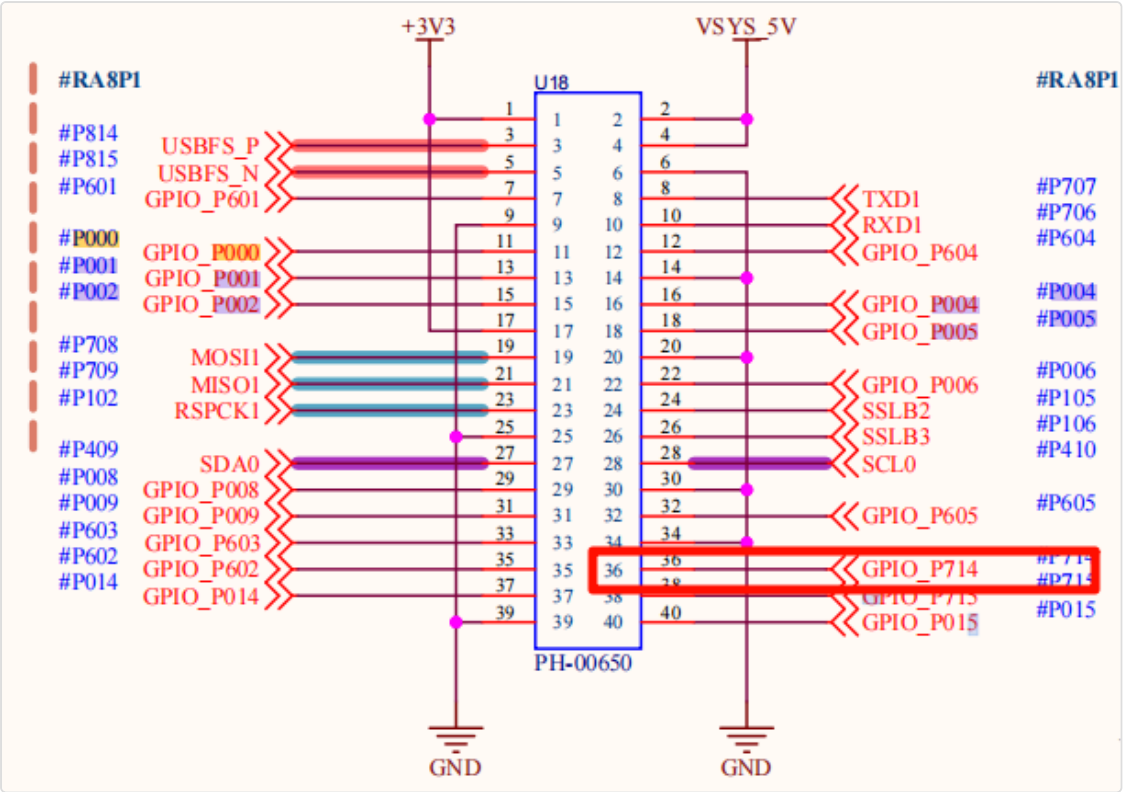
- `pwm_sample` 成功后会打印当前设备、通道、周期和脉宽，P601 可观测到对应波形。
- `key_irq_sample` 启动后，按键每触发一次下降沿会切换一次绿色 LED。

`pwm_sample 500000 250000` 的终端输出：

```
 \ | /
- RT -     Thread Operating System
 / | \     5.1.0 build Mar 25 2026 17:53:14
 2006 - 2024 Copyright by RT-Thread team

Hello RT-Thread!
============================================================
This example project is a Titan Board Mini template routine!
============================================================
[I/pwm] Tips
[I/pwm] Please enter one of the above commands to run the corresponding sample.
[I/pwm]    • hwtimer_sample : Demonstration of Hardware Timer usage.
[I/pwm]    • pwm_sample     : Demonstration of PWM (Pulse Width Modulation) usage
.
msh >pwm
pwm_sample
pwm
msh >pwm_s
pwm_sample
msh >pwm_sample
[I/pwm] Usage: pwm_sample <period> <pulse>
[I/pwm] Example: pwm_sample 500000 250000
msh >pwm_sample 500000 250000
[I/pwm] PWM started on device: pwm12, channel: 0
[I/pwm] Period: 500000 ns, Pulse: 250000 ns
[I/pwm] Please connect the 'P714' to a logic analyzer or oscilloscope for wavefo
rm observation.
msh >
```

将 P601 接入示波器观测到的 2kHz / 50% PWM 波形：



SD 卡示例

介绍

SD 卡示例基于 **RT-Thread DFS 文件系统框架** 和 **RA8 SDHI 硬件模块**，验证 `/sdcard` 挂载点能否正常创建、写入、读取和删除测试文件。SDHI 通过 SD 总线与 SD/SDHC/SDXC 卡通信，支持 1-bit/4-bit 数据线和 DMA 传输。

硬件参数

参数	规格	说明
支持卡类型	SDSC / SDHC / SDXC	兼容 SD v1.x / v2.x
总线宽度	1-bit / 4-bit	工程默认 1-bit
数据块大小	512 Byte	标准块大小
最高时钟	50 MHz SDCLK	取决于 MCU 时钟配置

参数	规格	说明
工作电压	3.3V	部分 Micro SD 支持 1.8V
错误校验	CRC7(命令) / CRC16(数据)	硬件自动校验

使用步骤

1. 确认 SD 卡已插入并已正确挂载到 `/sdcard`（由 `BSP_USING_FS_AUTO_MOUNT` + `BSP_USING_SDCARD_FATFS` 自动完成）。
2. 输入 `sdcard_sample` 执行一次文件读写校验。

关键代码

示例源码位于 `src/rt_example_sdcard.c`，使用 POSIX 标准 `fopen/fputs/fgets/unlink` 接口。

```
void sdcard_sample(void)
{
    const char *test_file = "/sdcard/test_sdcard.txt";
    const char *test_data = "SD Card test data - Titan Mini Boa
    char read_buf[64] = {0};

    /* 1. 写入测试数据 */
    FILE *fp = fopen(test_file, "w");
    if (fp == RT_NULL)
    {
        rt_kprintf("failed to create test file: %s\n", test_fil
        return;
    }
    fputs(test_data, fp);
    fclose(fp);

    /* 2. 读回校验 */
    fp = fopen(test_file, "r");
    if (fgets(read_buf, sizeof(read_buf), fp) == RT_NULL)
    {
        rt_kprintf("failed to read test data\n");
        fclose(fp);
        return;
    }
    fclose(fp);

    /* 3. 比对内容 */
```

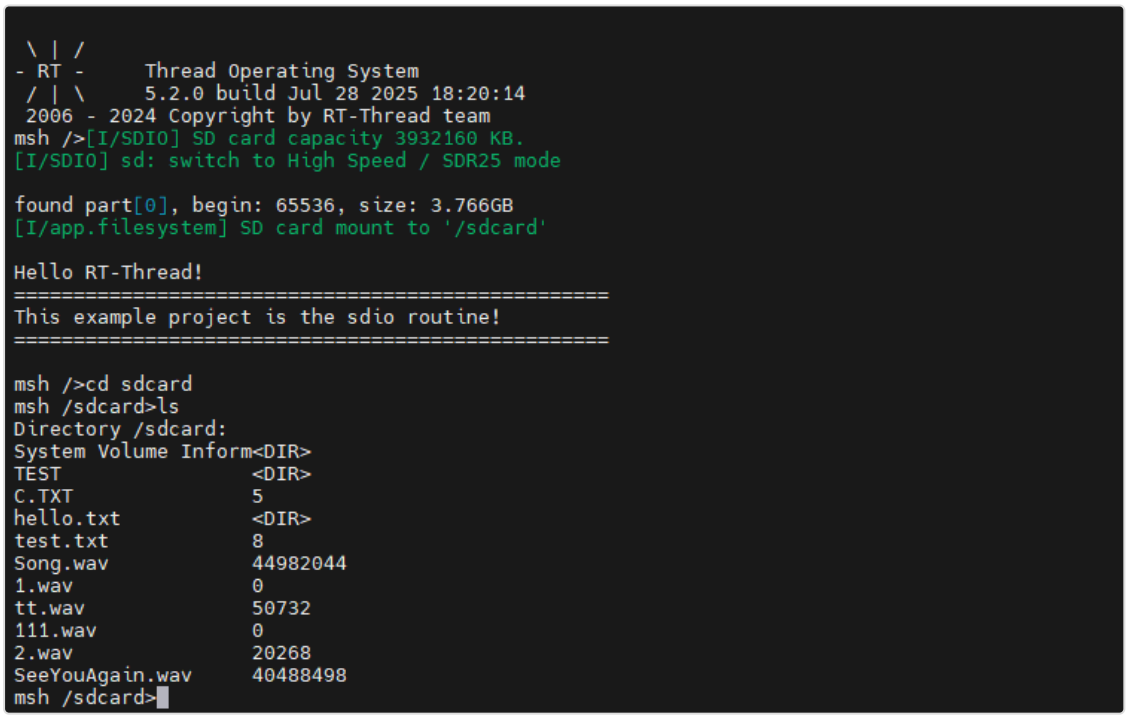
```
    if (strcmp(read_buf, test_data) != 0)
    {
        rt_kprintf("sdcard data mismatch!\n");
        return;
    }

    /* 4. 清理测试文件 */
    unlink(test_file);
    rt_kprintf("sdcard sample passed\n");
}
MSH CMD EXPORT(sdcard sample, sdcard file read write sample);
```

效果

- 成功时会完成测试文件写入、读取、比对和删除，并打印 `sdcard sample passed`。
- 若挂载异常、写入失败或读回数据不一致，会打印对应的错误原因（`failed to create/write/read` 或 `data mismatch`）。

SD 卡挂载成功后的终端效果：



Flash 文件系统示例

介绍

Flash 示例基于 **RT-Thread DFS + FAL + LittleFS** 三层架构，验证板载 QSPI Flash（W25Q64）上 `/fal` 挂载点的基本读写能力。底层通过 FAL 抽象层管理 Flash 分区，文件系统采用断电安全的 LittleFS。

硬件参数

参数	规格	说明
Flash 型号	W25Q64	板载 QSPI NOR Flash
容量	8MB (64Mbit)	4KB 扇区 / 64KB 块 / 256B 页
接口	QSPI (Quad SPI)	4 位数据线，最高 133MHz
工作电压	2.7V - 3.6V	—
擦写寿命	~10 万次/扇区	常温数据保持 ~20 年
文件系统分区	`filesystem` (1MB)	FAL 分区表定义

软件分层

```
应用层 (flash_sample)
    ↓
DFS (DFS 框架，提供 fopen/fread/POSIX 接口)
    ↓
LittleFS (断电安全 + 磨损均衡)
    ↓
FAL (Flash 抽象层，管理分区)
    ↓
W25Q64 QSPI 驱动 → OSPI_B 硬件
```

使用步骤

1. 确认 FAL 和 LittleFS 已启用并自动挂载到 `/fal`（由 `BSP_USING_FLASH_FS_AUTO_MOUNT` 完成，首次挂载失败会自动

`dfs_mkfs("lfs", ...)` 格式化）。

2. 输入 `flash_sample` 执行一次小文件读写校验（功能验证）。
3. 输入 `flash_speed_test` 执行 64KB 文件写入/读取/删除带宽测试。

关键代码

示例源码位于 `src/rt_example_flash.c`，提供两个命令：`flash_sample`（功能校验）和 `flash_speed_test`（带宽测速）。

功能校验（flash_sample）

```
void flash_sample(void)
{
    const char *test_file = "/fal/test_flash.txt";
    const char *test_data = "Flash test data - Titan Mini Board
    char read_buf[64] = {0};

    /* 1. 写入测试数据 */
    FILE *fp = fopen(test_file, "w");
    if (fp == RT_NULL)
    {
        rt_kprintf("failed to create test file: %s\n", test_fil
        return;
    }
    fputs(test_data, fp);
    fclose(fp);

    /* 2. 读回校验 */
    fp = fopen(test_file, "r");
    if (fgets(read_buf, sizeof(read_buf), fp) == RT_NULL)
    {
        rt_kprintf("failed to read test data\n");
        fclose(fp);
        return;
    }
    fclose(fp);

    /* 3. 比对内容 */
    if (strcmp(read_buf, test_data) != 0)
    {
        rt_kprintf("flash data mismatch!\n");
        return;
    }

    /* 4. 清理测试文件 */
    unlink(test_file);
```

```
    rt_kprintf("flash sample passed\n");
}
MSH CMD EXPORT(flash sample, flash file read write sample);
```

带宽测速（flash_speed_test）

使用 DWT 周期计数器对 64KB 文件的写入、读取、删除三项操作计时，并换算为 KB/s 带宽。读取阶段同时做数据校验。

```
#define FLASH_SPEED_TEST_SIZE   (64 * 1024)   /* 64KB 测试文件 */
#define FLASH_SPEED_BUF_SIZE    4096          /* 单次读写块大小 */

void flash_speed_test(void)
{
    /* 1. 顺序写入 64KB（每次 4KB），DWT 计时 */
    dwt_init();
    FILE *fp = fopen("/fal/speed_test.bin", "wb");
    uint32_t start = dwt_get_cycle();
    for (offset = 0; offset < total; offset += FLASH_SPEED_BUF_
        fwrite(wr_buf, 1, FLASH_SPEED_BUF_SIZE, fp);
    fflush(fp); fclose(fp);
    flash_speed_report("Write", dwt_get_cycle() - start, total)

    /* 2. 顺序读取 + 数据校验，DWT 计时 */
    fp = fopen("/fal/speed_test.bin", "rb");
    start = dwt_get_cycle();
    for (offset = 0; offset < total; offset += FLASH_SPEED_BUF_
    {
        fread(rd_buf, 1, FLASH_SPEED_BUF_SIZE, fp);
        if (memcmp(rd_buf, wr_buf, FLASH_SPEED_BUF_SIZE) != 0)
    }
    flash_speed_report("Read", dwt_get_cycle() - start, total);
    fclose(fp);

    /* 3. 删除文件（对应 Flash 擦除），DWT 计时 */
    start = dwt_get_cycle();
    unlink("/fal/speed_test.bin");
    flash_speed_report("Erase", dwt_get_cycle() - start, total)
}
MSH_CMD_EXPORT(flash_speed_test, flash read/write speed test);
```

带宽换算公式（与 SDRAM 一致）：

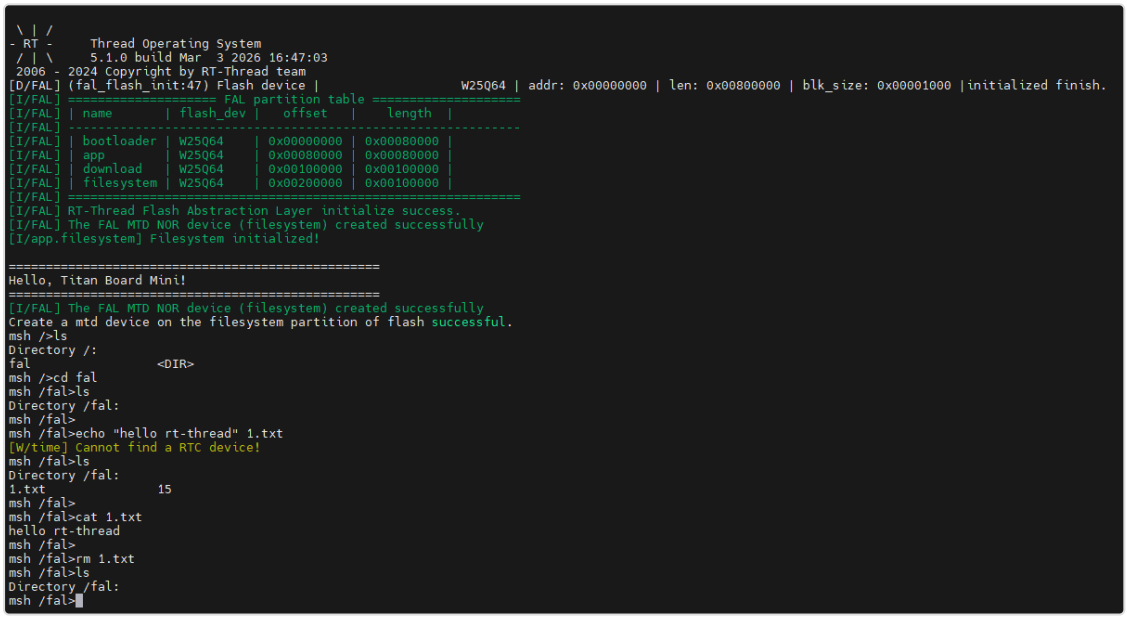
```
耗时(秒) = DWT 差值 / SystemCoreClock
带宽(KB/s) = (测试字节数 / 1024) / 耗时
```

效果

- `flash_sample` 成功时打印 `flash sample passed`；失败时打印 `failed to create/write/read` 或 `flash data mismatch!`。
- `flash_speed_test` 会输出 `Write`、`Read`、`Erase` 三项带宽（KB/s）和耗时（ms），并在读取阶段做数据校验。

> *Flash 写入前必须先擦除，且需按扇区/页对齐。LittleFS 会自动处理磨损均衡和断电保护，应用层直接用 POSIX 接口即可。*

Flash 文件系统挂载成功的终端效果：



SDRAM 示例

介绍

SDRAM 示例基于 **RA8P1 SDRAMC 控制器**，验证外部 64MB SDRAM 是否可读写，并通过 DWT 周期计数器统计写入、读取和 `memcpy` 三项操作的带宽表现。

硬件参数

参数	规格	说明
容量	64MB (256Mbit)	4 banks × 4M × 32bit
数据宽度	32 位	高吞吐量数据传输
基地址	`0x68000000`	访问范围 0x68000000 ~ 0x68FFFFFF
工作电压	3.3V	—
CAS 延迟	CL3	可编程时序
刷新机制	自动刷新	SDRAMC 内置，保证数据完整性

使用步骤

1. 输入 `sdram_speed_test` 执行一次 SDRAM 速度测试。

关键代码

示例源码位于 `src/rt_example_sdram.c`，使用 DWT 周期计数器实现纳秒级计时。

内存映射与计时辅助

```
#define SDRAM_TEST_WORDS    (8 * 1024 * 1024)             // 8M
#define SDRAM_BASE_ADDR     (0x68000000)

static volatile uint32_t *sdram_buf = (uint32_t *) SDRAM_BASE_A

static void dwt_init(void)
{
    CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
    DWT->CYCCNT = 0;
    DWT->CTRL  |= DWT_CTRL_CYCCNTENA_Msk;
}
```

效果

- 终端会输出测试容量、CPU 频率以及 `Write`、`Read`、`memcpy` 三项带宽结果（单位 MB/s 和 ms）。
- 若读回校验异常，会打印错误地址与期望值/实际值（`expect_low16` / `actual_low16`）。

`sdram_speed_test` 运行效果：

```
msh >[D/drv_sdram] sdram init success, mapped at 0x68C00000, size is 20971520 by
tes, data width is 32

 \ | /
- RT -     Thread Operating System
 / | \     5.1.0 build Mar 25 2026 17:28:15
 2006 - 2024 Copyright by RT-Thread team

Hello RT-Thread!
============================================================
This example project is a Titan Board Mini template routine!
============================================================
msh >sd
sdram_speed_test
msh >sdram_speed_test

========== SDRAM 32-BIT SPEED TEST ==========
CPU Freq: 1000000000 Hz
Test Size: 32 MB
Write:  233.44 MB/s  (137.081 ms)
Read:   166.99 MB/s  (191.626 ms)
memcpy: 82.48 MB/s  (387.954 ms)
=============================================
```

ADC 示例

介绍

ADC 示例基于 **RT-Thread ADC 设备驱动框架**，验证 RA8P1 的 `adc0` 通道 0 是否能正常采样，并将原始值换算为电压值。示例额外加入了**无效值检测**和**接近满量程告警**逻辑，避免输入超出参考电压时被误判为正常采样。

硬件参数

参数	规格	说明
分辨率	16 位	原始值范围 0 ~ 65535
参考电压	3.3V	代码中以 `330`（即 3.30V × 100）定点表示

参数	规格	说明
采样通道	`adc0` 通道 0	单极性输入，范围 0V ~ 3.3V
触发方式	软件触发	通过 `rt_adc_read` 周期采样

> ⚠️ ***重要：****输入电压必须低于参考电压 Vref。若直接施加 3.3V，可能触发满量程告警或溢出提示。*

使用步骤

1. 将待测模拟电压接入 ADC 对应通道（输入应低于 Vref）。
2. 输入 `adc_sample` 启动持续采样线程，每秒输出一次原始值与换算电压。

关键代码

示例源码位于 `src/rt_example_adc.c`，核心逻辑如下。

电压换算常量

```
#define ADC_DEV_NAME        "adc0"
#define ADC_DEV_CHANNEL     0
#define REFER_VOLTAGE       330               // 3.30V × 100，定点
#define CONVERT_BITS        (1 << 16)         // 16 位分辨率
#define ADC_INVALID_VALUE   0xFFFF            // ADC 超量程标记值
#define ADC_WARN_THRESHOLD  (CONVERT_BITS * 95 / 100)  // 满量程
```

采样线程主循环

```
static void adc_vol_sample(void *parameter)
{
    rt_adc_device_t adc_dev = (rt_adc_device_t)rt_device_find(A
    if (adc_dev == RT_NULL)
    {
        rt_kprintf("adc sample run failed! can't find %s device
        return;
    }

    if (rt_adc_enable(adc_dev, ADC_DEV_CHANNEL) != RT_EOK)
```

```
    {
        rt_kprintf("adc enable failed! channel = %d\n", ADC_DEV
        return;
    }

    while (1)
    {
        rt_uint32_t value = rt_adc_read(adc_dev, ADC_DEV_CHANNE

        if (value == ADC_INVALID_VALUE)
        {
            rt_kprintf("adc overrange: input is too high, pleas
        }
        else
        {
            rt_uint32_t vol = value * REFER_VOLTAGE / CONVERT_B
            rt_kprintf("the value is :%d\n", value);

            if (value >= ADC_WARN_THRESHOLD)
                rt_kprintf("warning: near full scale, the volta
            else
                rt_kprintf("the voltage is :%d.%02d\n", vol / 1
        }

        rt_thread_mdelay(1000);
    }
}
```

命令导出

```
void adc_sample(void)
{
    rt_thread_t adc = rt_thread_create("adc", adc_vol_sample, R
    if (adc == RT_NULL)
    {
        rt_kprintf("create adc thread failed!\n");
        return;
    }
    rt_thread_startup(adc);
}
MSH_CMD_EXPORT(adc_sample, adc sample demo);
```

电压换算说明

采用定点数计算避免浮点开销：

```
电压(mV) = ADC 读数 × 参考电压(mV) / 转换位数
         = value × 3300 / 65536

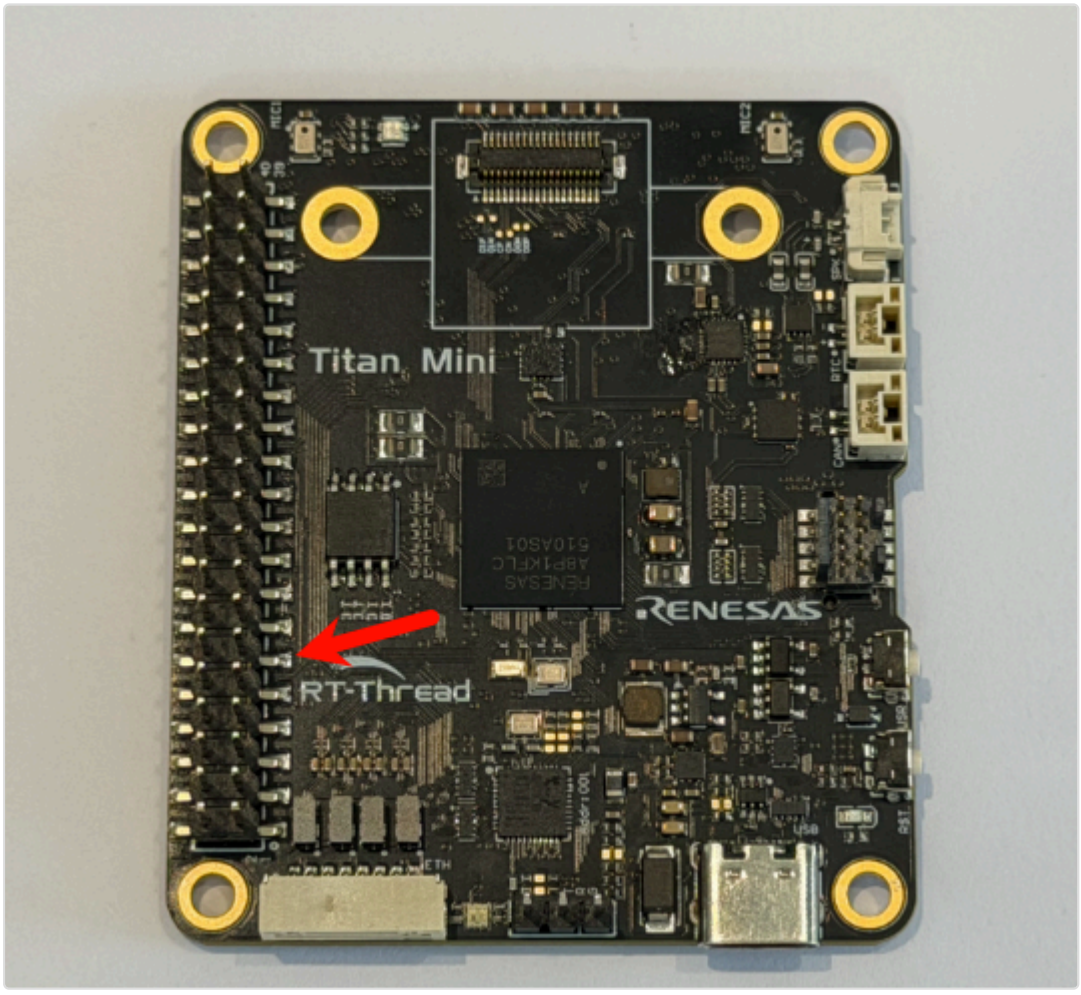
代码中: vol = value * 330 / 65536    （单位为 0.01V）
显示:   vol / 100  -> 整数部分（V）
        vol % 100  -> 小数部分（0.01V）
```

例如 `value = 32768` 时，`vol ≈ 165`，输出 `the voltage is :1.65`。

效果

- `adc_sample` 会周期性打印原始采样值和换算后的电压。
- 当读数为 `0xFFFF` 时提示 `adc overrange`，输入超出量程。
- 当读数接近满量程（≥ 95%）时输出 `warning: near full scale`。

测试时可用杜邦线将 3.3V 接到 ADC 引脚进行验证：



运行 `adc_sample` 后的终端采样输出：

```
 \ | /
- RT -     Thread Operating System
 / | \     5.1.0 build Mar 25 2026 15:46:49
 2006 - 2024 Copyright by RT-Thread team
[D/drv.adc] adc0 init success

Hello RT-Thread!
============================================================
This example project is a Titan Board Mini template routine!
============================================================
msh >adc_sample
the value is :0
the voltage is :0.00
msh >the value is :12472
the voltage is :0.62
the value is :12464
the voltage is :0.62
the value is :65452
the voltage is :3.29
the value is :65408
the voltage is :3.29
the value is :65412
the voltage is :3.29
the value is :65400
the voltage is :3.29
the value is :65444
the voltage is :3.29
the value is :12468
the voltage is :0.62
the value is :12488
the voltage is :0.62
the value is :12472
the voltage is :0.62
the value is :12468
the voltage is :0.62
the value is :12464
the voltage is :0.62
```

SPI 示例

介绍

SPI 示例基于 **RT-Thread SPI 设备驱动框架**，在 `spi1` 总线上进行回环测试，默认以 1 MHz、Mode 0、8 位数据宽度工作。测试前需要将 MOSI 与 MISO 短接。

硬件参数

参数	规格	说明
SPI 总线	`spi1`	RA8 硬件 SPI

参数	规格	说明
挂载设备	`spi10`	通过 `rt_hw_spi_device_attach` 创建
数据宽度	8 位	可配置 1-16 位
最大时钟	25 MHz（取决于系统时钟）	示例使用 1 MHz
工作模式	Mode 0 (CPOL=0, CPHA=0)	主模式 + MSB 优先
片选	软件控制（`RT_SPI_NO_CS`）	回环测试无需片选
MOSI/MISO	P708 / P709	树莓派扩展接口

使用步骤

1. 按照硬件连接要求短接 P708（MOSI）与 P709（MISO）。
2. 输入 `spi_loop_test` 执行基础 SPI 回环示例。

关键代码

示例源码位于 `src/rt_example_spi.c`。

```
#define SPI_BUS_NAME        "spi1"
#define SPI_NAME            "spi10"

void spi_loop_test(void)
{
    static uint8_t sendbuf[1024];
    static uint8_t readbuf[1024];

    /* 1. 准备测试数据 0x00 ~ 0xFF 循环 */
    for (int i = 0; i < (int) sizeof(sendbuf); i++)
        sendbuf[i] = (uint8_t) i;

    /* 2. 挂载 spi 设备并配置 */
    rt_hw_spi_device_attach(SPI_BUS_NAME, SPI_NAME, RT_NULL);
    struct rt_spi_configuration cfg = {
```

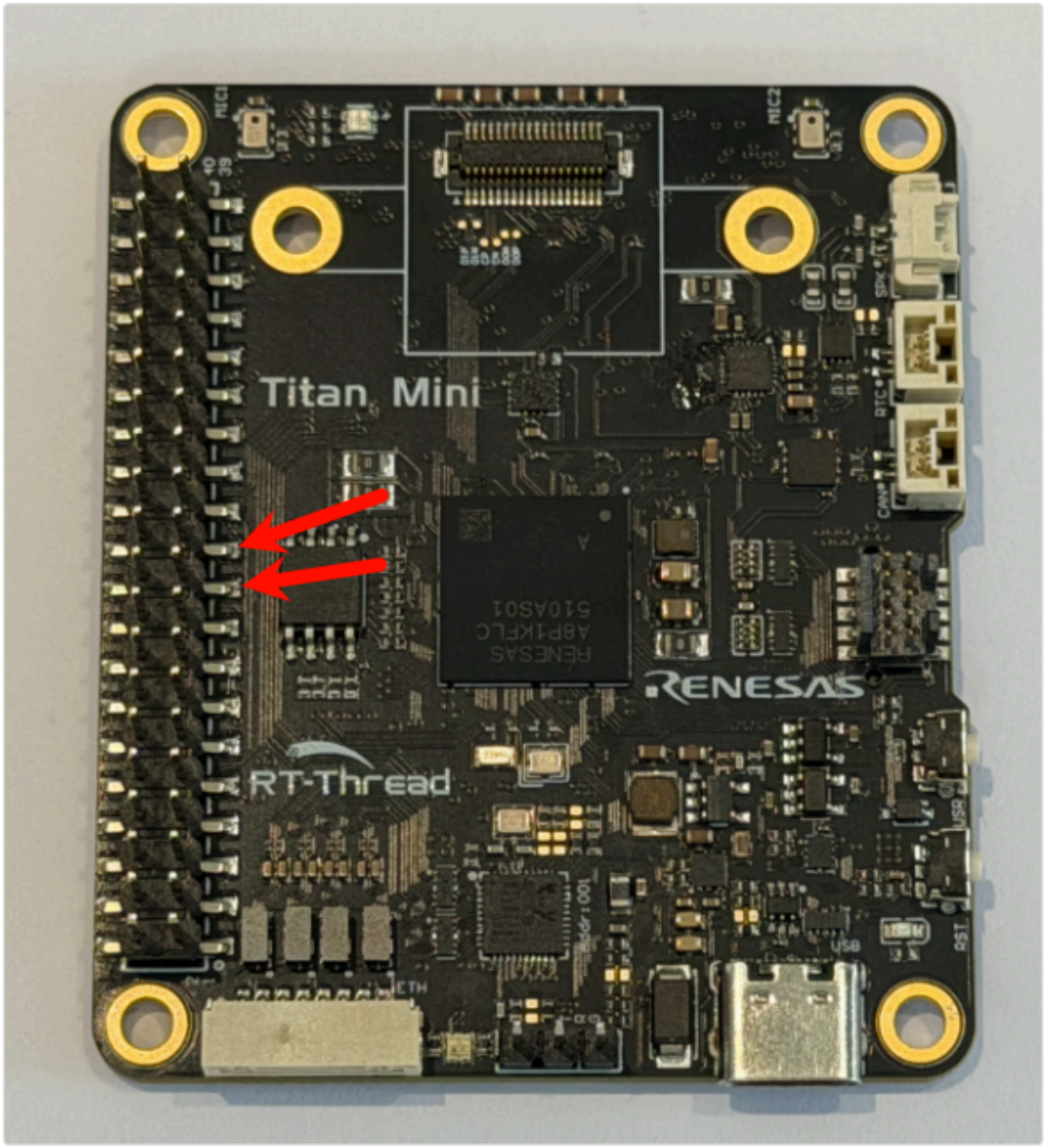
```
        .data_width = 8,
        .mode = RT_SPI_MASTER | RT_SPI_MODE_0 | RT_SPI_MSB | RT
        .max_hz = 1 * 1000 * 1000,                  // 1 MHz
    };
    struct rt_spi_device *spi_dev = (struct rt_spi_device *) rt
    rt_spi_configure(spi_dev, &cfg);

    /* 3. 全双工传输 + 逐字节比对 */
    rt_spi_transfer(spi_dev, sendbuf, readbuf, sizeof(sendbuf))
    for (int i = 0; i < (int) sizeof(readbuf); i++)
    {
        if (readbuf[i] != sendbuf[i])
        {
            rt_kprintf("SPI test fail!!!\n");
            break;
        }
    }
    rt_kprintf("\n\nSPI test end\n");
}
MSH_CMD_EXPORT(spi_loop_test, test spi1);
```

效果

- `spi_loop_test` 会打印发送缓冲区和接收缓冲区内容，全部一致时输出 `SPI test end`。

短接 P708（MOSI）与 P709（MISO）构建回环的硬件连接位置：



运行 `spi_loop_test` 后的终端效果：

```
 \ | /
- RT -     Thread Operating System
 / | \     5.1.0 build Mar  3 2026 17:52:47
 2006 - 2024 Copyright by RT-Thread team

Hello RT-Thread!
==========================================================
This example project is a Titan Board Mini template routine!
==========================================================
msh >spi_loop_test
spi10 send:
00 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f 10 11 12 13 14 15 16 17 18 19 1a 1b 1c 1d 1e 1f 20 21 22 23 24 25 26 27 28 29 2a 2b 2c 2d 2e 2f 30 31 32 33 34 35 36 37 38 39 3a 3b 3c 3d 3e 3f 40 41 42 43 44 45 46
 47 48 49 4a 4b 4c 4d 4e 4f 50 51 52 53 54 55 56 57 58 59 5a 5b 5c 5d 5e 5f 60 61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 78 79 7a 7b 7c 7d 7e 7f 80 81 82 83 84 85 86 87 88 89 8a 8b 8c 8
d 8e 8f 90 91 92 93 94 95 96 97 98 99 9a 9b 9c 9d 9e 9f a0 a1 a2 a3 a4 a5 a6 a7 a8 a9 aa ab ac ad ae af b0 b1 b2 b3 b4 b5 b6 b7 b8 b9 ba bb bc bd be bf c0 c1 c2 c3 c4 c5 c6 c7 c8 c9 ca cb cc cd ce cf d0 d1 d2 d3
d4 d5 d6 d7 d8 d9 da db dc dd de df e0 e1 e2 e3 e4 e5 e6 e7 e8 e9 ea eb ec ed ee ef f0 f1 f2 f3 f4 f5 f6 f7 f8 f9 fa fb fc fd fe ff 00 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f 10 11 12 13 14 15 16 17 18 19 1a
 1b 1c 1d 1e 1f 20 21 22 23 24 25 26 27 28 29 2a 2b 2c 2d 2e 2f 30 31 32 33 34 35 36 37 38 39 3a 3b 3c 3d 3e 3f 40 41 42 43 44 45 46 47 48 49 4a 4b 4c 4d 4e 4f 50 51 52 53 54 55 56 57 58 59 5a 5b 5c 5d 5e 5f 60 6
1 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 78 79 7a 7b 7c 7d 7e 7f 80 81 82 83 84 85 86 87 88 89 8a 8b 8c 8d 8e 8f 90 91 92 93 94 95 96 97 98 99 9a 9b 9c 9d 9e 9f a0 a1 a2 a3 a4 a5 a6 a7
 a8 a9 aa ab ac ad ae af b0 b1 b2 b3 b4 b5 b6 b7 b8 b9 ba bb bc bd be bf c0 c1 c2 c3 c4 c5 c6 c7 c8 c9 ca cb cc cd ce cf d0 d1 d2 d3 d4 d5 d6 d7 d8 d9 da db dc dd de df e0 e1 e2 e3 e4 e5 e6 e7 e8 e9 ea eb ec ed ee
 ef f0 f1 f2 f3 f4 f5 f6 f7 f8 f9 fa fb fc fd fe ff 00 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f 10 11 12 13 14 15 16 17 18 19 1a 1b 1c 1d 1e 1f 20 21 22 23 24 25 26 27 28 29 2a 2b 2c 2d 2e 2f 30 31 32 33 34 3
5 36 37 38 39 3a 3b 3c 3d 3e 3f 40 41 42 43 44 45 46 47 48 49 4a 4b 4c 4d 4e 4f 50 51 52 53 54 55 56 57 58 59 5a 5b 5c 5d 5e 5f 60 61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 78 79 7a 7b
7c 7d 7e 7f 80 81 82 83 84 85 86 87 88 89 8a 8b 8c 8d 8e 8f 90 91 92 93 94 95 96 97 98 99 9a 9b 9c 9d 9e 9f a0 a1 a2 a3 a4 a5 a6 a7 a8 a9 aa ab ac ad ae af b0 b1 b2 b3 b4 b5 b6 b7 b8 b9 ba bb bc bd be bf c0 c1 c2
 c3 c4 c5 c6 c7 c8 c9 ca cb cc cd ce cf d0 d1 d2 d3 d4 d5 d6 d7 d8 d9 da db dc dd de df e0 e1 e2 e3 e4 e5 e6 e7 e8 e9 ea eb ec ed ee ef f0 f1 f2 f3 f4 f5 f6 f7 f8 f9 fa fb fc fd fe ff 00 01 02 03 04 05 06 07 08 0
9 0a 0b 0c 0d 0e 0f 10 11 12 13 14 15 16 17 18 19 1a 1b 1c 1d 1e 1f 20 21 22 23 24 25 26 27 28 29 2a 2b 2c 2d 2e 2f 30 31 32 33 34 35 36 37 38 39 3a 3b 3c 3d 3e 3f 40 41 42 43 44 45 46 47 48 49 4a 4b 4c 4d 4e 4f
50 51 52 53 54 55 56 57 58 59 5a 5b 5c 5d 5e 5f 60 61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 78 79 7a 7b 7c 7d 7e 7f 80 81 82 83 84 85 86 87 88 89 8a 8b 8c 8d 8e 8f 90 91 92 93 94 95 96
 97 98 99 9a 9b 9c 9d 9e 9f a0 a1 a2 a3 a4 a5 a6 a7 a8 a9 aa ab ac ad ae af b0 b1 b2 b3 b4 b5 b6 b7 b8 b9 ba bb bc bd be bf c0 c1 c2 c3 c4 c5 c6 c7 c8 c9 ca cb cc cd ce cf d0 d1 d2 d3 d4 d5 d6 d7 d8 d9 da db dc d
d de df e0 e1 e2 e3 e4 e5 e6 e7 e8 e9 ea eb ec ed ee ef f0 f1 f2 f3 f4 f5 f6 f7 f8 f9 fa fb fc fd fe ff

spi10 rcv:
00 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f 10 11 12 13 14 15 16 17 18 19 1a 1b 1c 1d 1e 1f 20 21 22 23 24 25 26 27 28 29 2a 2b 2c 2d 2e 2f 30 31 32 33 34 35 36 37 38 39 3a 3b 3c 3d 3e 3f 40 41 42 43 44 45 46
 47 48 49 4a 4b 4c 4d 4e 4f 50 51 52 53 54 55 56 57 58 59 5a 5b 5c 5d 5e 5f 60 61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 78 79 7a 7b 7c 7d 7e 7f 80 81 82 83 84 85 86 87 88 89 8a 8b 8c 8
d 8e 8f 90 91 92 93 94 95 96 97 98 99 9a 9b 9c 9d 9e 9f a0 a1 a2 a3 a4 a5 a6 a7 a8 a9 aa ab ac ad ae af b0 b1 b2 b3 b4 b5 b6 b7 b8 b9 ba bb bc bd be bf c0 c1 c2 c3 c4 c5 c6 c7 c8 c9 ca cb cc cd ce cf d0 d1 d2 d3
d4 d5 d6 d7 d8 d9 da db dc dd de df e0 e1 e2 e3 e4 e5 e6 e7 e8 e9 ea eb ec ed ee ef f0 f1 f2 f3 f4 f5 f6 f7 f8 f9 fa fb fc fd fe ff 00 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f 10 11 12 13 14 15 16 17 18 19 1a
 1b 1c 1d 1e 1f 20 21 22 23 24 25 26 27 28 29 2a 2b 2c 2d 2e 2f 30 31 32 33 34 35 36 37 38 39 3a 3b 3c 3d 3e 3f 40 41 42 43 44 45 46 47 48 49 4a 4b 4c 4d 4e 4f 50 51 52 53 54 55 56 57 58 59 5a 5b 5c 5d 5e 5f 60 6
1 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 78 79 7a 7b 7c 7d 7e 7f 80 81 82 83 84 85 86 87 88 89 8a 8b 8c 8d 8e 8f 90 91 92 93 94 95 96 97 98 99 9a 9b 9c 9d 9e 9f a0 a1 a2 a3 a4 a5 a6 a7
 a8 a9 aa ab ac ad ae af b0 b1 b2 b3 b4 b5 b6 b7 b8 b9 ba bb bc bd be bf c0 c1 c2 c3 c4 c5 c6 c7 c8 c9 ca cb cc cd ce cf d0 d1 d2 d3 d4 d5 d6 d7 d8 d9 da db dc dd de df e0 e1 e2 e3 e4 e5 e6 e7 e8 e9 ea eb ec ed ee
 ef f0 f1 f2 f3 f4 f5 f6 f7 f8 f9 fa fb fc fd fe ff 00 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f 10 11 12 13 14 15 16 17 18 19 1a 1b 1c 1d 1e 1f 20 21 22 23 24 25 26 27 28 29 2a 2b 2c 2d 2e 2f 30 31 32 33 34 3
5 36 37 38 39 3a 3b 3c 3d 3e 3f 40 41 42 43 44 45 46 47 48 49 4a 4b 4c 4d 4e 4f 50 51 52 53 54 55 56 57 58 59 5a 5b 5c 5d 5e 5f 60 61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 78 79 7a 7b
7c 7d 7e 7f 80 81 82 83 84 85 86 87 88 89 8a 8b 8c 8d 8e 8f 90 91 92 93 94 95 96 97 98 99 9a 9b 9c 9d 9e 9f a0 a1 a2 a3 a4 a5 a6 a7 a8 a9 aa ab ac ad ae af b0 b1 b2 b3 b4 b5 b6 b7 b8 b9 ba bb bc bd be bf c0 c1 c2
 c3 c4 c5 c6 c7 c8 c9 ca cb cc cd ce cf d0 d1 d2 d3 d4 d5 d6 d7 d8 d9 da db dc dd de df e0 e1 e2 e3 e4 e5 e6 e7 e8 e9 ea eb ec ed ee ef f0 f1 f2 f3 f4 f5 f6 f7 f8 f9 fa fb fc fd fe ff 00 01 02 03 04 05 06 07 08 0
9 0a 0b 0c 0d 0e 0f 10 11 12 13 14 15 16 17 18 19 1a 1b 1c 1d 1e 1f 20 21 22 23 24 25 26 27 28 29 2a 2b 2c 2d 2e 2f 30 31 32 33 34 35 36 37 38 39 3a 3b 3c 3d 3e 3f 40 41 42 43 44 45 46 47 48 49 4a 4b 4c 4d 4e 4f
50 51 52 53 54 55 56 57 58 59 5a 5b 5c 5d 5e 5f 60 61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 78 79 7a 7b 7c 7d 7e 7f 80 81 82 83 84 85 86 87 88 89 8a 8b 8c 8d 8e 8f 90 91 92 93 94 95 96
 97 98 99 9a 9b 9c 9d 9e 9f a0 a1 a2 a3 a4 a5 a6 a7 a8 a9 aa ab ac ad ae af b0 b1 b2 b3 b4 b5 b6 b7 b8 b9 ba bb bc bd be bf c0 c1 c2 c3 c4 c5 c6 c7 c8 c9 ca cb cc cd ce cf d0 d1 d2 d3 d4 d5 d6 d7 d8 d9 da db dc d
d de df e0 e1 e2 e3 e4 e5 e6 e7 e8 e9 ea eb ec ed ee ef f0 f1 f2 f3 f4 f5 f6 f7 f8 f9 fa fb fc fd fe ff

SPI test end
msh >
```

CANFD 示例

介绍

CANFD 示例基于 **RT-Thread CAN 设备驱动框架** 和 **RA8P1 CANFD 控制器**，使用中断驱动 + 信号量机制实现异步收发，提供 `canfd_test` 命令启动独立收发线程，持续发送和接收 CAN 报文。

硬件参数

参数	规格	说明
CANFD 通道	`canfd0`	RA8P1 内置 2 路独立 CANFD
仲裁波特率	最高 1 Mbps	传统 CAN 兼容
数据波特率	最高 8 Mbps	CANFD 高速段
报文 ID	`0x78`（标准帧）	8 字节数据 `0x00~0x07`
发送缓冲区	16 个 / 通道	—
接收 FIFO	2 个 / 通道	中断 + 信号量通知
过滤器	16 条规则 / 通道	—
错误校验	CRC17 / CRC21	增强型 CRC

使用步骤

1. 输入 `canfd_test` 启动收发线程（启动后每秒发送一帧 ID 为 `0x78` 的 8 字节报文）。
2. 如果对接外部节点，请准备总线回环或对端 CAN 设备，以便观察接收数据。

关键代码

示例源码位于 `src/rt_example_canfd.c`，使用中断接收 + 信号量唤醒接收线程。

接收回调与接收线程

```
static struct rt_semaphore g_can_rx_sem;

static rt_err_t can_rx_indicate(rt_device_t dev, rt_size_t size
{
    rt_sem_release(&g_can_rx_sem);          // 中断里释放信号量唤醒
    return RT_EOK;
}

static void can_rx_thread(void *parameter)
{
    struct rt_can_msg rxmsg = {0};
    rt_device_set_rx_indicate(g_can_dev, can_rx_indicate);

    while (1)
    {
        rxmsg.hdrindex = -1;
        rt_sem_take(&g_can_rx_sem, RT_WAITING_FOREVER);

        if (rt_device_read(g_can_dev, 0, &rxmsg, sizeof(rxmsg))
        {
            rt_kprintf("ID:%x message:", rxmsg.id);
            for (rt_uint8_t i = 0; i < rxmsg.len; i++)
                rt_kprintf("%d ", rxmsg.data[i]);
            rt_kprintf("\n");
        }
    }
}
```

发送线程与初始化

```
static void can_tx_thread(void *parameter)
{
    struct rt_can_msg txmsg = {0};
    txmsg.id  = 0x78;
    txmsg.ide = RT_CAN_STDID;       // 标准帧
    txmsg.rtr = RT_CAN_DTR;         // 数据帧
    txmsg.len = 8;

    while (1)
    {
        for (rt_uint8_t i = 0; i < txmsg.len; i++)
            txmsg.data[i] = i;

        rt_device_write(g_can_dev, 0, &txmsg, sizeof(txmsg));
        rt_thread_mdelay(1000);
```

```
    }
}

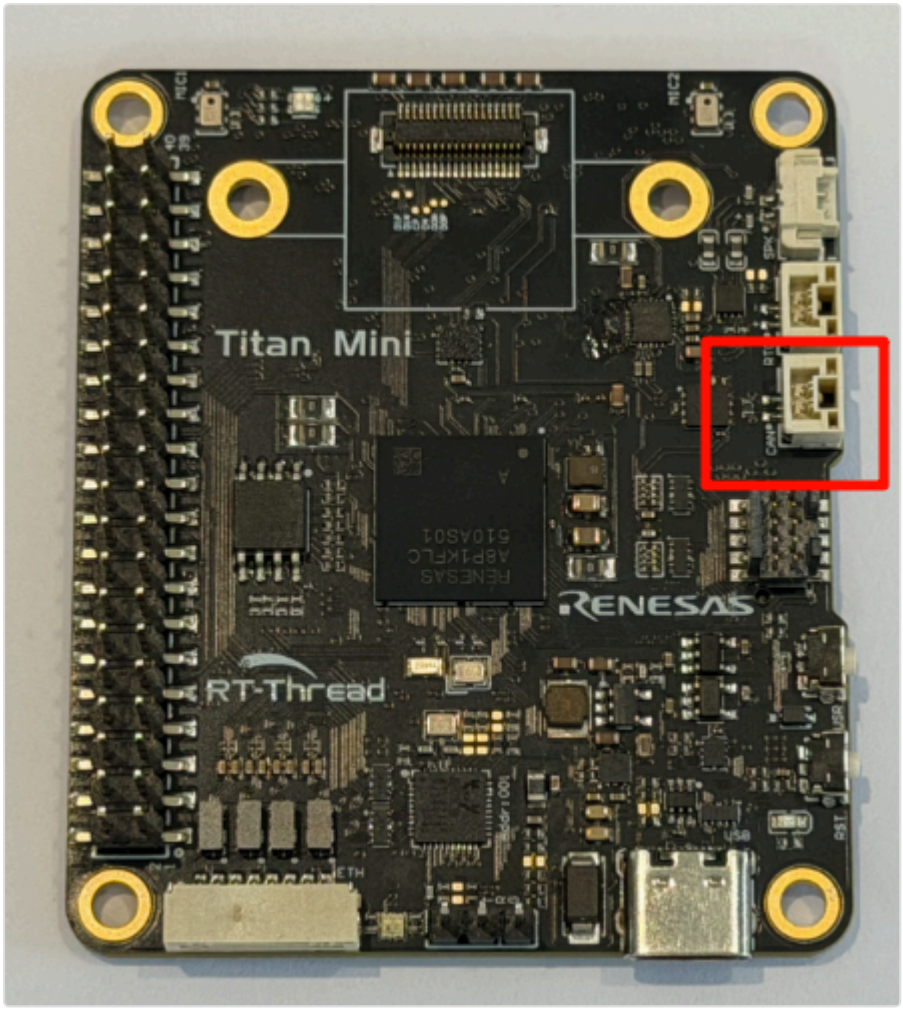
int canfd_test(void)
{
    rt_sem_init(&g_can_rx_sem, "canrx", 0, RT_IPC_FLAG_FIFO);
    g_can_dev = rt_device_find("canfd0");
    rt_device_open(g_can_dev, RT_DEVICE_FLAG_INT_TX | RT_DEVICE

    rt_thread_startup(rt_thread_create("can_rx", can_rx_thread,
    rt_thread_startup(rt_thread_create("can_tx", can_tx_thread,
    return RT_EOK;
}
MSH_CMD_EXPORT(canfd_test, canfd test);
```

效果

- `canfd_test` 启动后会周期性打印 `canfd tx ok`，收到报文时打印 `ID: <id> message:<data...>`。

开发板 CAN 接口位置（接线时注意 CAN_H / CAN_L 不可接反）：



运行 `canfd_test` 后的终端收发效果：

```
msh />canfd_test
cacanfd tx ok
nfd tx ok
arted
msh />canfd tx ok
canfd tx ok
canfd tx ok
canfd tx ok
canfd tx ok
canfd tx ok
canfd tx ok
canfd tx ok
ID:78 message:0 1 2 3 4 5 6 7
canfd tx ok
ID:78 message:0 1 2 3 4 5 6 7
canfd tx ok
ID:78 message:0 1 2 3 4 5 6 7
canfd tx ok
ID:78 message:0 1 2 3 4 5 6 7
canfd tx ok
ID:78 message:0 1 2 3 4 5 6 7
canfd tx ok
```

I2C 示例

介绍

I2C 示例基于 **RT-Thread I2C 设备驱动框架**，通过板载软件 I2C 总线 `i2c1` 读取 **LSM6DS3TR-C** 六轴 IMU 传感器（加速度 + 角速度 + 温度），并打印解析后的物理量与静态姿态角估算结果，是一个完整的 I2C 传感器读出示例。

硬件参数

参数	规格	说明
I2C 总线	`i2c1`	板载软件 I2C（SCL=PIN1292, SDA=PIN1291）
目标设备	LSM6DS3TR-C	板载六轴 IMU
设备地址	`0x6A`	7 位地址，WHO_AM_I=0x6A
加速度满量程	±2g	ODR 12.5Hz，1 LSB ≈ 0.061 mg
陀螺仪满量程	±2000 dps	ODR 12.5Hz，1 LSB ≈ 70 mdps
温度精度	0.0625 °C / LSB	偏移 25 °C

使用步骤

1. 确认板载 LSM6DS3TR-C IMU 已通过 SDA/SCL/GND/VCC 接入 `i2c1`（开发板出厂已连接）。
2. 输入 `i2c_sample` 命令，将自动完成：设备探测 → 寄存器配置 → 单次数据采集 → 解析打印。

关键代码

示例源码位于 `src/rt_example_i2c.c`。核心流程为：先做 WHO_AM_I 校验，再配置 CTRL1_XL/CTRL2_G/CTRL3_C 寄存器，最后读取 STATUS_REG 判

断数据就绪并按小端顺序拼接 6 字节加速度 / 6 字节角速度 / 2 字节温度。

```
#define I2C_BUS_NAME            "i2c1"
#define LSM6DS3TR_C_I2C_ADDR    0x6A
#define LSM6DS3TR_C_WHO_AM_I    0x0F

/* 寄存器读: 先写寄存器地址, 再读 N 字节 (地址自增) */
static rt_err_t imu_reg_read(rt_uint8_t reg, rt_uint8_t *buf, r
{
    struct rt_i2c_msg msgs[2];
    msgs[0].addr  = LSM6DS3TR_C_I2C_ADDR;
    msgs[0].flags = RT_I2C_WR;
    msgs[0].buf   = &reg;
    msgs[0].len   = 1;
    msgs[1].addr  = LSM6DS3TR_C_I2C_ADDR;
    msgs[1].flags = RT_I2C_RD;
    msgs[1].buf   = buf;
    msgs[1].len   = len;
    if (rt_i2c_transfer(i2c_bus, msgs, 2) != 2)
        return -RT_ERROR;
    return RT_EOK;
}

void i2c_sample(void)
{
    if (imu_init() != RT_EOK)              /* WHO_AM_I 校验 + 配
        return;
    imu_read_and_print();                  /* 读 accel/gyro/temp
}
MSH_CMD_EXPORT(i2c_sample, read LSM6DS3TR-C IMU sensor via i2c1
```

效果

成功时会打印一次完整的 IMU 采样，包含加速度（mg）、角速度（dps）、温度（℃）以及由重力方向估算的俯仰角 / 横滚角：

```
[I/i2c.imu] LSM6DS3TR-C detected, WHO_AM_I=0x6A
[I/i2c.imu] IMU configured: ODR=12.5Hz, accel ±2g, gyro ±2000dp
------ LSM6DS3TR-C IMU Sample ------
Accel  (mg) : X=   -8.5  Y=   12.3  Z= 1003.4  | |a|= 1003.5
Gyro   (dps): X=   0.12  Y=   -0.05  Z=   0.08  | |w|=   0.15
Temp   (C)  :   28.50
Tilt   (deg): pitch=  -0.49  roll=   0.70  (static estimate)
-------------------------------------
```

失败时可能提示：

- `cannot find i2c bus i2c1`：软件 I2C 总线未注册（检查 `RT_USING_SOFT_I2C1`）
- `IMU not found, WHO_AM_I=0xXX`：传感器未应答或地址不匹配，检查接线 / 电源

```
Angular rate [mdps]:0.00        0.00    0.00
Temperature [degC]: 29.23
Acceleration [mg]:-5.92 30.26   1039.07
Angular rate [mdps]:0.00        0.00    0.00
Temperature [degC]: 29.16
Acceleration [mg]:-6.04 30.26   1039.44
Angular rate [mdps]:0.00        0.00    0.00
Temperature [degC]: 29.22
Acceleration [mg]:-6.04 30.32   1039.68
Angular rate [mdps]:0.00        0.00    0.00
Temperature [degC]: 29.20
Acceleration [mg]:-6.10 30.26   1039.56
Angular rate [mdps]:0.00        0.00    0.00
Temperature [degC]: 29.18
Acceleration [mg]:-6.22 30.26   1039.44
Angular rate [mdps]:0.00        0.00    70.00
Temperature [degC]: 29.18
Acceleration [mg]:-6.22 30.32   1039.44
Angular rate [mdps]:0.00        -70.00  -70.00
Temperature [degC]: 29.21
Acceleration [mg]:-6.22 30.32   1039.56
Angular rate [mdps]:0.00        0.00    -70.00
Temperature [degC]: 29.19
Acceleration [mg]:-6.22 30.38   1039.74
Angular rate [mdps]:0.00        0.00    0.00
Temperature [degC]: 29.24
```

Watchdog 示例

介绍

Watchdog 示例基于 **RT-Thread 看门狗设备框架** 和 **RA8P1 WDT 硬件**，验证 `wdt` 设备能否正常启动，并通过独立线程模拟"正常喂狗 10 次后停止喂狗"的异常场景，演示看门狗超时复位机制。

硬件参数

参数	规格	说明
WDT 设备	`wdt`	RA8P1 内置独立看门狗
独立运行	是	CPU 停止时仍可计数

参数	规格	说明
超时时间	128/512/.../16384 个时钟周期	可通过 FSP 配置
时钟分频	1/4/16/.../8192 分频	多档可选
复位模式	复位模式 / NMI 模式	复位模式自动重启
喂狗方式	`RT_DEVICE_CTRL_WDT_KEEPALIVE`	软件控制命令
喂狗间隔	1000 ms	由 `WDT_FEED_INTERVAL` 定义

使用步骤

1. 输入 `wdt_sample` 启动看门狗示例。
2. 观察前 10 次喂狗日志（每秒一次）。
3. 第 11 次起停止喂狗，保持终端连接，观察系统复位动作。

关键代码

示例源码位于 `src/rt_example_wdt.c`。

喂狗线程（前 10 次喂狗，之后停止）

```
#define WDT_FEED_INTERVAL   1000

static void feed_dog_entry(void *parameter)
{
    int count = 0;
    while (1)
    {
        if (count < 10)
        {
            rt_device_control(g_wdt_dev, RT_DEVICE_CTRL_WDT_KEE
            rt_kprintf("[FeedDog] Feeding watchdog... %d\n", co
        }
        else
```

```
        {
            rt_kprintf("[FeedDog] Simulate exception! Stop feed
        }
        count++;
        rt_thread_mdelay(WDT_FEED_INTERVAL);
    }
}
```

启动看门狗与喂狗线程

```
int wdt_sample(void)
{
    g_wdt_dev = rt_device_find("wdt");
    if (g_wdt_dev == RT_NULL)
    {
        rt_kprintf("cannot find wdt device!\n");
        return -RT_ERROR;
    }

    if (rt_device_control(g_wdt_dev, RT_DEVICE_CTRL_WDT_START,
    {
        rt_kprintf("start watchdog failed!\n");
        return -RT_ERROR;
    }

    g_feed_thread = rt_thread_create("feed_dog", feed_dog_entry
    rt_thread_startup(g_feed_thread);
    rt_kprintf("watchdog sample started\n");
    return RT_EOK;
}
MSH_CMD_EXPORT(wdt_sample, wdt_sample);
```

效果

- 启动后会打印 `watchdog sample started`。
- 前 10 次每秒打印 `[FeedDog] Feeding watchdog... <n>`。
- 之后会持续打印 `[FeedDog] Simulate exception! Stop feeding.`，若 WDT 已配置为复位模式，系统将在超时后自动复位。

运行 `wdt_sample` 后的喂狗与停止喂狗效果：

```
 \ | /
- RT -     Thread Operating System
 / | \     5.2.0 build Jul 28 2025 10:09:53
 2006 - 2024 Copyright by RT-Thread team

Hello RT-Thread!
================================================
This example project is an driver wdt routine!
================================================
[I/wdt] Tips:
[I/wdt] You can run wdt testcase by executing the instruction: 'wdt_sample'
msh >wdt_sample
[I/wdt] Watchdog started...
[I/wdt] [FeedDog] Feeding watchdog... 0
msh >[I/wdt] [FeedDog] Feeding watchdog... 1
[I/wdt] [FeedDog] Feeding watchdog... 2
[I/wdt] [FeedDog] Feeding watchdog... 3
[I/wdt] [FeedDog] Feeding watchdog... 4
[I/wdt] [FeedDog] Feeding watchdog... 5
[I/wdt] [FeedDog] Feeding watchdog... 6
[I/wdt] [FeedDog] Feeding watchdog... 7
[I/wdt] [FeedDog] Feeding watchdog... 8
[I/wdt] [FeedDog] Feeding watchdog... 9
[E/wdt] [FeedDog] Simulate exception! Stop feeding.
[E/wdt] [FeedDog] Simulate exception! Stop feeding.
```

3.2. Ethernet 示例说明

中文 | **English**

简介

本示例展示了如何在 **Titan Board Mini** 上使用 **以太网接口**，结合 **RT-Thread Ethernet 驱动框架** 实现网络通信功能。

主要功能包括：

- 初始化 RA8 系列以太网硬件
- 配置 IP 地址、子网掩码和网关
- 发送和接收以太网帧
- 集成 RT-Thread `netdev` 框架，实现统一网络设备管理
- 支持 DMA 和中断，实现高速数据传输

以太网（Ethernet）简介

1. 概述

以太网（Ethernet） 是最广泛使用的局域网（LAN）技术，由 Xerox PARC 在 1970 年代提出，后由 IEEE 802.3 标准化。以太网具有以下特点：

- **数据传输方式**：基于 **帧（Frame）** 的分组交换，物理上可通过双绞线、光纤或无线介质传输。
- **拓扑结构**：传统以太网为总线或星型拓扑，现代以太网主要采用星型和树型。
- **协议层次**：属于 OSI 模型的数据链路层（第 2 层）及物理层（第 1 层）技术。

2. 以太网帧结构

以太网使用 **帧（Frame）** 作为数据传输单位。以太网帧由以下字段组成：

字段	长度	描述
前导码（Preamble）	7 字节	用于帧同步
帧开始定界符（SFD）	1 字节	帧起始标志，值为 10101011
目的 MAC 地址	6 字节	接收方硬件地址
源 MAC 地址	6 字节	发送方硬件地址
类型/长度	2 字节	上层协议类型或帧长度
数据载荷	46~1500 字节	上层数据（如 IP 包）
CRC 校验	4 字节	循环冗余校验，保证帧完整性

> ***最小帧长度：****64 字节*
> ***最大帧长度：****1518 字节（不含 VLAN 标记时）*

RA8 系列以太网特性

RA8 系列 MCU（如 RA8P1）集成高性能 **以太网 MAC**，支持 RGMII、RMII 和 MII 接口，具备稳定可靠的高速网络通信能力。MAC 可配合外部 PHY 使用，并兼容 LwIP TCP/IP 协议栈。

1. 网络接口特性

1. **接口类型**

- **RMII（Reduced Media Independent Interface）**：节省引脚，支持 10/100 Mbps
- **MII（Media Independent Interface）**：标准接口，支持 10/100 Mbps
- **RGMII（Reduced Gigabit MII）**：支持 10/100/1000 Mbps，高速接口，适用于千兆以太网

2. **PHY 连接**

- 外部 PHY 通过 MDC/MDIO 接口连接

- 支持自动协商速率和双工模式
- 可读取和写入 PHY 寄存器，用于配置和状态监控

2. MAC 特性

1. **双工与速率支持**

- 全/半双工
- 支持 10/100/1000 Mbps（RGMII）
- 支持自动协商或强制配置

2. **帧处理**

- VLAN 标签支持（IEEE 802.1Q，可选）
- 支持多播和广播帧过滤
- 硬件 CRC 生成和校验

3. **MAC 地址管理**

- 支持单个或多个 MAC 地址
- 可通过 FSP 或软件动态配置

3. DMA 与缓冲特性

1. **独立 TX/RX DMA 引擎**

- 支持同时发送和接收数据
- 减少 CPU 占用，提高吞吐量

2. **描述符队列**

- TX/RX 缓冲区数量可配置
- 支持链式 DMA，提高大数据传输效率

3. **多缓冲区管理**

- 支持环形缓冲区，实现连续传输
- 减少丢包

4. **硬件加速**

- 帧过滤、长度检测和 CRC 校验

4. 中断机制

1. **中断类型**

- 接收完成（RX）
- 发送完成（TX）
- 错误中断（CRC 错误、缓冲溢出）

2. **中断配置**

- 优先级可通过 FSP 配置
- 支持 RT-Thread ISR 集成

3. **优化**

- RX/TX 中断可与 DMA 联动
- 可选择性启用中断，提高性能

5. PHY 管理

1. **MDIO 接口**

- 可读写 PHY 寄存器，实现配置、复位和状态监控

2. **自动协商**

- 支持速率（10/100/1000 Mbps）和双工模式自动协商

3. **链路监控**

- 检测链路状态（Up/Down）
- CRC 错误和冲突检测

6. 协议与栈支持

1. **TCP/IP 协议栈集成**

- 兼容 LwIP
- 支持 TCP/UDP/ICMP、DHCP 客户端/服务器、ARP

2. **应用层支持**

- 支持 Telnet、HTTP、MQTT 等应用
- 多线程安全，支持并发访问

7. 性能与可靠性

1. **吞吐量优化**

- DMA + 中断减少 CPU 占用
- TX/RX 缓冲区大小可调

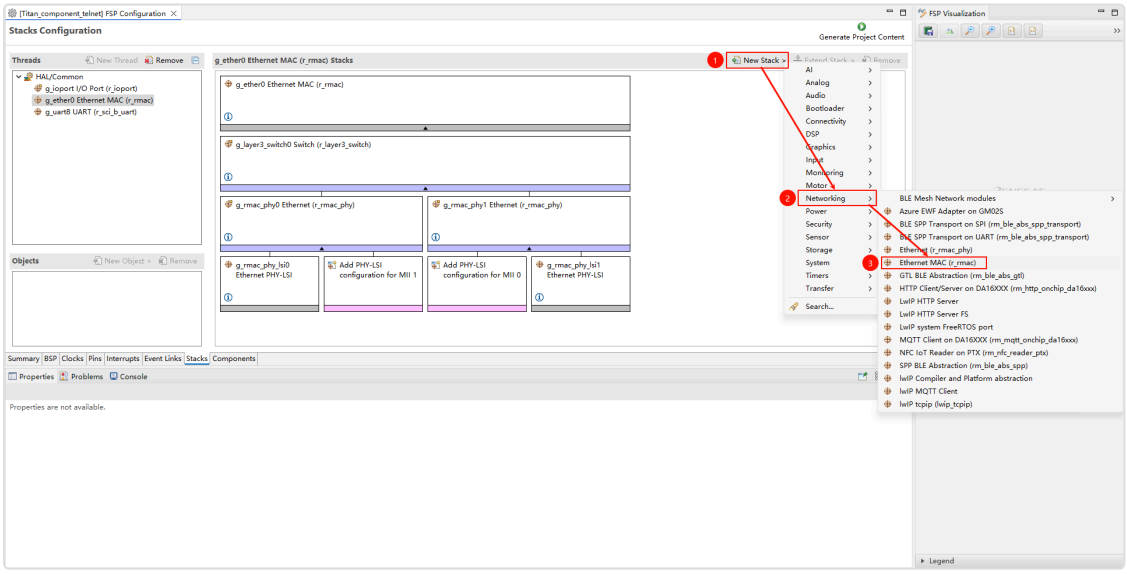
2. **可靠性特性**

- 硬件 CRC 校验
- VLAN 和多播过滤减少干扰
- 链路检测与自动重连

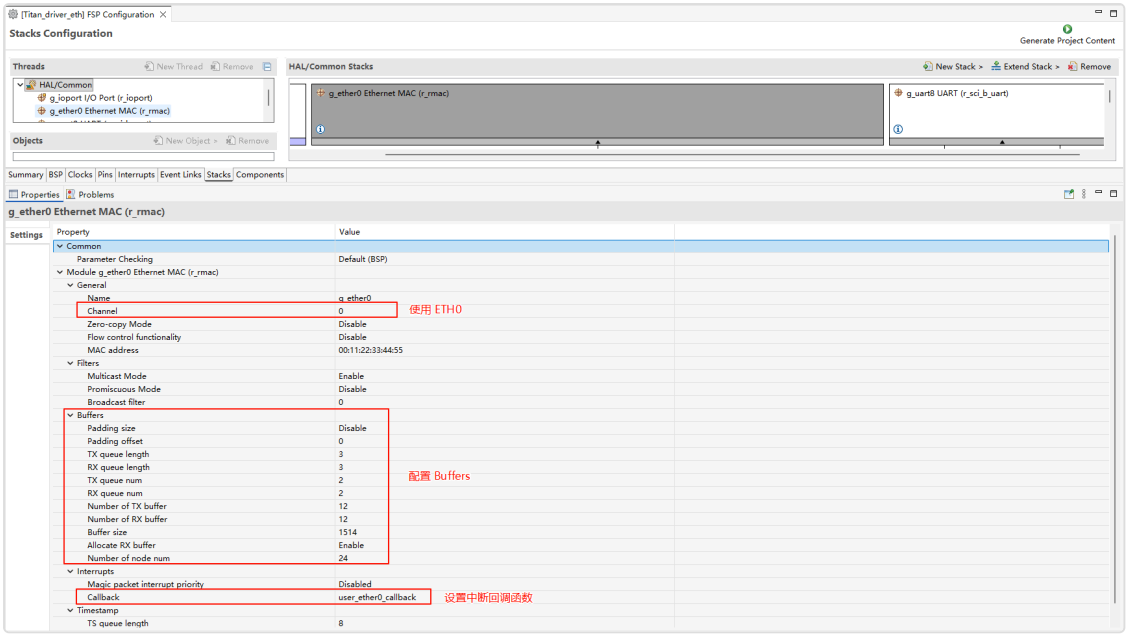
FSP 配置

> ***注意:*** *本工程所使用的 FSP 版本为 6.4.0，在使用 FSP 配置功能时请使用 FSP 6.4.0。*

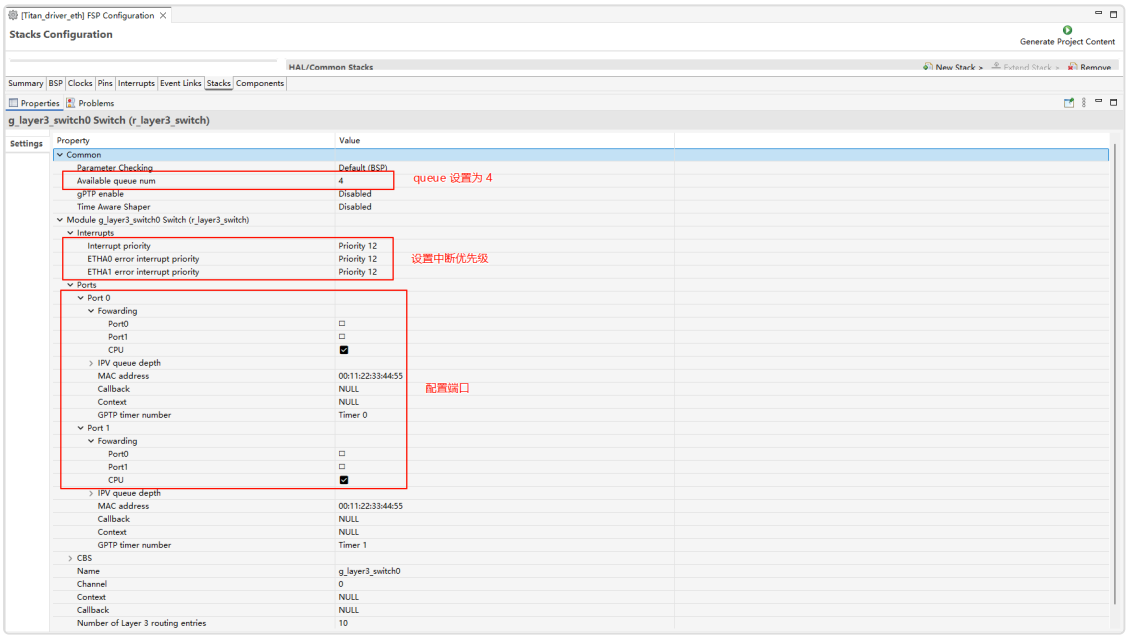
- 新建 r_rmac stack：



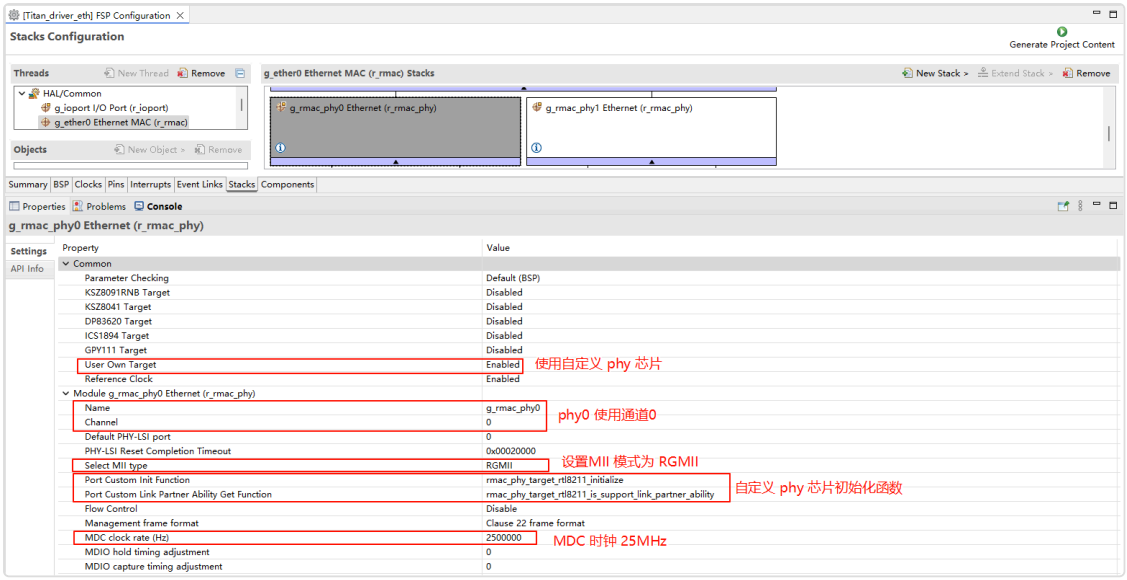
- 配置 r_mac stack：



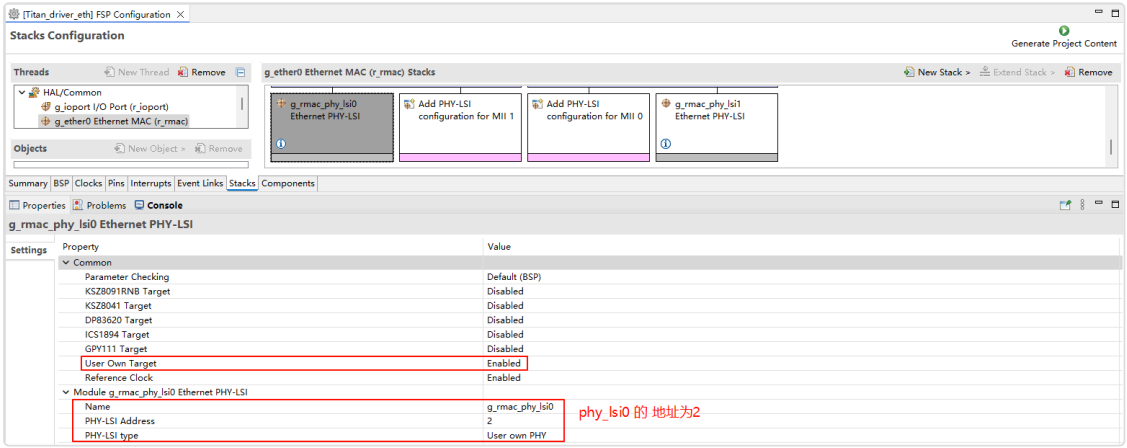
- 配置 r_layer3_switch：



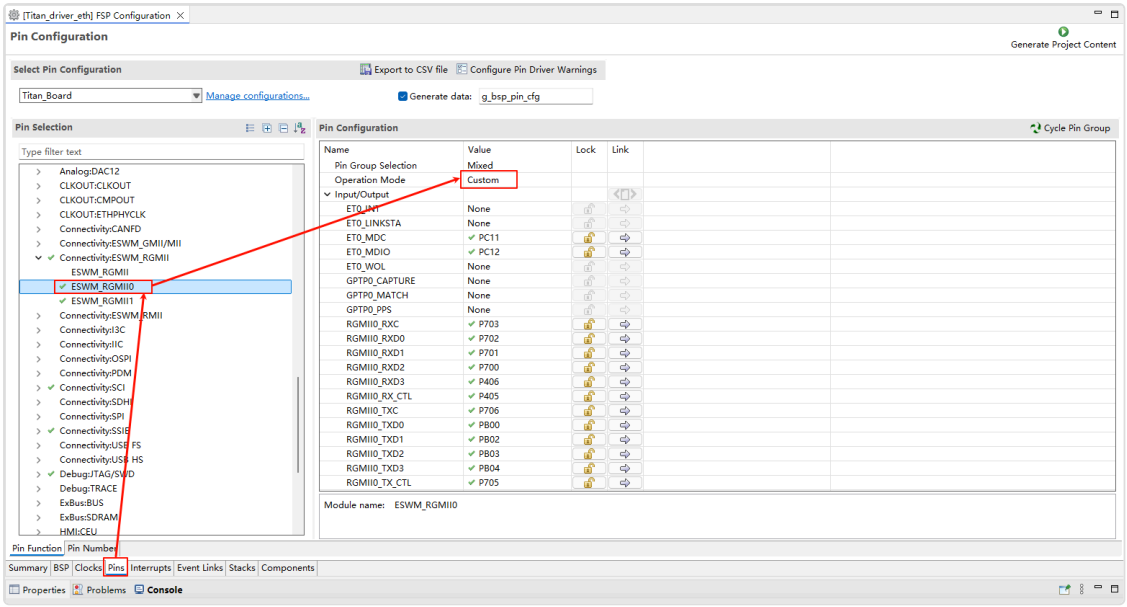
- 配置 r_rmac_phy：



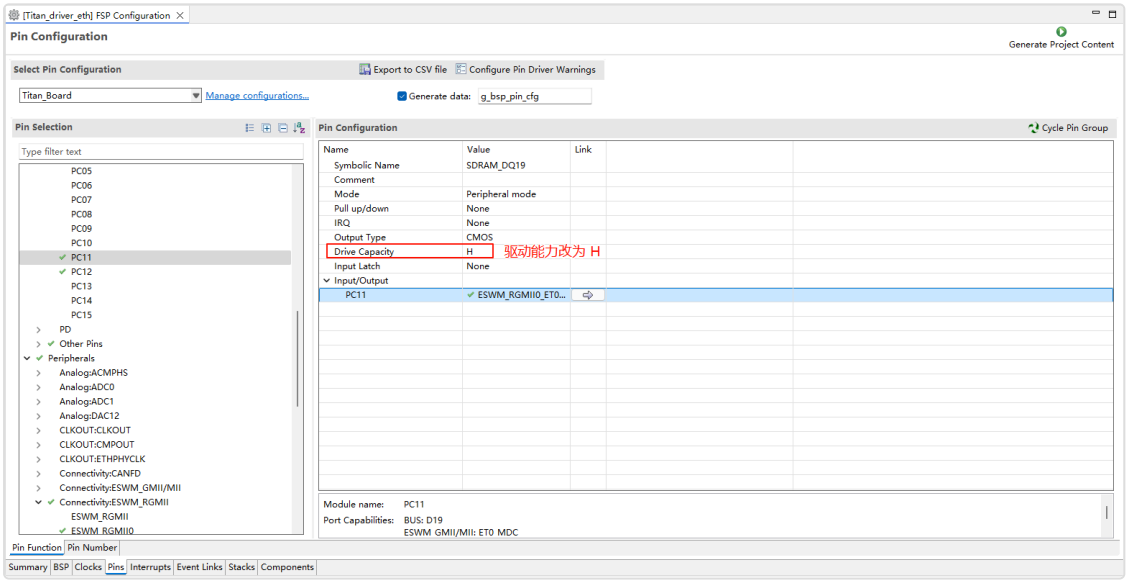
- 配置 g_rmac_phy_lsi0：



- ETH0 引脚配置：

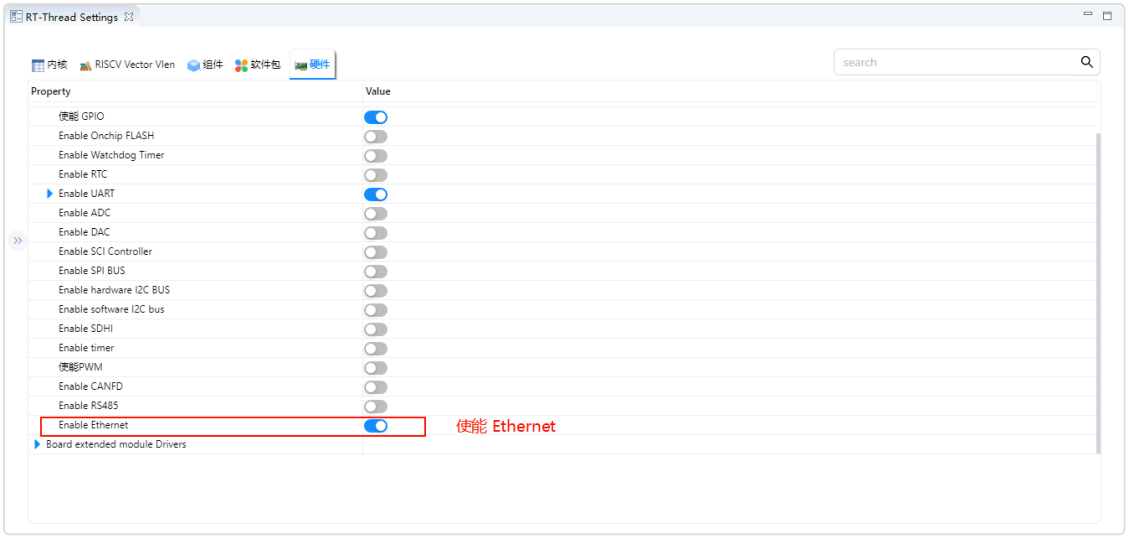


- **注意**：ETH 相关的所有引脚需要将驱动能力改为 H。



RT-Thread Settings 配置

- 在 RT-Thread Settings 中使能 Ethernet。



软件说明

LwIP 性能调优建议

RT-Thread 上游 `rt-thread/components/net/lwip/Kconfig` 中的 LwIP 默认参数偏保守（PBUF 数量 16、TCP 发送缓冲 8196 等），无法充分发挥 RA8P1 千兆以太网 + DMA 的吞吐能力。建议在启用 `RT_USING_LWIP`（例如勾选 `BSP_USING_ETH`）后，按下列推荐值手动调优：

参数	默认值	说明
`RT_LWIP_PBUF_NUM`	256	PBUF 缓冲池数量（上游默认 16）
`RT_LWIP_RAW_PCB_NUM`	4	RAW socket 数量
`RT_LWIP_UDP_PCB_NUM`	24	UDP PCB 数量（上游默认 4）
`RT_LWIP_TCP_PCB_NUM`	24	TCP PCB 数量（上游默认 4）
`RT_LWIP_TCP_SEG_NUM`	512	TCP 段数量（上游默认 40）
`RT_LWIP_TCP_SND_BUF`	65535	TCP 发送缓冲（上游默认 8196）
`RT_LWIP_TCP_WND`	65535	TCP 窗口大小（上游默认 8196）

参数	默认值	说明
`RT_LWIP_TCPTHREAD_PRIORITY`	6	lwIP 主线程优先级（上游默认 10，数值越小优先级越高）
`RT_LWIP_TCPTHREAD_MBOX_SIZE`	144	lwIP 主线程邮箱大小（上游默认 8）
`RT_LWIP_TCPTHREAD_STACKSIZE`	2048	lwIP 主线程栈（上游默认 1024）
`LWIP_NO_TX_THREAD`	y	不使用独立 TX 线程（发送路径直接调用，减少上下文切换）
`RT_LWIP_ETHTHREAD_PRIORITY`	5	以太网线程优先级（上游默认 12）
`RT_LWIP_ETHTHREAD_STACKSIZE`	2048	以太网线程栈（上游默认 1024）
`RT_LWIP_ETHTHREAD_MBOX_SIZE`	144	以太网线程邮箱大小（上游默认 8）

如需应用上述推荐值，可在 RT-Thread Studio 的 `RT-Thread Settings → RT-Thread Components → Network → lwIP` 中修改对应项，或直接编辑工程的 `.config`。

> ⚠️ ***内存占用提示：*** *上述参数对 RAM 占用影响较大。`RT_LWIP_PBUF_NUM=256` + `TCP_SND_BUF/WND=65535` 在默认 `RT_LWIP_PBUF_POOL_BUFSIZE` 下会预留较多堆内存，请确认链接脚本中 heap 大小足够（建议 ≥ 256 KB）。*

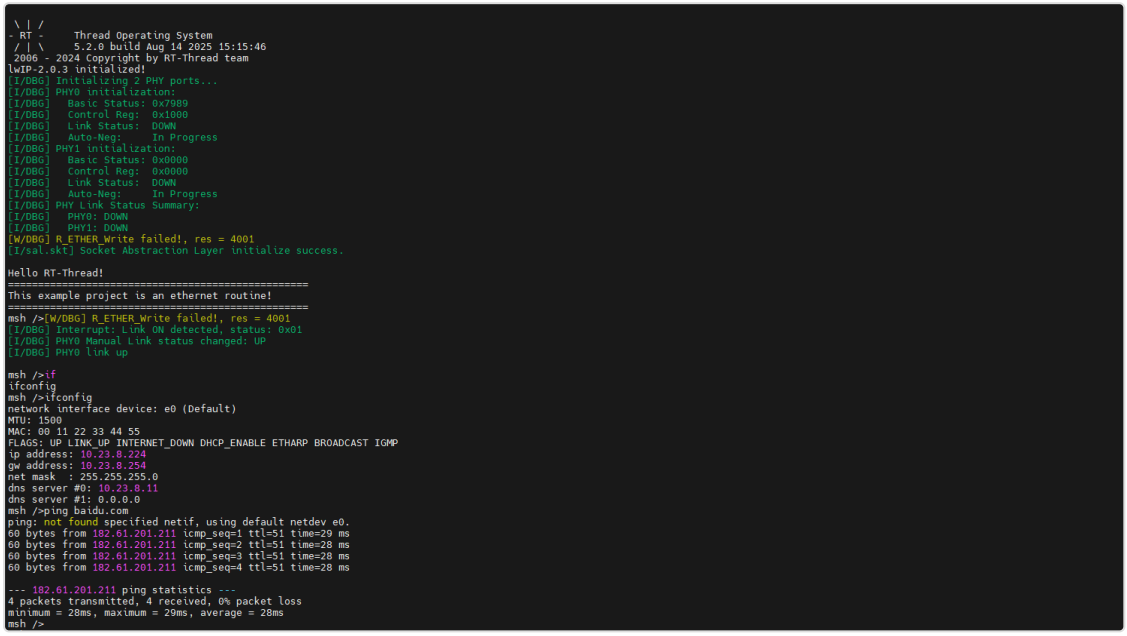
编译&下载

- RT-Thread Studio：在RT-Thread Studio 的包管理器中下载 Titan Board 资源包，然后创建新工程，执行编译。

编译完成后，将开发板的 USB-DBG 接口与 PC 机连接，然后将固件下载至开发板。

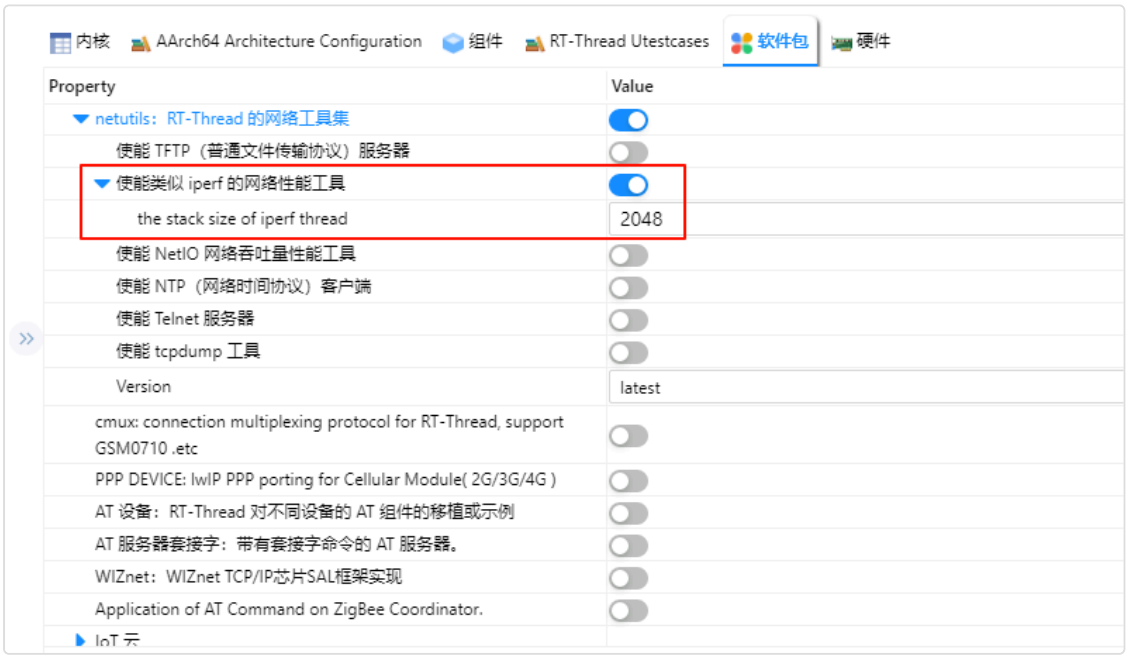
运行效果

将网线插入 网口，按下复位按键重启开发板，等待 PHY0 link up 之后输入 `ifconfig` 可以查看开发板获取到的 IP 地址，然后输入 `ping baidu.com` 命令进行连通性测试。

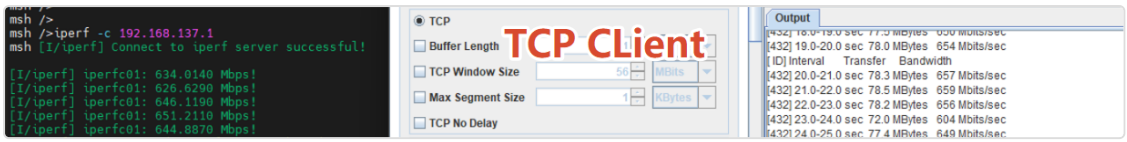


iperf 测试

打开 RT-Thread Settings 添加 netutils 软件包并使能 iperf 工具。



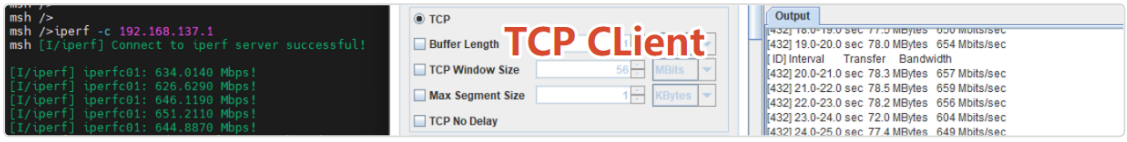
编译下载后在串口终端输入 `iperf -c 主机IP -p 5001` 进行 iperf 测试。



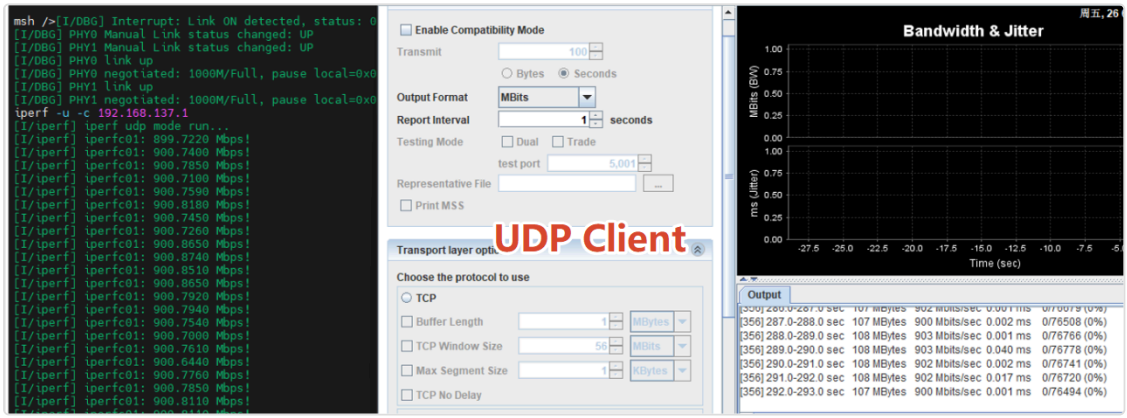
网络应用示例

借助 netutils 软件包，可以实现 TCP/UDP 的客户端与服务器收发测试。

TCP Client 测试：开发板作为客户端，主动连接远程主机并收发数据。



UDP Client 测试：开发板作为客户端，向远程主机发送或接收数据报。



4. 多媒体显示篇

4.1. MIPI CSI 摄像头示例说明

中文|English

简介

本示例展示了如何在 **Titan Board** 上使用 **MIPI CSI（Camera Serial Interface）** 接口连接 **OV5640 摄像头**，并通过 **RT-Thread LCD 框架**将捕获的图像显示到 **RGB565 LCD 屏幕**。

主要功能包括：

- 初始化 MIPI CSI 摄像头接口，捕获实时视频流
- 配置 OV5640 摄像头参数（分辨率、帧率、输出格式）
- 使用 RT-Thread LCD 驱动显示摄像头捕获的图像
- 支持图像格式转换（YUV422 → RGB565）

RA8 系列 MIPI CSI 特性

RA8 系列 MCU 集成 **MIPI CSI 硬件模块**，用于高速、低功耗摄像头数据采集，适合高清视频和实时图像处理。

1. 硬件接口特性

1. **接口类型**

- **MIPI CSI-2** D-PHY 接口，支持高带宽串行传输
- 支持 1–4 数据通道 (Lanes)
- 同步信号由 MIPI D-PHY 控制，无需额外 HSYNC/VSYNC

2. **数据速率与分辨率**

- 支持每通道最高 1.5–2.5 Gbps（取决 MCU 系列和 PHY 配置）

- 可驱动常见摄像头分辨率：VGA、QVGA、SXGA、UXGA、1080p 等

3. **摄像头兼容性**

- 兼容 OV5640、IMX219 等常见 CMOS 摄像头
- 支持摄像头寄存器配置和自动初始化

2. 图像格式与处理能力

1. **图像格式支持**

- **RAW10 / RAW12**（用于算法开发和图像处理）
- **YUV422**（用于视频显示）
- **RGB565**（适合 LCD 显示）

2. **图像处理功能**

- **颜色空间转换**（YUV → RGB）
- **图像裁剪**（ROI capture）
- **镜像与翻转**
- **硬件缩放加速**

3. **硬件加速**

- MIPI CSI 内置 DMA，可减轻 CPU 负担
- 高速格式转换与缩放

3. DMA 支持与缓冲机制

1. **高速 DMA 传输**

- 与 MCU DMAC 协作，将图像直接写入内存或 LCD 缓冲区
- 减少 CPU 干预，提高帧率

2. **多缓冲区机制**

- 支持双缓冲或环形缓冲，实现连续视频采集
- 避免帧丢失和显示延迟

3. **DMA 配置灵活**

- 可配置缓冲区起始地址和大小
- 支持中断回调处理

4. 中断机制

1. **中断类型**

- **帧完成中断**：每帧采集完成时触发
- **行完成中断（可选）**
- **错误中断**：缓冲区溢出、PHY 错误等

2. **中断特点**

- 支持 RT-Thread ISR 回调
- 可与 DMA 联动，实现实时显示

5. 时序与同步特性

1. **同步方式**

- CSI 使用 MIPI D-PHY 提供时钟和数据同步
- 无需额外 HSYNC/VSYNC

2. **像素时钟与数据对齐**

- 支持按像素对齐或按字节对齐
- 自动处理 RAW/YUV/RGB 数据对齐

6. 性能优化

1. **高吞吐量**

- DMA + 双缓冲实现无间断视频采集
- CPU 占用低，适合实时应用

2. **可靠性**

- PHY 错误、数据丢失中断可触发异常处理
- 支持缓冲区溢出检测

3. **灵活性**

- 多分辨率、多格式支持
- 支持 ROI 捕获和硬件缩放，提高显示效率

7. 应用场景

- 实时视频显示到 LCD
- 视频采集与处理算法验证
- 嵌入式视觉应用，如监控、手势识别、机器人视觉

RA8 系列 MCU GLCDC（Graphics LCD Controller）特性

RA8 系列 MCU（如 RA8P1）集成 **GLCDC 硬件模块**，用于驱动 TFT/LCD 显示屏，实现高速图形渲染和视频显示，支持多种分辨率、色彩格式和显示模式。

1. 硬件特性

1. **分辨率支持**

- 可驱动从 QVGA (320×240) 到 WQVGA/XGA 等常用分辨率
- 受片上 RAM 和显示接口带宽限制

2. **色彩支持**

- 支持 1/4/8/16/24/32 位色深
- 常用 RGB565、RGB888 格式
- 支持调色板模式（Palette mode）
- 可硬件进行色彩转换

3. **接口类型**

- 并行 RGB（TFT LCD 接口）
- 支持 16/18/24 位数据总线
- 可与外部 LCD 面板直接连接
- 时序可编程：HSYNC、VSYNC、DE、PCLK、RGB 输出

2. 图层与显示模式

1. **图层支持**

- 单图层模式（单画面显示）
- 多图层模式（通过调色板或硬件 alpha 混合叠加）

- 支持透明/半透明叠加

2. **显示模式**

- RGB 模式（直显颜色输出）
- CLUT/Palette 模式（索引颜色，通过查表转换）
- 可配置扫描方向（水平/垂直扫描）

3. DMA 与帧缓冲

1. **帧缓冲访问**

- GLCDC 可直接访问片上/外部 SRAM 中的帧缓冲
- 支持单缓冲、双缓冲模式
- 支持环形缓冲，用于连续刷新

2. **DMA 支持**

- 与 MCU DMAC 协同工作，减少 CPU 占用
- 可将图像直接从内存传输到 LCD
- 支持行、块或全帧传输

4. 硬件图形功能

1. **窗口裁剪与缩放**

- 可指定显示窗口区域
- 可支持简单缩放（水平/垂直）

2. **硬件图形加速**

- 支持矩形填充、颜色替换
- 支持图像透明处理
- 可结合 CEU 或 DMA 进行视频显示

3. **颜色格式转换**

- YCbCr → RGB
- RGB888 → RGB565
- 硬件加速，减少 CPU 负担

5. 中断机制

1. **中断类型**

- 帧完成中断（Frame End）
- 行中断（Line End，可选）
- 访问错误/溢出中断

2. **中断应用**

- 与 RT-Thread ISR 集成
- 可在帧完成时触发更新缓冲区或切换双缓冲
- 方便实现动画和视频显示

6. 性能优化

1. **双缓冲机制**

- 支持双缓冲减少闪烁
- CPU 可在后台绘制下一帧
- GLCDC 硬件自动切换显示缓冲

2. **帧率控制**

- 可编程时钟和行/帧同步
- 支持 30fps、60fps 等常见刷新率

3. **CPU 卸载**

- 大量图形操作由硬件完成
- DMA + GLCDC 结合，实现高效图像显示

RT-Thread Settings 配置

- 使能 MIPI CSI 摄像头，使用 i2c0；使能 RGB565 LCD，使用 pwm7 输出背光。



编译&下载

- RT-Thread Studio：在RT-Thread Studio 的包管理器中下载 Titan Board 资源包，然后创建新工程，执行编译。

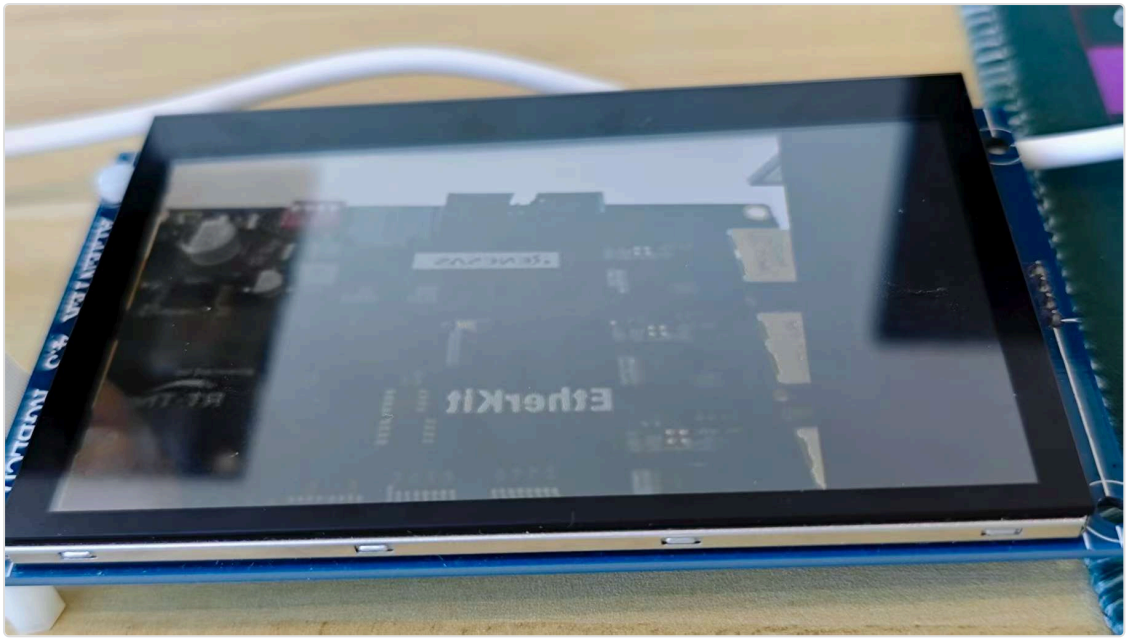
编译完成后，将开发板的 USB-DBG 接口与PC 机连接，然后将固件下载至开发板。

运行效果

复位 Titan Board 后终端会输出如下信息：

```
 \ | /
- RT -     Thread Operating System
 / | \     5.2.0 build Aug 15 2025 15:02:50
 2006 - 2024 Copyright by RT-Thread team
[D/drv_lcd]
lcd framebuffer address:0x22020d40
msh >
Hello Titan Board!
================================================
This example project is an mipi-csi camera display routine!
================================================
[I/camera.i2c] I2C bus initialized: i2c0
The vin driver uses hardware mailboxes for the buffer.
```

下面是 LCD 屏幕上显示的图像：



通过按下usr按键可以实现对画面的对焦

4.2. LVGL 示例说明

中文 | **English**

简介

本示例基于 **LVGL** 图形库，构建了一个适用于嵌入式设备的图形用户界面（GUI）示例项目，结合实际的显示驱动和输入驱动，展示了从底层驱动适配到上层界面构建的完整流程。通过该示例，开发者不仅可以学习如何在嵌入式环境中移植和配置 LVGL，还能掌握界面布局、事件响应、主题定制等关键技术点，从而为后续构建实际产品界面打下基础。

LVGL 简介

LVGL（Light and Versatile Graphics Library，轻量且多功能的图形库）是一个开源的嵌入式 GUI（图形用户界面）开发框架。它的设计初衷是为资源受限的嵌入式设备提供流畅、现代化的图形界面，因此在运行效率、内存占用和可移植性方面都做了大量优化。无论是在简单的低端 MCU 上，还是在功能更强大的 MPU 平台上，LVGL 都能够高效运行，并提供丰富的图形控件和交互体验。

主要特点

1. **轻量级**
   LVGL 最大的优势在于轻量化，它对资源的需求非常低。在一个内存只有几十 KB 的微控制器上，LVGL 依然可以流畅运行。框架本身的内存开销较小，渲染算法经过精心优化，既可以保证较低的 CPU 使用率，又能在有限的硬件条件下呈现出较好的视觉效果。这使得 LVGL 特别适合那些对硬件资源敏感的应用场景，比如低功耗设备、可穿戴设备、家用电器控制面板等。

2. **跨平台**
   LVGL 的跨平台特性非常突出，它能够运行在多种操作系统之上，包括 FreeRTOS、RT-Thread、Zephyr、Linux 等，也可以直接运行在裸机环境中。开发者只需要为 LVGL 提供底层的显示驱动和输入驱动接口，便可以快速完成移植，从而在不同的硬件平台之间复用同一套 UI 代码。这种灵活性大大降低了开发成本，使得 LVGL 成为嵌入式 GUI 的通用解决方案。

3. **丰富的控件（Widgets）**
   LVGL 内置了种类丰富的 GUI 控件，例如按钮、标签、滑条、进度条、复选

框、列表、表格、图表等。这些控件几乎涵盖了常见人机交互界面的需求，开发者无需从零开始设计和实现组件，大大缩短了开发周期。同时，LVGL 还允许用户基于现有控件扩展新的组件，从而构建符合特定需求的界面。无论是简单的数字显示，还是复杂的图形控制界面，LVGL 都能胜任。

4. **多样的渲染能力**
   在视觉呈现方面，LVGL 提供了丰富的渲染功能。它支持抗锯齿、透明度、渐变、阴影、边框、圆角等效果，可以让界面看起来更加美观和现代。除此之外，LVGL 还内置动画系统，支持多种缓动函数，能够实现平滑的控件移动、渐变过渡和动态效果。这些特性使得界面不仅功能完善，还能带来良好的用户体验。

5. **输入设备支持**
   LVGL 支持多种输入设备类型，包括触摸屏、电容屏、鼠标、键盘和编码器，甚至可以实现多点触控交互。它提供了统一的输入接口层，开发者只需实现底层的驱动适配，就能轻松把输入事件传递到 LVGL 的事件系统中。这样一来，开发者可以更加专注于上层的界面逻辑，而不必花费过多时间在输入事件处理上。

6. **国际化与多语言**
   LVGL 在国际化方面也有很好的支持。它使用 UTF-8 编码，可以处理几乎所有语言的字符集。同时，它支持双向文本渲染，可以正确显示如阿拉伯语、希伯来语等从右到左书写的语言。这使得 LVGL 能够应用在面向全球用户的产品中，不同国家和地区的用户都能通过本地化界面获得良好的体验。

7. **可扩展性**
   LVGL 提供了灵活的主题和样式系统，开发者可以方便地定制控件的外观样式，以实现不同风格的 UI。通过更换主题，可以快速切换界面的整体视觉风格。此外，LVGL 还可以与第三方图形库、文件系统和图像解码器结合使用，从而扩展其功能。例如，可以使用文件系统来加载外部字体和图片，或者集成 JPEG/PNG 解码库来显示复杂图像。

应用场景

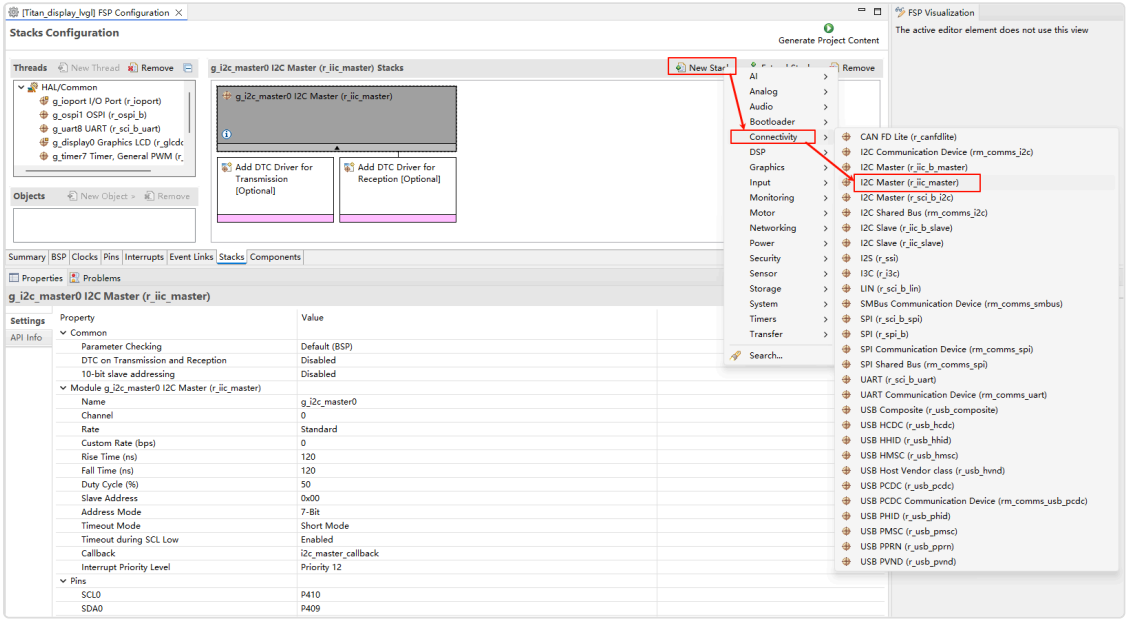
LVGL 在实际应用中有着非常广泛的覆盖范围。在消费电子领域，它常用于智能家居控制面板、家电显示屏、智能手表和健身设备等，这些产品往往需要在有限硬件资源下提供友好的用户交互界面。在工业控制领域，LVGL 被用于 HMI（人机交互界面）和各种仪器仪表，帮助用户直观地监控和操作设备。在汽车电子中，LVGL 可以驱动中控屏、副驾娱乐屏，甚至车载仪表盘。而在医疗设备方面，它则适合做小型显示界面，例如手持检测仪、便携式监护设备等。

生态与社区

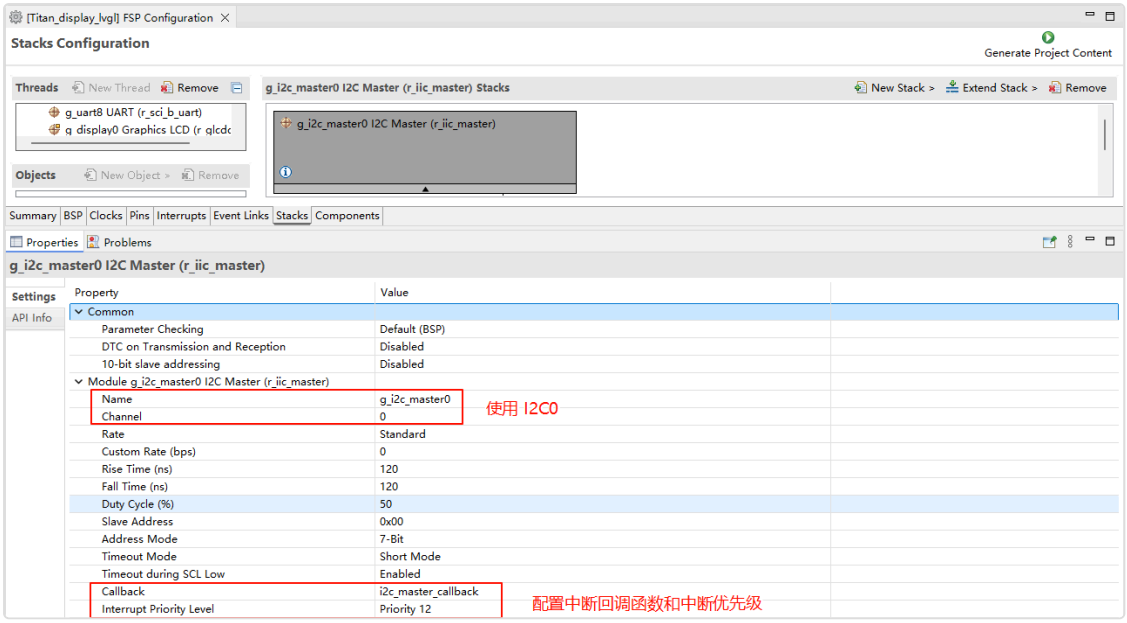
LVGL 作为一个开源项目，采用 **MIT License** 开源协议，既适合个人学习，也可以在商业产品中免费使用。围绕 LVGL，已经形成了一个活跃的生态系统。官方提供了 **SquareLine Studio** 这类可视化设计工具，支持拖拽式设计界面并导出 LVGL 代码，极大地提升了开发效率。此外，**LVGL Simulator** 可以让开发者直接在 PC 上进行界面调试，不需要每次都烧录到目标硬件上。全球开发者社区十分活跃，贡献了大量开源控件、主题和移植案例，为新手和企业开发者提供了丰富的资源和支持。

I2C0 配置

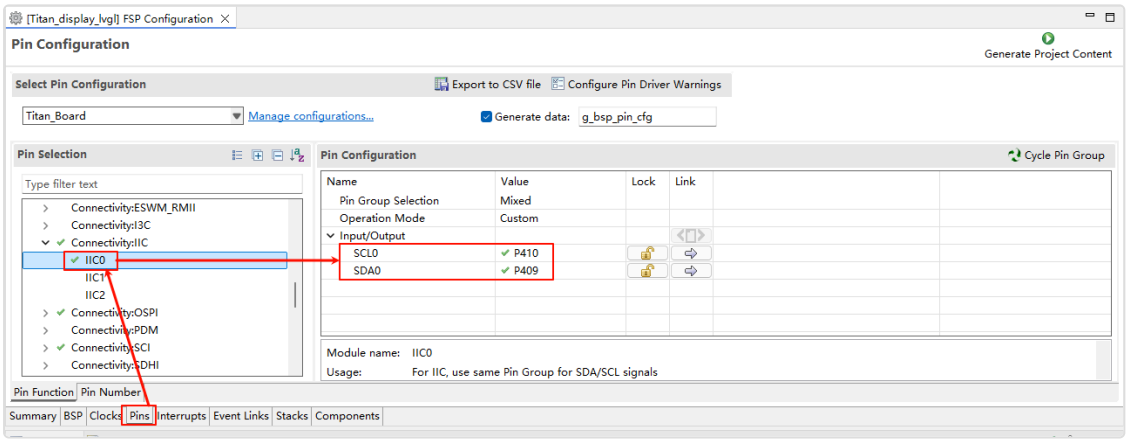
- 新建 `r_iic_master` stack：



- 配置 `r_iic_master` stack：

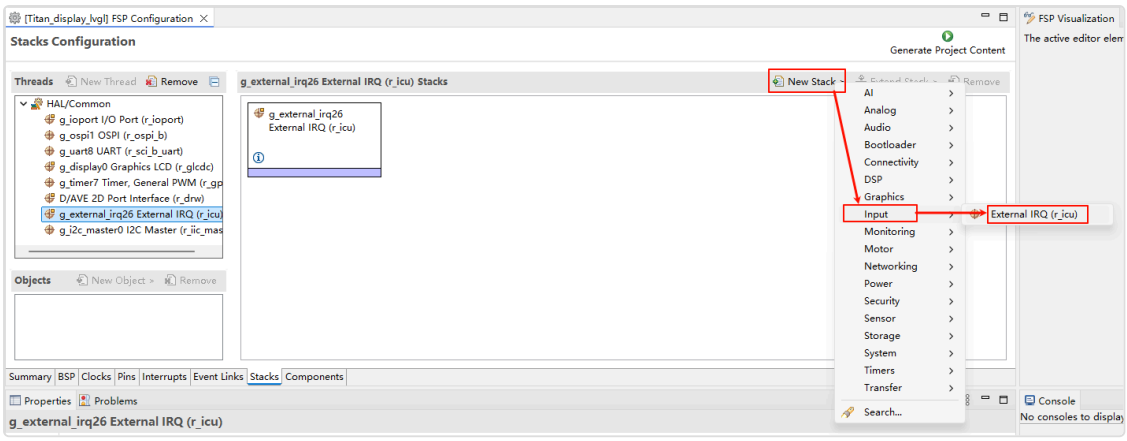


- 配置 I2C0 引脚：

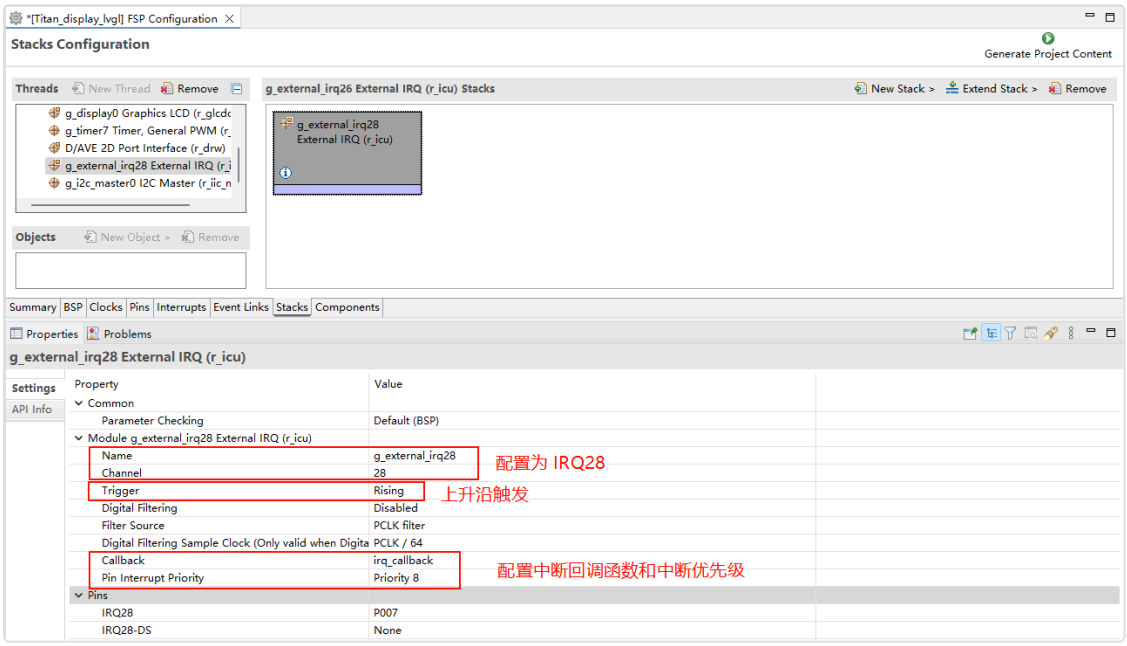


触摸中断配置

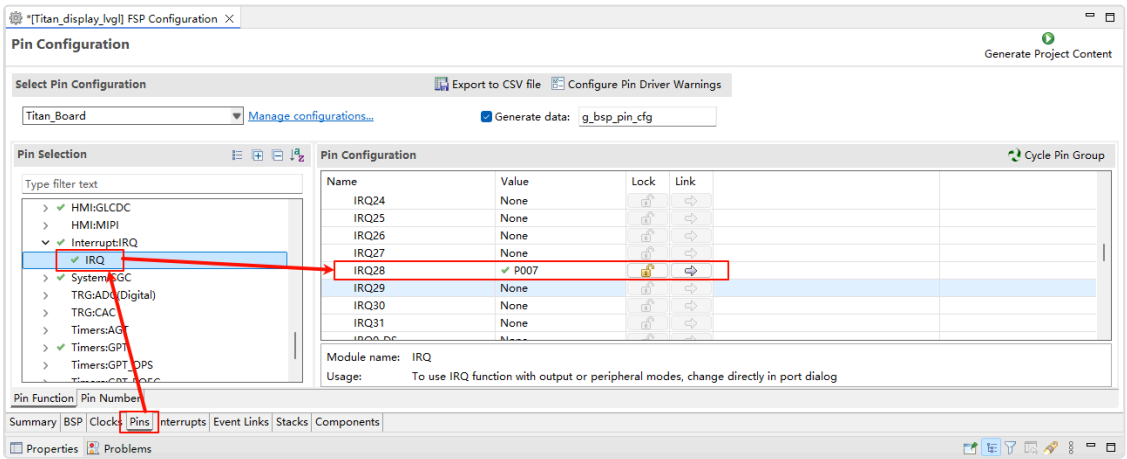
- 新建 `r_icu` stack：



- 配置 IRQ28：

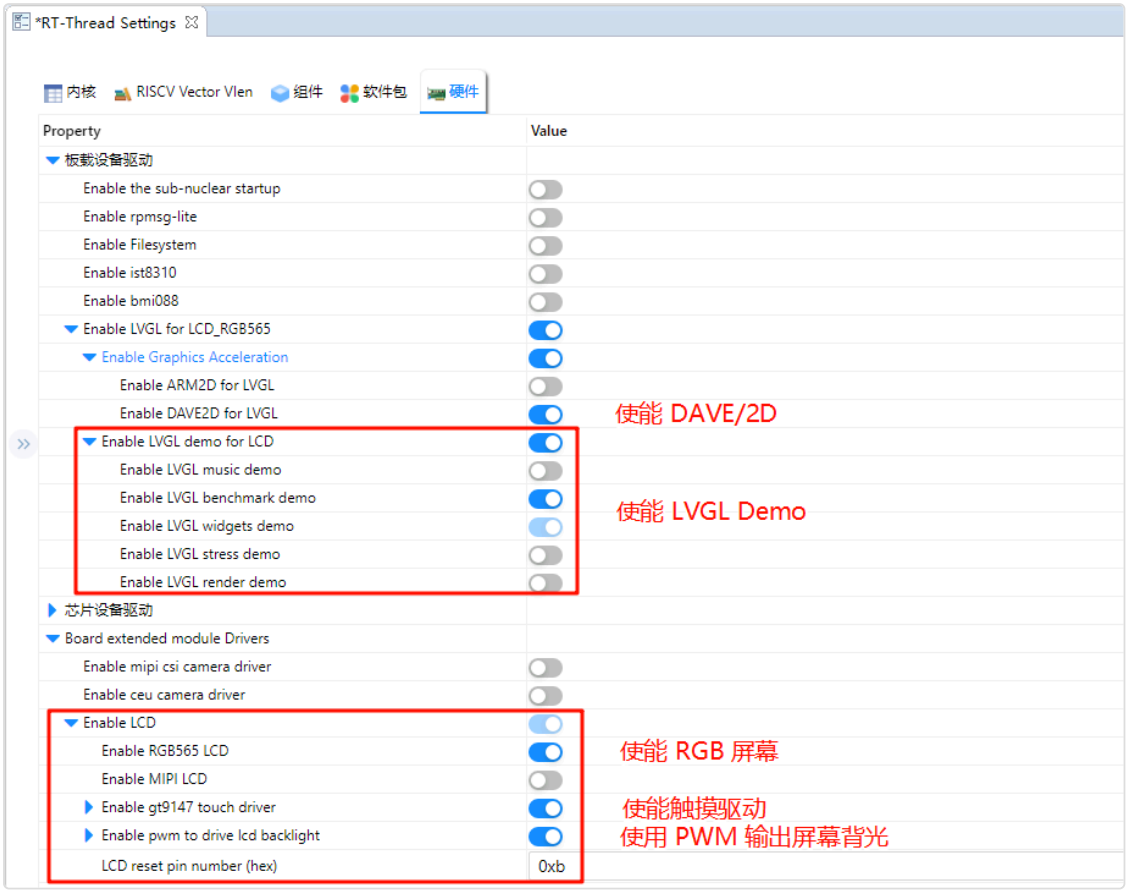


- 配置 IRQ28 引脚：



RT-Thread Settings 配置

- 使能 RGB565 LCD、GT9147触摸驱动、使能 LVGL。



软件说明

LVGL 相关代码在 `./board/lvgl` 中。

- lv_port_disp.c

```
#include <lvgl.h>
#include <rtthread.h>
#include "ra8/lcd_config.h"
#include "hal_data.h"

static rt_sem_t _SemaphoreVsync = RT_NULL;
static uint8_t lvgl_init_flag = 0;

void DisplayVsyncCallback(display_callback_args_t *p_args)
{
    rt_interrupt_enter();
    if (DISPLAY_EVENT_LINE_DETECTION == p_args->event)
```

```
    {
        if (lvgl_init_flag != 0)
            rt_sem_release(_SemaphoreVsync);
    }
    rt_interrupt_leave();
}

static void vsync_wait_cb(lv_display_t *display)
{
    if (!lv_display_flush_is_last(display)) return;

    // If Vsync semaphore has already been set, clear it then w
    rt_sem_take(_SemaphoreVsync, RT_WAITING_FOREVER);
}

static void disp_flush(lv_display_t *display, const lv_area_t *
{
    if (!lv_display_flush_is_last(display)) return;

#if (BSP_CFG_DCACHE_ENABLED)
    int32_t size;
    /* Invalidate cache - so the HW can access any data written
    size = sizeof(fb_background[0]);

    SCB_CleanInvalidateDCache_by_Addr(px_map, size);
#endif

    R_GLCDC_BufferChange(&g_display0_ctrl,
                         (uint8_t *) px_map,
                         (display_frame_layer_t) 0);
}

void lv_port_disp_init(void)
{
    static rt_device_t device;
    /* LCD Device Init */
    device = rt_device_find("lcd");
    RT_ASSERT(device != RT_NULL);

    _SemaphoreVsync = rt_sem_create("lvgl_sem", 1, RT_IPC_FLAG_

    if (RT_NULL == _SemaphoreVsync)
    {
        rt_kprintf("lvgl semaphore create failed\r\n");
        RT_ASSERT(0);
    }

    /*-----------------------------------
     * Create a display and set a flush_cb
```

```
     * ----------------------------------*/
    lv_display_t *disp = lv_display_create(LV_HOR_RES_MAX, LV_V
    lv_display_set_flush_cb(disp, disp_flush);
    lv_display_set_flush_wait_cb(disp, vsync_wait_cb);
    lv_display_set_buffers(disp, &fb_background[0][0], &fb_back

    lvgl_init_flag = 1;
}
```

- lv_port_indev.c

```
#include <lvgl.h>
#include <rtdevice.h>

#ifdef RT_USING_TOUCH

#include "gt9147.h"
#define GT9147_IRQ_PIN   BSP_IO_PORT_00_PIN_07
#define GT9147_RST_PIN   BSP_IO_PORT_04_PIN_12

#define DBG_TAG "lv_port_indev"
#define DBG_LVL DBG_LOG
#include <rtdbg.h>

#include "hal_data.h"

static rt_device_t touch_dev;
struct rt_touch_data *read_data;
static lv_indev_t *touch_indev;

#if LVGL_VERSION_MAJOR < 9
static void touchpad_read(lv_indev_drv_t *indev, lv_indev_data_
#else
static void touchpad_read(lv_indev_t *indev, lv_indev_data_t *d
#endif
{
    if (RT_NULL != rt_device_read(touch_dev, 0, read_data, 1))
    {
        if (read_data->event == RT_TOUCH_EVENT_NONE)
            return;

        data->point.x = LV_HOR_RES_MAX - read_data->x_coordinat
        data->point.y = LV_VER_RES_MAX - read_data->y_coordinat

        if (read_data->event == RT_TOUCH_EVENT_DOWN)
            data->state = LV_INDEV_STATE_PR;
        if (read_data->event == RT_TOUCH_EVENT_MOVE)
            data->state = LV_INDEV_STATE_PR;
```

```
        if (read_data->event == RT_TOUCH_EVENT_UP)
            data->state = LV_INDEV_STATE_REL;
    }
}

static rt_err_t gt9147_probe(rt_uint16_t x, rt_uint16_t y)
{
    void *id;

    touch_dev = rt_device_find("gt9147");
    if (touch_dev == RT_NULL)
    {
        rt_kprintf("can't find device gt9147\n");
        return -1;
    }

    if (rt_device_open(touch_dev, RT_DEVICE_FLAG_INT_RX) != RT_
    {
        rt_kprintf("open device failed!");
        return -1;
    }

    id = rt_malloc(sizeof(struct rt_touch_info));
    rt_device_control(touch_dev, RT_TOUCH_CTRL_GET_ID, id);
    rt_uint8_t *read_id = (rt_uint8_t *)id;
    rt_kprintf("id = GT%d%d%d \n", read_id[0] - '0', read_id[1]

    rt_device_control(touch_dev, RT_TOUCH_CTRL_SET_X_RANGE, &x)
    rt_device_control(touch_dev, RT_TOUCH_CTRL_SET_Y_RANGE, &y)
    rt_device_control(touch_dev, RT_TOUCH_CTRL_GET_INFO, id);
    rt_kprintf("range_x = %d \n", (*(struct rt_touch_info *)id)
    rt_kprintf("range_y = %d \n", (*(struct rt_touch_info *)id)
    rt_kprintf("point_num = %d \n", (*(struct rt_touch_info *)i
    rt_free(id);

    read_data = (struct rt_touch_data *)rt_calloc(1, sizeof(str
    if (!read_data)
    {
        return -RT_ENOMEM;
    }

    return RT_EOK;
}

rt_err_t rt_hw_gt9147_register(void)
{
    struct rt_touch_config config;
    rt_uint8_t rst;
    rst = GT9147_RST_PIN;
```

```
    config.dev_name = "i2c0";
    config.irq_pin.pin = GT9147_IRQ_PIN;
    config.irq_pin.mode = PIN_MODE_INPUT_PULLDOWN;
    config.user_data = &rst;

    if (rt_hw_gt9147_init("gt9147", &config) != RT_EOK)
    {
        rt_kprintf("touch device gt9147 init failed.\n");
        return -RT_ERROR;
    }

    if (gt9147_probe(LV_HOR_RES_MAX, LV_VER_RES_MAX) != RT_EOK)
    {
        rt_kprintf("probe gt9147 failed.\n");
        return -RT_ERROR;
    }
    return RT_EOK;
}
#endif

void lv_port_indev_init(void)
{
#ifdef RT_USING_TOUCH

#if LVGL_VERSION_MAJOR < 9
    static lv_indev_drv_t indev_drv;         /* Descriptor of a
    lv_indev_drv_init(&indev_drv);           /* Basic initializ
    indev_drv.type = LV_INDEV_TYPE_POINTER;  /* Touch pad is a
    indev_drv.read_cb = touchpad_read;       /* Set your driver

    /* Register the driver in LVGL and save the created input d
    touch_indev = lv_indev_drv_register(&indev_drv);
#else
    /*Register a touchpad input device*/
    touch_indev = lv_indev_create();
    lv_indev_set_type(touch_indev, LV_INDEV_TYPE_POINTER);
    lv_indev_set_read_cb(touch_indev, touchpad_read);
#endif
    /* Register touch device */
    rt_err_t res = rt_hw_gt9147_register();
    RT_ASSERT(res == RT_EOK);

#endif
}
```

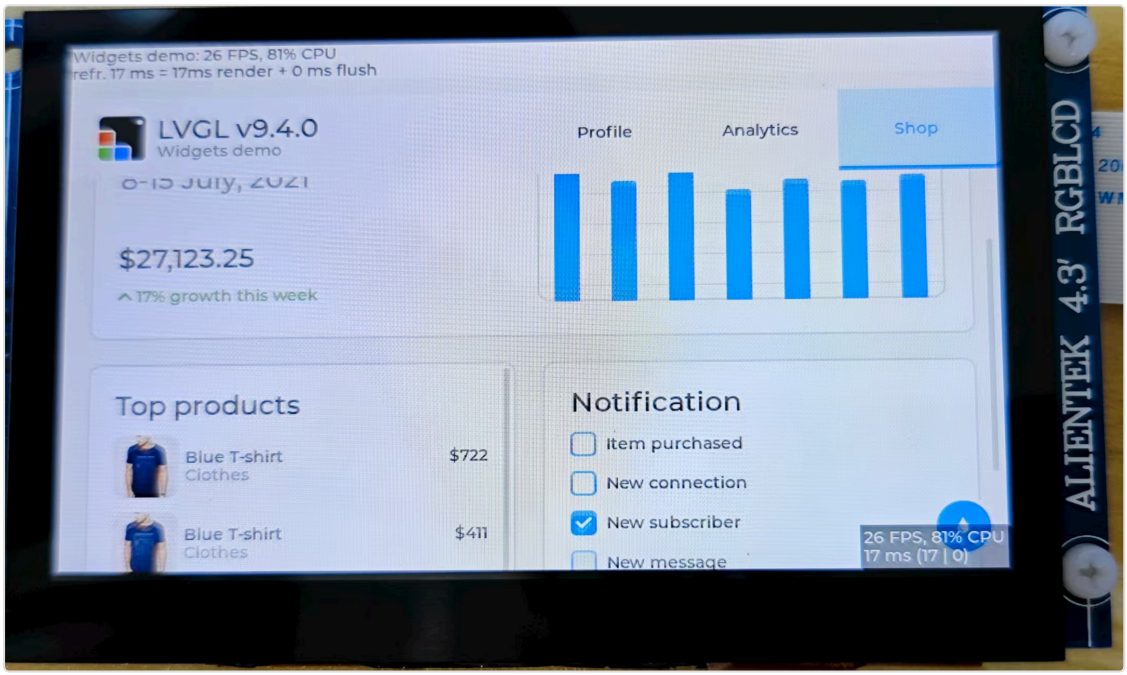
编译&下载

- RT-Thread Studio：在RT-Thread Studio 的包管理器中下载 Titan Board 资源包，然后创建新工程，执行编译。

编译完成后，将开发板的 USB-DBG 接口与 PC 机连接，然后将固件下载至开发板。

运行效果

复位开发板后就能在 LCD 屏幕上看到 LVGL Demo 正在运行。



4.3. MIPI NPU 人脸检测

English | 中文

简介

该示例在 **Titan Board Mini** 上实现了基于 **MIPI CSI 摄像头** 的实时人脸检测，主要使用以下模块：

- **OV5640** 通过 **MIPI CSI** 接入
- **RA8P1 VIN** 负责图像采集
- **Arm Ethos-U55 NPU** 负责 YOLO-Fastest 推理
- **RGB565 LCD** 负责实时预览和人脸框叠加显示

功能

- 通过 **MIPI CSI + VIN** 采集摄像头图像
- 使用 **YOLO-Fastest 人脸检测模型** 进行 NPU 推理
- 在 LCD 上叠加绿色人脸框
- 支持通过用户按键触发 **OV5640 自动对焦**
- 支持通过宏定义控制检测日志输出

数据流

```
OV5640
  -> MIPI CSI
  -> VIN 帧缓冲
  -> CPU 预处理（缩放 + 灰度化 + 量化）
  -> Ethos-U55 推理
  -> CPU 后处理（解码 + NMS）
  -> D/AVE2D 叠框
  -> GLCDC
  -> RGB565 LCD
```

模型信息

- 模型：**YOLO-Fastest 人脸检测**
- 框架：**TensorFlow Lite INT8**
- 模型输入尺寸：**192 x 192**
- 摄像头采集尺寸：**640 x 480**

运行控制

- **自动对焦**
- 按下用户按键后触发 OV5640 自动对焦。
- **检测日志开关**
- 在 `src/hal_entry.c` 中修改 `DETECT_RESULT_LOG_ENABLE`
- `0`：关闭 `detect box num` / `Time elapsed` 日志
- `1`：打开日志

说明

- 当前工程在人脸框显示时使用 framebuffer 叠加渲染路径。
- 当前 MIPI 摄像头链路中，OV5640 通过 **I2C0** 配置。

RT-Thread / FSP 配置要点

工程需要启用以下模块：

- **MIPI CSI**
- **VIN**
- **Ethos-U55 / rm_ethosu**
- **I2C0 摄像头控制**
- **RGB565 LCD / GLCDC**

编译与下载

可通过 **RT-Thread Studio** 或仓库现有的工程构建流程完成编译，然后通过开发板调试接口下载固件。

运行效果

系统启动后：

- LCD 显示 OV5640 实时画面
- 检测到人脸时在屏幕上绘制绿色矩形框
- 按下用户按键可触发摄像头自动对焦



5. 多核通信篇

5.1. Titan_Mini_rpmsg_master 核间通信示例（M85）

中文 | English

简介

本工程是 Titan Board Mini 双核 RPMSG 通信示例的 Master 端，运行在 Cortex-M85（Core0）。工程上电后初始化 RPMSG 主端，不主动发送消息；通过控制台命令触发，与 Cortex-M33（Core1）上的 `Titan_Mini_rpmsg_remote` 完成一次双向 hello 通信。

配套工程：`project/Titan_Mini_rpmsg/Titan_Mini_rpmsg_remote`。

工程结构

- `src/hal_entry.c`：RPMSG Master 初始化、接收处理、`rpmsg_hello` 命令
- `board/linker_scripts/fsp.ld`：Core0 链接脚本，Flash 起始 0x02000000，长度 0xC0000
- `packages/rpmsg-lite-latest`：RPMSG Lite 软件包
- `rt-thread`、`libraries`：链接到仓库根目录的共享代码

通信流程

1. M85 初始化 RPMSG Master，等待与 M33 的 RPMSG 通道就绪。
2. 在 M85 控制台输入 `rpmsg_hello`。
3. M85 发送字符串 `hello` 到 M33，并打印 `M85 TX: hello`。
4. M33 收到后打印 `M33 RX: hello`，并自动回发 `hello`。
5. M85 收到回包后打印 `M85 RX: hello`。

使用说明

1. 先编译并烧写 `Titan_Mini_rpmsg_remote` 到 Core1 Flash（0x020C0000）。
2. 再编译并烧写本工程到 Core0 Flash（0x02000000）。
3. M85 调试串口为 UART1，M33 调试串口为 UART2。
4. M85 启动后看到 `RPMSG master ready`，在 M85 控制台输入 `rpmsg_hello`。

验证输出

M85 侧控制台：

```
RPMSG master ready
M85 TX: hello
M85 RX: hello
```

M33 侧控制台：

```
M33 RX: hello
M33 TX: hello
```

5.2. Titan_Mini_rpmsg_remote 核间通信示例（M33）

中文 | **English**

简介

本工程是 Titan Board Mini 双核 RPMSG 通信示例的 Remote 端，运行在 Cortex-M33（Core1）。工程由 M85 启动，初始化 RPMSG 从端后等待接收消息；收到 M85 发来的 `hello` 后自动回发 `hello`。

配套工程：`project/Titan_Mini_rpmsg/Titan_Mini_rpmsg_master`。

工程结构

- `src/hal_entry.c`：RPMSG Remote 初始化、接收处理、自动回包逻辑
- `board/linker_scripts/fsp.ld`：Core1 链接脚本，Flash 起始 0x020C0000，长度 0x30000
- `packages/rpmsg-lite-latest`：RPMSG Lite 软件包
- `rt-thread`、`libraries`：链接到仓库根目录的共享代码

通信流程

1. M33 初始化 RPMSG Remote，等待 M85 的 RPMSG 通道就绪。
2. M85 控制台执行 `rpmsg_hello` 后，向 M33 发送 `hello`。
3. M33 收到后打印 `M33 RX: hello`，并自动回发 `hello`。
4. M33 回包后打印 `M33 TX: hello`，M85 收到后打印 `M85 RX: hello`。

使用说明

1. 先编译并烧写本工程到 Core1 Flash（0x020C0000）。
2. 再编译并烧写 `Titan_Mini_rpmsg_master` 到 Core0 Flash（0x02000000）。
3. M33 调试串口为 UART2，M85 调试串口为 UART1。
4. 复位开发板后，在 M85 控制台输入 `rpmsg_hello` 验证双向通信。

验证输出

M33 侧控制台：

```
M33 RX: hello
M33 TX: hello
```

M85 侧控制台：

```
RPMSG master ready
M85 TX: hello
M85 RX: hello
```

6. 外设扩展篇

6.1. PDM 麦克风实时回环示例说明

中文 | **English**

简介

本示例展示了如何在 **Titan Board Mini** 上使用 **RA8P1 的 PDM (Pulse Density Modulation) 接口** 读取 **PDM 麦克风** 数据，并通过 **ES8156 音频编解码器** 实时播放，实现 **麦克风 → 扬声器** 的音频回环功能。

主要功能包括：

- 使用 RA8P1 内置 PDM 接口读取 PDM 麦克风数据
- PDM 数据转 PCM 格式（支持 16-bit 和 20-bit 模式）
- 单声道转双声道音频处理
- 双缓冲机制实现连续音频流播放
- 通过 SSI 接口和 ES8156 编解码器输出音频

硬件介绍

1. PDM 麦克风

Titan Board Mini 板载 **PDM 麦克风**：

参数	说明
类型	MEMS PDM 麦克风
接口	PDM (Pulse Density Modulation)
输出格式	1-bit 密度流

参数	说明
采样率	最高 16kHz（本示例）
声道	单声道

2. RA8P1 PDM 接口

RA8P1 的 PDM 接口功能：

- **内置 PDM 解调器**：硬件实现 PDM → PCM 转换
- **可编程滤波器**：SINC 滤波器、HPF、LPF
- **多通道支持**：支持多路麦克风输入
- **可配置输出**：支持 16-bit 或 20-bit PCM 输出
- **DMA 支持**：支持 DMA 传输，减少 CPU 占用

3. 音频输出

- **编解码器**：ES8156（24-bit DAC，支持 8kHz - 192kHz）
- **接口**：SSI (I2S 协议)
- **输出**：3.5mm 耳机接口 / 板载扬声器

软件架构

1. 数据流程

```
PDM 麦克风
    ↓ (PDM 信号)
RA8P1 PDM 接口
    ↓ (硬件滤波 + PDM→PCM)
PDM 缓冲区 (int32_t[])
    ↓ (软件转换)
DAC 缓冲区 (int16_t[], 双声道)
    ↓ (I2S 协议)
ES8156 编解码器
    ↓ (模拟音频)
扬声器/耳机
```

2. 核心组件

PDM 驱动

使用 Renesas FSP 的 PDM 驱动：

```
/* 打开 PDM 模块 */
R_PDM_Open(&g_pdm0_ctrl, &g_pdm0_cfg);

/* 启动数据采集 */
R_PDM_Start(&g_pdm0_ctrl, buffer, buf_size.bytes, samples);

/* 停止采集 */
R_PDM_Stop(&g_pdm0_ctrl);

/* 关闭 PDM 模块 */
R_PDM_Close(&g_pdm0_ctrl);
```

数据转换函数

```
/* PDM 数据转 DAC 数据 */
static void convert_pdm_to_dac(int32_t *pdm_data, uint32_t pdm_
{
    for (uint32_t i = 0; i < pdm_samples; i++)
    {
        int16_t sample_16;
        if (g_pcm_bits == PCM_20BITS)
        {
            /* 20 位 -> 16 位：右移 4 位 */
            sample_16 = (int16_t)(pdm_data[i] >> 4);
        }
        else
        {
            /* 16 位模式：左移 1 位 */
            sample_16 = (int16_t)(pdm_data[i] << 1);
        }

        /* 单声道转双声道 */
        dac_data[i * 2] = sample_16;      /* 左声道 */
        dac_data[i * 2 + 1] = sample_16;  /* 右声道 */
    }
}
```

3. 双缓冲机制

为了实现连续音频流，采用双缓冲模式：

```
时间轴:
    t0      t1      t2      t3      t4
    |-------|-------|-------|-------|
缓冲区0: [录音 ] [播放          ]
缓冲区1:                 [录音 ] [播放          ]
```

- **缓冲区 0**：录音 → 播放
- **缓冲区 1**：录音 → 播放
- 两个缓冲区交替工作，实现无缝连续播放

4. 工程结构

```
Titan_Mini_pdm/
├── src/
│   └── hal_entry.c          # 主程序入口
├── libraries/
│   └── HAL_Drivers/
│       ├── drv_i2s.c        # SSI/I2S 驱动
│       └── ports/
│           └── es8156/      # ES8156 编解码器驱动
└── ra_gen/
    └── hal_data.c           # FSP 配置（PDM、SSI 等）
```

配置说明

1. PDM 配置

在 `ra_gen/hal_data.c` 中配置 PDM 参数：

```
const pdm_cfg_t g_pdm0_cfg =
{
    .unit                                = 0,
    .channel                             = 2,        /* PDM 通道 *
    .pcm_width                           = PDM_PCM_WIDTH_16_BITS_
    .pcm_edge                            = PDM_INPUT_DATA_EDGE_RI
    .p_extend                            = &g_pdm0_cfg_extend,
```

```
    // ...
};
```

2. 关键参数

参数	值	说明
`PDM_FS_HZ`	16000	输出采样率 16kHz
`PDM_CHANNELS`	1	单声道输入
`PDM_PCM_WIDTH`	16_BITS_0_14	16-bit PCM，有效位 [14:0]
`NUM_BUFFERS`	2	双缓冲模式
`PDM_RECORD_DURATION_SEC`	1	每个缓冲区 1 秒音频

运行效果

```
 \ | /
- RT -     Thread Operating System
 / | \     5.1.0 build Apr 22 2026 15:56:26
 2006 - 2024 Copyright by RT-Thread team
[I/I2C] I2C bus [i2c1] registered
[I/es8156] ES8156 driver initialized
Audio init success.
[I/SDIO] SD card capacity 15730176 KB.
found part[0], begin: 4194304, size: 14.661GB
[I/app.filesystem] SD card mount to '/sdcard'

********************************************************************************
*   PDM Microphone -> Speaker Real-time Loopback (Double Buffer)               *
********************************************************************************
Sample Rate: 16000 Hz, Channels: 1
Buffer: 16000 samples x 2 buffers
********************************************************************************

PDM PCM bits: 16
[I/main] PDM opened successfully
[I/main] PDM ready
[I/main] Speaker opened
msh />
```

1. 上电后，绿色 LED 点亮表示系统就绪
2. 蓝色 LED 点亮表示音频回环正在运行
3. 对着麦克风说话，可以在耳机/扬声器中实时听到声音
4. 蓝色 LED 每 50 次循环闪烁一次（心跳指示）

故障排查

1. 没有声音输出

- 检查耳机/扬声器是否正确连接
- 检查音量设置（默认 60%）
- 检查 ES8156 初始化是否成功

2. 声音卡顿

- 当前使用 1 秒缓冲区，可尝试调整 `PDM_RECORD_DURATION_SEC`
- 检查系统负载，减少其他任务

3. 声音失真

- 检查 `pcm_width` 配置是否与硬件匹配
- 调整数据转换函数中的位移参数

6.2. IMU 传感器示例说明

中文 | **English**

简介

本示例展示了如何在 **Titan Board Mini** 上使用 **LSM6DS3TR-C 六轴 IMU 传感器** 实现惯性测量功能,通过 **I2C 接口**读取 **3轴加速度计** 和 **3轴陀螺仪** 数据,结合 **RT-Thread 传感器框架** 实现完整的传感器数据采集和处理。

主要功能包括:

- 使用 LSM6DS3TR-C 实现 6 轴惯性测量
- 读取 3 轴加速度计数据 (X/Y/Z)
- 读取 3 轴陀螺仪数据 (X/Y/Z)
- 支持多种量程和采样率配置
- 集成 RT-Thread 传感器框架

硬件介绍

1. LSM6DS3TR-C IMU 传感器

Titan Board Mini 板载 **LSM6DS3TR-C** 高性能六轴 IMU 传感器:

参数	说明
型号	LSM6DS3TR-C
制造商	STMicroelectronics (意法半导体)
类型	6 轴 IMU (3轴加速度计 + 3轴陀螺仪)
接口	I2C / SPI
工作电压	1.71V - 3.6V
温度范围	-40°C ~ +85°C

参数	说明
封装	2.5mm x 3mm x 0.83mm LGA-14

2. 加速度计特性

LSM6DS3TR-C 内置高性能 3 轴加速度计：

- **量程选择**：±2g / ±4g / ±8g / ±16g
- **分辨率**：16-bit ADC
- **输出数据率**：1.6Hz - 6.66kHz
- **噪声密度**：90μg/√Hz
- **零偏偏差**：±40mg
- **带宽**：可配置 (通常 50Hz - 1.6kHz)

3. 陀螺仪特性

LSM6DS3TR-C 内置高精度 3 轴陀螺仪：

- **量程选择**：±125 / ±250 / ±500 / ±1000 / ±2000 dps
- **分辨率**：16-bit ADC
- **输出数据率**：1.6Hz - 6.66kHz
- **噪声密度**：3.8mdps/√Hz
- **零偏稳定性**：±5dps
- **带宽**：可配置

4. 主要功能

高级功能

- **FIFO 缓存**：9KB FIFO,支持多种模式
- **中断功能**：运动唤醒、自由落体、6D方向检测
- **传感器融合**：内置低功耗传感器融合算法
- **自检功能**：支持自检模式
- **低功耗模式**：多种低功耗工作模式

数据处理

- **硬件滤波**：可配置数字滤波器
- **数据融合**：支持加速度计+陀螺仪数据融合
- **时间戳**：内置时间戳功能
- **轮询/中断**：支持轮询和中断数据读取

软件架构

1. 分层设计

IMU 传感器系统采用分层架构：

```
应用程序层 (用户代码)
    ↓
RT-Thread Sensor Framework - 传感器框架
    ↓
LSM6DS3TR-C Driver - IMU驱动
    ↓
Sensor HAL - 传感器硬件抽象层
    ↓
I2C/SPI Driver - I2C/SPI驱动
    ↓
FSP I2C/SPI HAL - 硬件抽象层
```

2. 核心组件

移植层接口

需要实现的平台相关接口（`lsm6ds3tr-c_port.c`）：

```
/* I2C 读写接口 */
int32_t platform_write(void *handle, uint8_t reg, const uint8_t
int32_t platform_read(void *handle, uint8_t reg, uint8_t *bufp,

/* 延时接口 */
void platform_delay(uint32_t ms);
```

RT-Thread 传感器框架

RT-Thread 提供的统一传感器设备接口：

```
/* 查找传感器设备 */
rt_device_t rt_device_find(const char *name);

/* 打开传感器设备 */
rt_err_t rt_device_open(rt_device_t dev, rt_uint16_t oflags);

/* 读取传感器数据 */
rt_size_t rt_device_read(rt_device_t dev, rt_off_t pos, void *b

/* 接收传感器数据 */
rt_err_t rt_device_set_rx_indicate(rt_device_t dev, rt_err_t (*
```

3. 工程结构

```
Titan_Mini_peripheral_imu/
├── src/
│   └── hal_entry.c          # 主程序入口（RGB LED 闪灯演示）
└── packages/
    └── lsm6ds3tr/           # LSM6DS3TR-C 驱动包
        ├── lsm6ds3tr-c_reg.h    # 寄存器定义和驱动接口
        ├── lsm6ds3tr-c_reg.c    # 寄存器级驱动实现
        └── lsm6ds3tr-c_port.c   # 平台移植层 + INIT_APP_EXPORT 自
```

使用说明

1. 工作模式

本工程采用 **开机自动初始化 + MSH 命令运行** 的模式：

- **开机时**：RT-Thread 启动后由 `INIT_APP_EXPORT` 自动完成 LSM6DS3TR-C 的初始化（含 ID 检测、复位、ODR/满量程/滤波配置），用户无需手动 `init`。
- **开机后**：主线程只跑 RGB LED 闪灯演示，IMU 数据读取**不自动运行**，需要用户在 msh 输入命令才会读传感器。

这样既保证开机流程不被 IMU 采样死循环阻塞，也避免用户忘记初始化。

2. MSH 命令

烧录后,在串口终端（msh />）输入以下命令：

命令	说明
`imu`	读取并打印一次传感器信息（含姿态解析）
`imu start [ms]`	开始周期性采样，可选周期（默认 1000ms，最小 50ms）
`imu stop`	停止周期性采样

示例：

```
msh /> imu start 500        # 每 500ms 输出一帧
msh /> imu stop             # 停止采样
msh /> imu                  # 单次打印
```

3. 数据输出格式

每帧输出包含 3 轴加速度、3 轴角速度、温度，以及解析后的合矢量与姿态角：

```
------ LSM6DS3TR-C Sample ------
Accel  (mg) : X=  -12.0  Y=   35.0  Z=  998.0  | |a|= 998.7
Gyro   (dps): X=  0.02  Y=  -0.01  Z=   0.03  | |w|=  0.04
Temp   (C)  :  28.45
Tilt   (deg): pitch= -0.69  roll=  2.01  (static estimate)
--------------------------------
```

- **Accel**：三轴加速度 (mg)，`|a|` 为加速度合矢量（静止时约 1000mg ≈ 1g）
- **Gyro**：三轴角速度 (dps)，`|w|` 为角速度合矢量
- **Temp**：板载温度 (°C)
- **Tilt**：由重力分量估算的 pitch / roll 姿态角（仅静止时有效，动态下需结合陀螺仪做融合）

4. 初始化流程说明

开机自动初始化内部完成：

1. 通过 I2C (`i2c1` , 7 位地址 `0x6A`) 查找设备并校验 WHO_AM I (期望 `0x6A`)
2. 触发软件复位并等待复位完成
3. 打开块数据更新 (BDU)

4. 设置加速度计/陀螺仪 ODR = 12.5Hz，满量程 ±2g / ±2000dps
5. 配置加速度计模拟滤波 + LPF2，陀螺仪带通滤波
6. 初始化成功后打印 `[imu] auto-init OK`

初始化函数 `lsm6ds3_init()` 同样导出，可在代码中手动调用（例如改 ODR 后重新初始化）。

配置说明

1. Kconfig 配置

在 `libraries/M85_Config/Kconfig` 中配置 IMU 选项：

```
menuconfig BSP_USING_IMU
    bool "Enable IMU (LSM6DS3TR-C)"
    select BSP_USING_I2C2
    default n
    if BSP_USING_IMU
        config BSP_IMU_I2C_BUS
            string "I2C bus name"
            default "i2c2"

        config BSP_IMU_ACC_ODR
            int "Accelerometer ODR (Hz)"
            default 104

        config BSP_IMU_GYRO_ODR
            int "Gyroscope ODR (Hz)"
            default 104
    endif
```

2. RT-Thread Settings

在 RT-Thread Studio 中,需要启用以下组件：

1. **设备驱动**

- 启用 I2C 设备驱动
- 配置 I2C2 接口

2. **传感器**

- 启用 RT-Thread Sensor 框架

- 启用 Accel (加速度计) 传感器
- 启用 Gyro (陀螺仪) 传感器

3. **软件包**

- 添加 LSM6DS3TR-C 驱动包

运行效果

1. 开机启动信息

复位 Titan Board Mini 后,串口首先输出系统启动和 IMU 自动初始化信息：

```
Hello RT-Thread!
=================================================
Titan Mini peripheral IMU demo
  - LED RGB blink runs on boot
  - LSM6DS3TR-C IMU auto-init on boot, commands:
      imu                 read & print one sample
      imu start [ms]      start periodic sampling
      imu stop            stop periodic sampling
=================================================
\ | /
- RT -     Thread Operating System
 / | \     5.x.x build ...
...
[imu] device detected, id=0x6A
[imu] initialized: ODR=12.5Hz, FS=±2g/±2000dps
[imu] auto-init OK, run 'imu' to read sample
msh />
```

随后 msh 就绪，此时 LED 在跑 RGB 闪灯，IMU 待命。

2. IMU 命令运行效果

输入 `imu start 1000` 后周期性输出：

```
Angular rate [mdps]:0.00        0.00    0.00
Temperature [degC]: 29.23
Acceleration [mg]:-5.92 30.26   1039.07
Angular rate [mdps]:0.00        0.00    0.00
Temperature [degC]: 29.16
Acceleration [mg]:-6.04 30.26   1039.44
Angular rate [mdps]:0.00        0.00    0.00
Temperature [degC]: 29.22
Acceleration [mg]:-6.04 30.32   1039.68
Angular rate [mdps]:0.00        0.00    0.00
Temperature [degC]: 29.20
Acceleration [mg]:-6.10 30.26   1039.56
Angular rate [mdps]:0.00        0.00    0.00
Temperature [degC]: 29.18
Acceleration [mg]:-6.22 30.26   1039.44
Angular rate [mdps]:0.00        0.00    70.00
Temperature [degC]: 29.18
Acceleration [mg]:-6.22 30.32   1039.44
Angular rate [mdps]:0.00        -70.00  -70.00
Temperature [degC]: 29.21
Acceleration [mg]:-6.22 30.32   1039.56
Angular rate [mdps]:0.00        0.00    -70.00
Temperature [degC]: 29.19
Acceleration [mg]:-6.22 30.38   1039.74
Angular rate [mdps]:0.00        0.00    0.00
Temperature [degC]: 29.24
```

相关资料

- RT-Thread 传感器框架文档

6.3. RGB LCD 示例说明

中文 | **English**

简介

本示例演示了如何在 **Titan Board** 上使用 **RA8 系列 MCU 的 GLCDC 模块**结合 **RT-Thread 的 LCD 驱动框架**驱动 RGB LCD 屏幕，实现图像显示与界面更新。文档将详细介绍 **RA8 系列 GLCDC 外设特性**及 **RT-Thread LCD 驱动框架**，并提供示例配置和操作方法。

RA8 系列 GLCDC 模块

1. 简介

GLCDC（Graphics LCD Controller）是 RA8 系列 MCU 内置的高性能图形控制器模块，专门用于驱动 TFT/RGB LCD 屏幕，支持各种分辨率、色彩格式和图像处理功能。结合 RT-Thread 的 **LCD 驱动框架**，可以实现统一接口下的屏幕初始化、刷新、图像绘制及 DMA 加速等功能。

RA8 系列 **GLCDC (Graphics LCD Controller)** 提供了从 MCU 内部存储器或外部图像缓存向 RGB/LCD 显示屏输出图像的能力。它集成了：

- **帧缓冲控制**：可配置多帧缓冲，实现页面切换或双缓冲显示
- **颜色格式支持**：RGB565, RGB888, ARGB8888 等
- **图形处理功能**：背景图层、文字/图形合成、透明度混合、调色板等
- **同步信号生成**：HSYNC, VSYNC, DE（Data Enable）等
- **DMA 支持**：高速数据传输，减少 CPU 占用
- **中断功能**：帧结束中断、行结束中断等

2. 模块架构

RA8 GLCDC 模块主要包含以下子模块：

1. **图层合成单元（Layer Composition Unit）**

- 支持多图层叠加
- 提供 alpha blending、透明度控制、颜色键控

- 可以对图层进行旋转、翻转处理

2. **帧缓冲接口（Frame Buffer Interface）**

- 支持访问 MCU 内部 SRAM 或外部存储器
- 提供单/双缓冲模式，保证连续显示
- 配合 DMA 自动读取图像数据

3. **DMA 控制器（DMA Controller）**

- 自动传输像素数据到 RGB 输出端口
- 可配置突发长度，提高带宽利用率
- 支持循环传输，适合视频或动画场景

4. **同步信号生成器（Timing Generator）**

- 自动生成 HSYNC/VSYNC/DE 信号
- 支持 TTL 接口 RGB 时序
- 可配置极性、同步宽度、前后肩时间等

5. **中断与事件处理单元（Interrupt/Event Controller）**

- 提供帧结束中断、行结束中断
- 可用于页面切换、动态绘制或滚动显示
- 支持 DMA 传输完成触发中断

3. GLCDC 工作原理

1. **帧缓冲读取**

- GLCDC 通过 DMA 从内存读取图像数据，支持单/双缓冲模式，保证连续显示。

2. **图层合成**

- 支持多图层叠加，例如背景图层 + 前景图层 + 图标/文字图层
- 提供透明度控制和调色板映射

3. **像素时序输出**

- 根据 LCD 的接口要求生成 HSYNC/VSYNC/DE 信号
- 支持 RGB 并行接口、TTL 接口或 LVDS（视具体板级实现）

4. **中断与事件**

- **帧结束中断（VBlank）**：可用于更新下一帧数据
- **行结束中断**：可用于滚动显示或动态绘制

4. GLCDC 支持的功能与特性

功能类别	描述
分辨率	最高可达 800×480 (具体取决于 MCU 型号及时钟配置)
色彩模式	RGB565、RGB888、ARGB8888 等
多图层	背景 + 前景 + 符号图层，可叠加混合
帧缓冲	支持单帧/双帧缓冲模式，DMA 传输提高性能
调色板	支持 8/16 位调色板映射，实现色彩转换
同步信号	HSYNC, VSYNC, DE，可配置极性和时序
DMA 支持	自动从内存传输图像数据，无需 CPU 干预
中断	帧结束、行结束中断，可用于屏幕刷新同步
旋转/翻转	支持 90°/180°/270°旋转及 X/Y 翻转

RT-Thread LCD 驱动框架

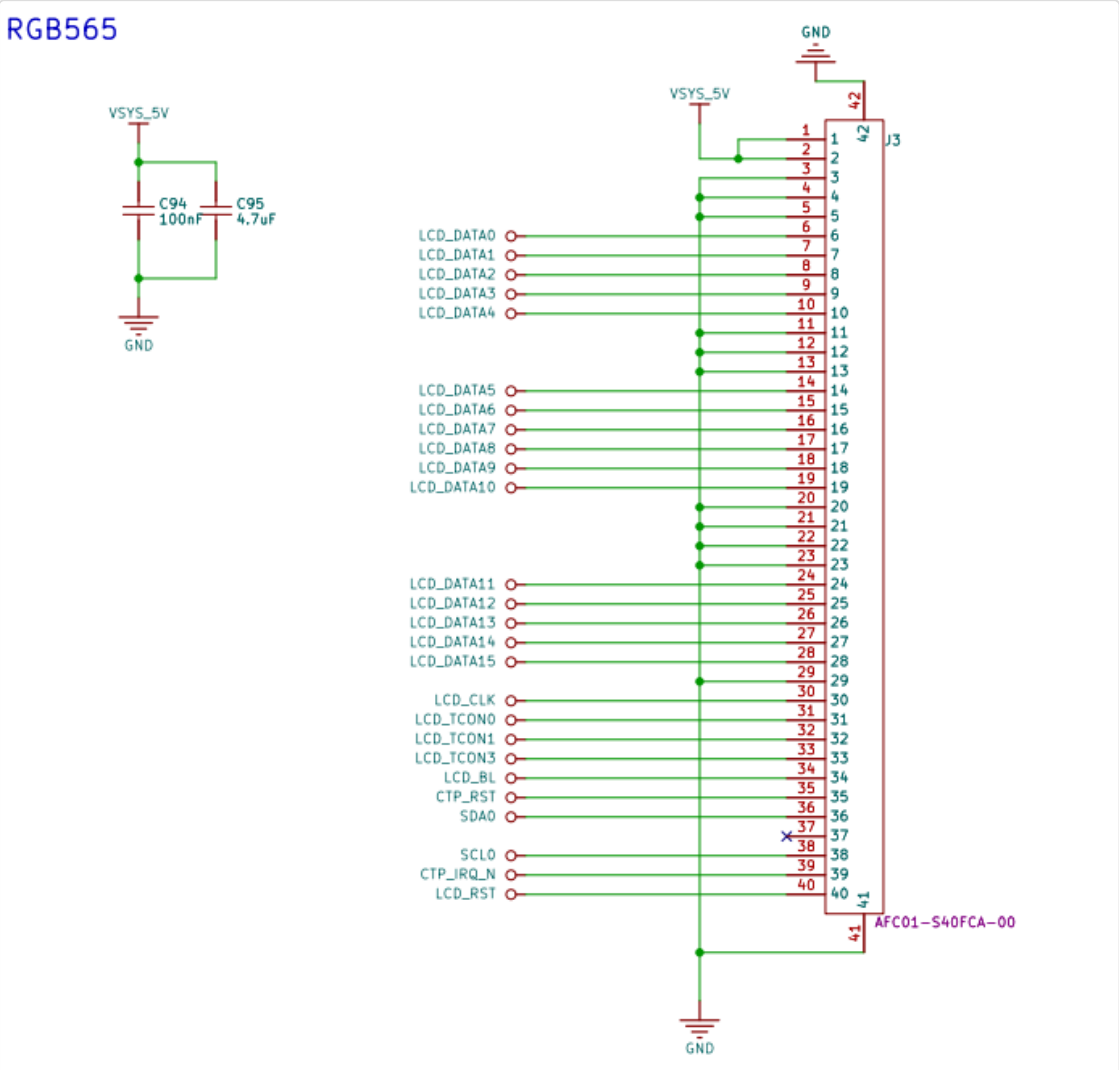
RT-Thread 提供了 **统一的 LCD 驱动框架**，通过封装底层控制器，支持不同屏幕类型与 MCU 外设。框架主要接口如下：

主要接口

函数/宏	功能
`rt_device_find("lcd")`	查找 LCD 设备句柄
`rt_device_open(dev, flags)`	打开设备，初始化 LCD 硬件

函数/宏	功能
`rt_device_control(dev, cmd, args)`	控制 LCD，例如更新lcd图像、设置背光、读取屏幕信息
`rt_device_write(dev, pos, buffer, size)`	向 LCD 写入像素数据
`rt_device_read(dev, pos, buffer, size)`	从 LCD 读取像素数据（部分屏幕支持）
`rt_device_close(dev)`	关闭设备

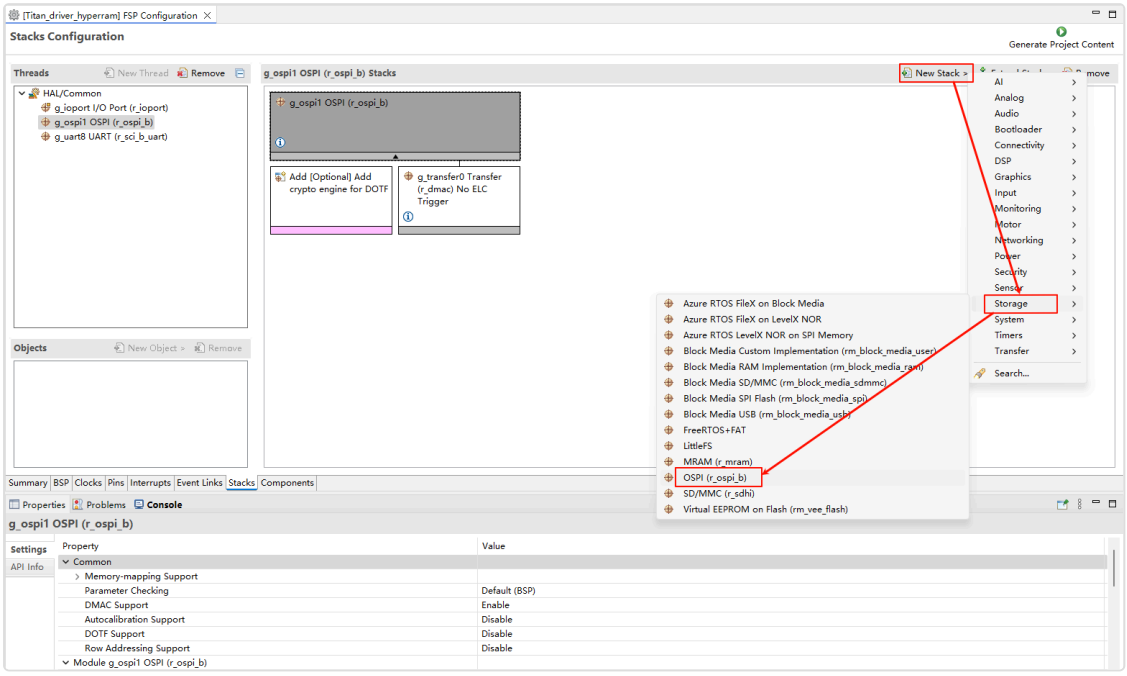
硬件说明



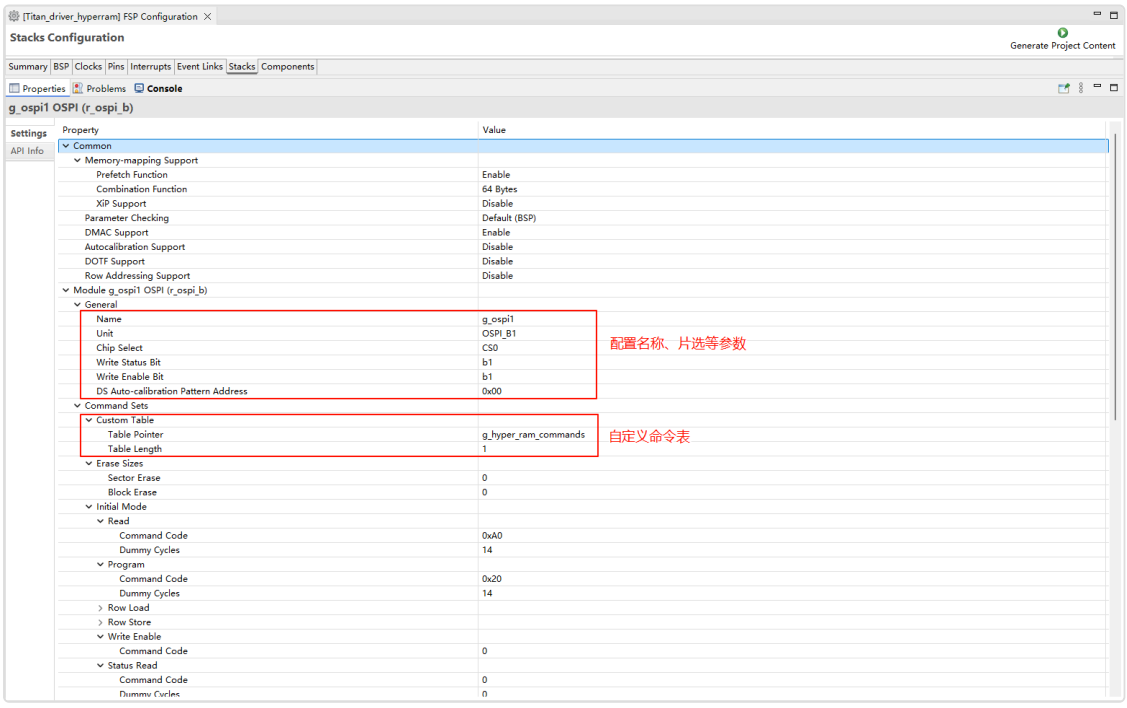
FSP 配置

HyperRAM 配置

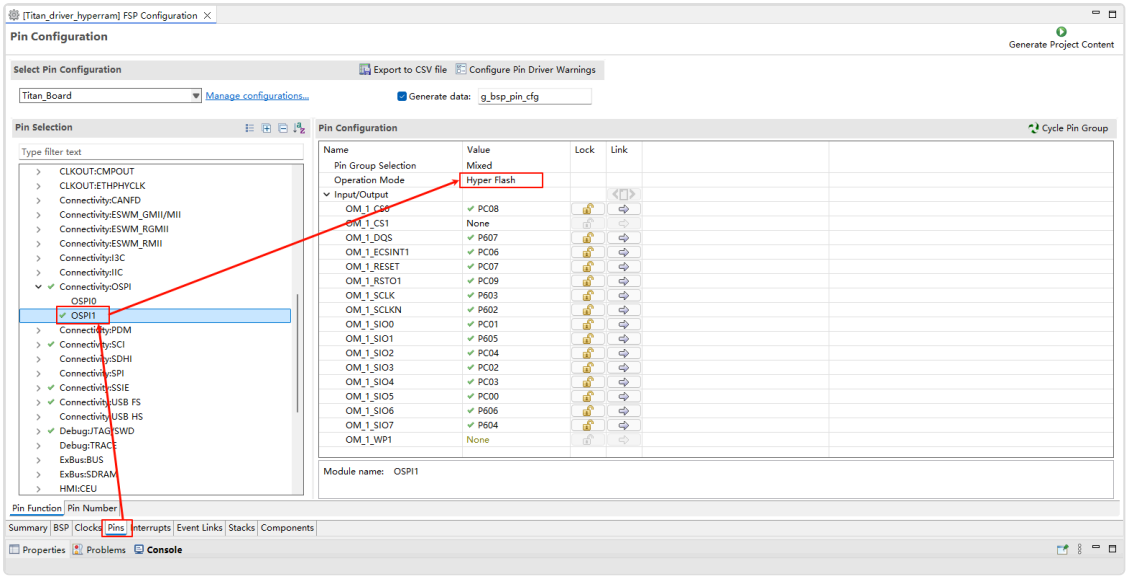
- 新建 r_ospi_b stack：



- 配置 r_ospi_b stack：

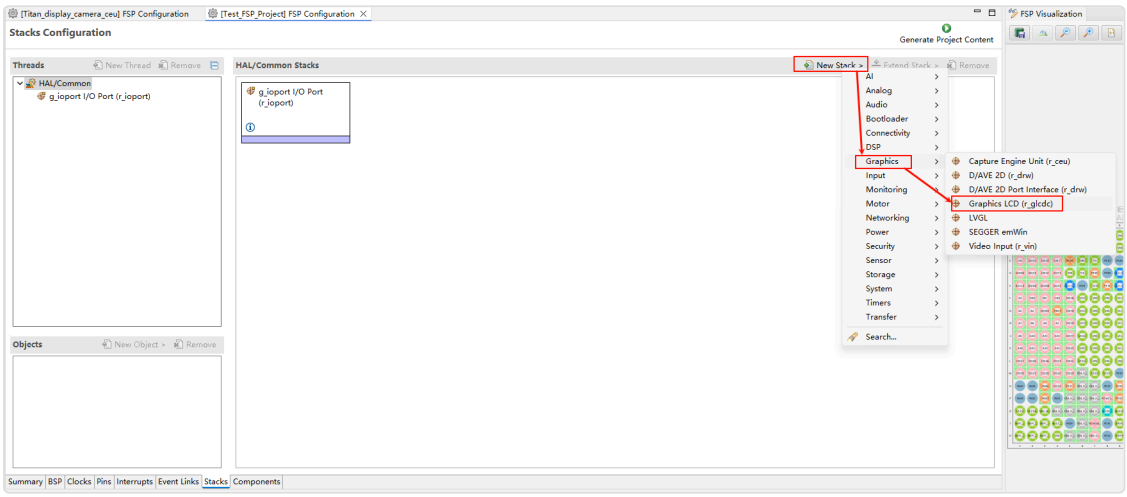


- HyperRAM 引脚配置：

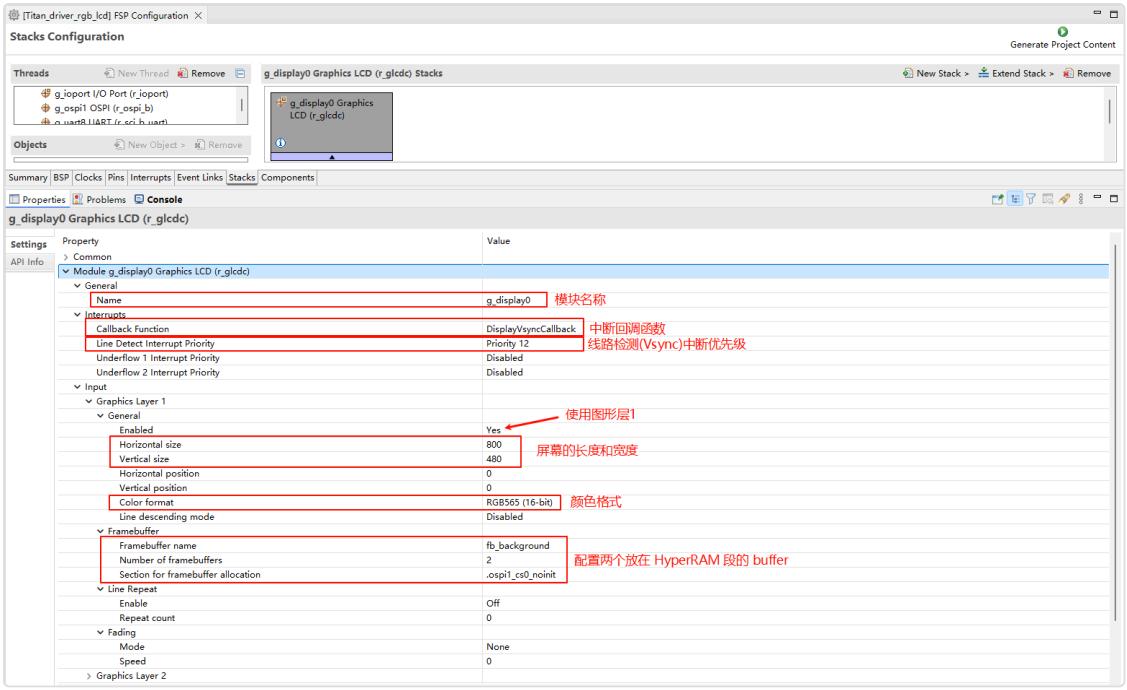


RGB LCD 配置

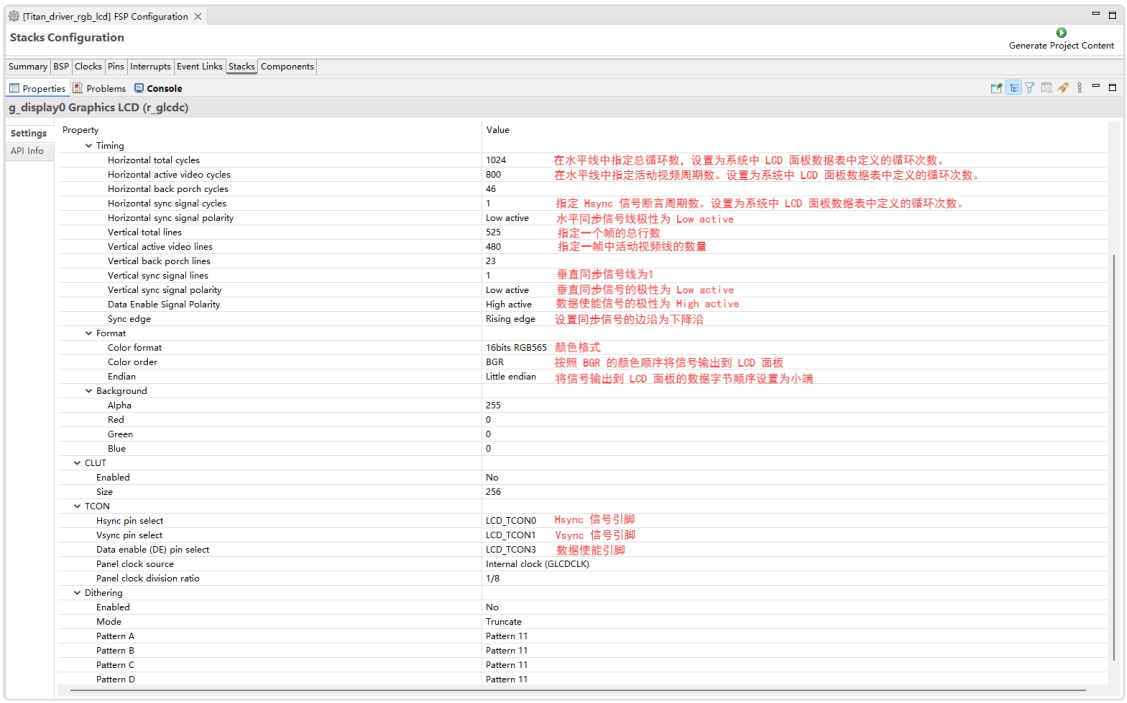
- 新建 `r_glcdc` stack：



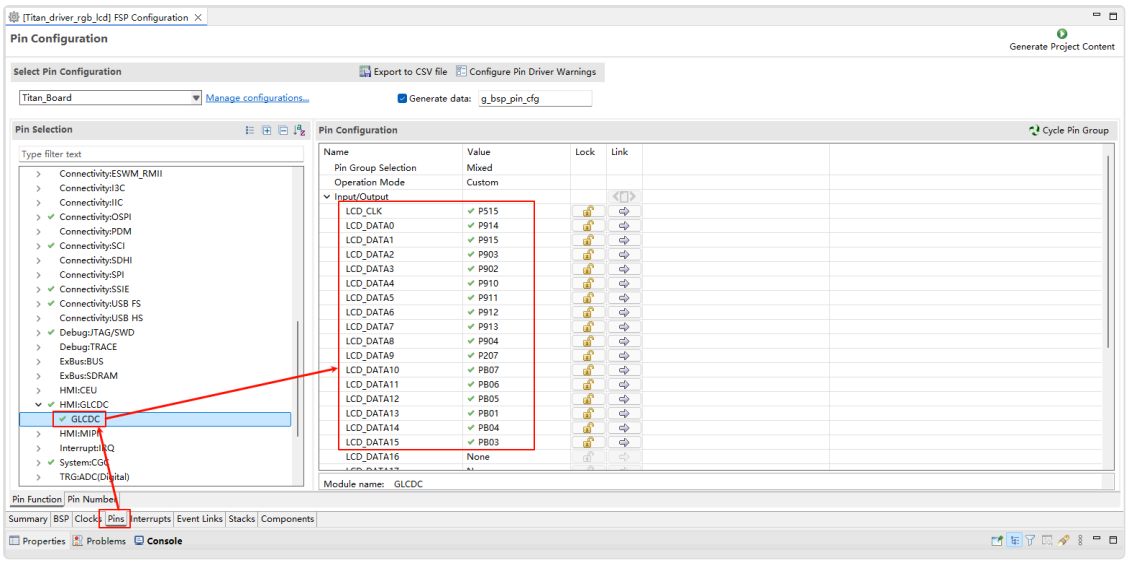
- 配置中断回调和图形层1：

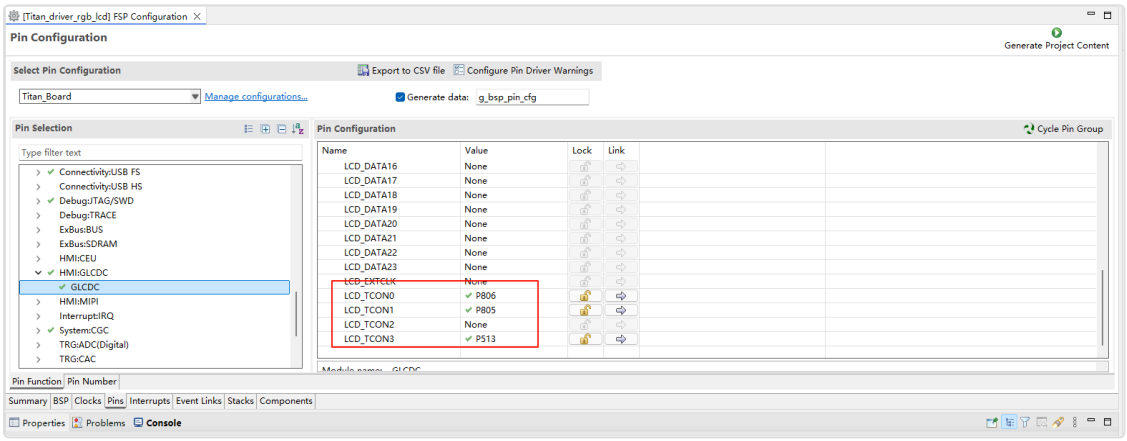


- 配置输出参数、CLUT、TCON和抖动。



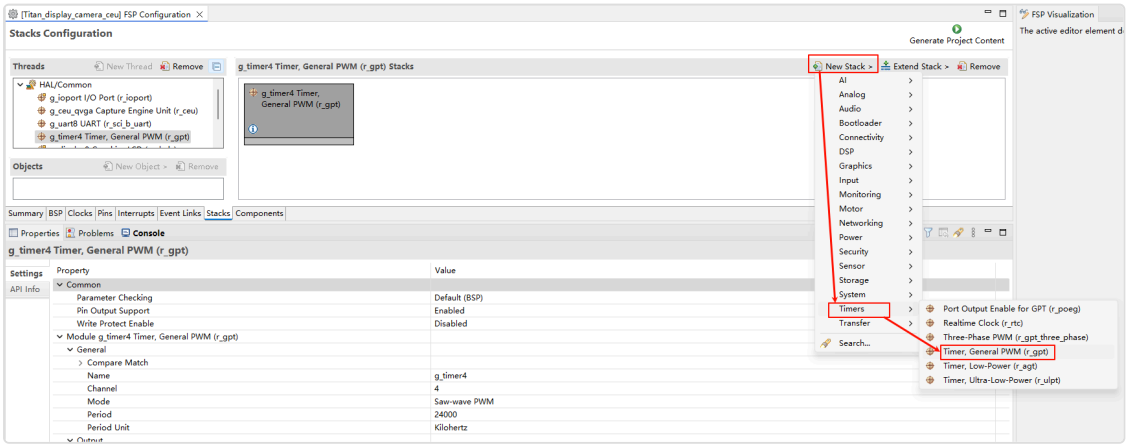
- 配置 GLCDC 的引脚：



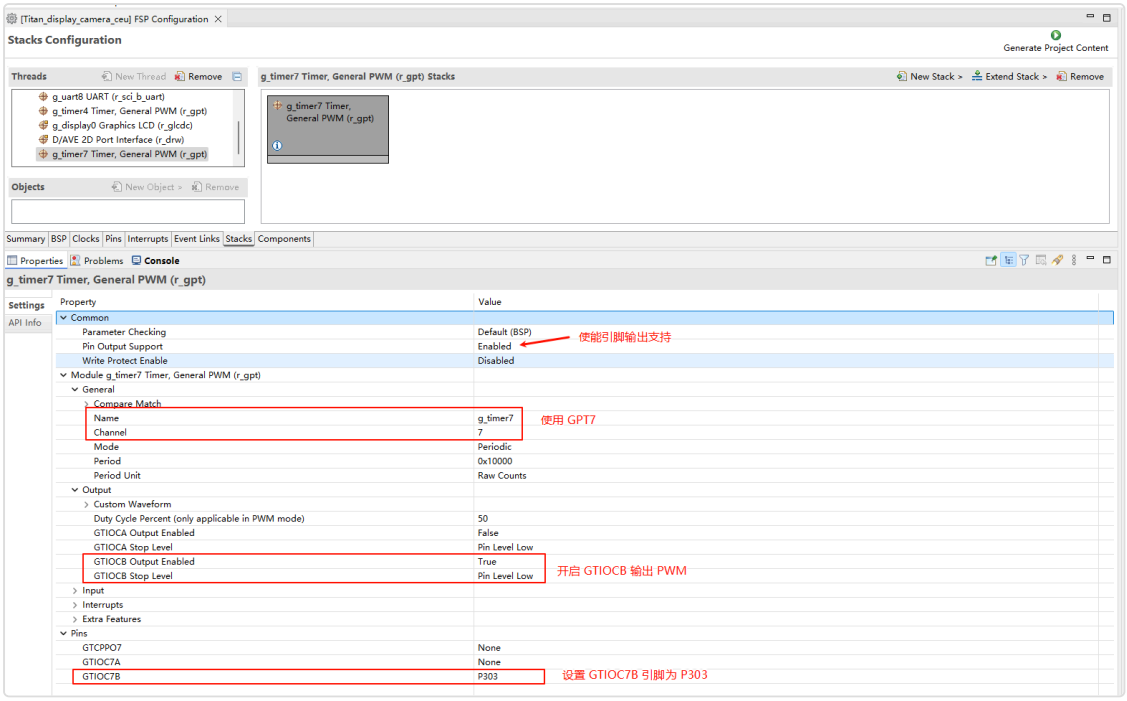


LCD 背光配置

- 新建 `r_gpt` stack：



- 配置背光 PWM 输出：



D/AVE 2D 配置

- 新建 `r_drw` stack：



RT-Thread Settings 配置

- 在 RT-Thread Settings 中使能 RGB565 LCD，使用 pwm7 输出屏幕背光。



编译&下载

- RT-Thread Studio：在RT-Thread Studio 的包管理器中下载 Titan Board 资源包，然后创建新工程，执行编译。

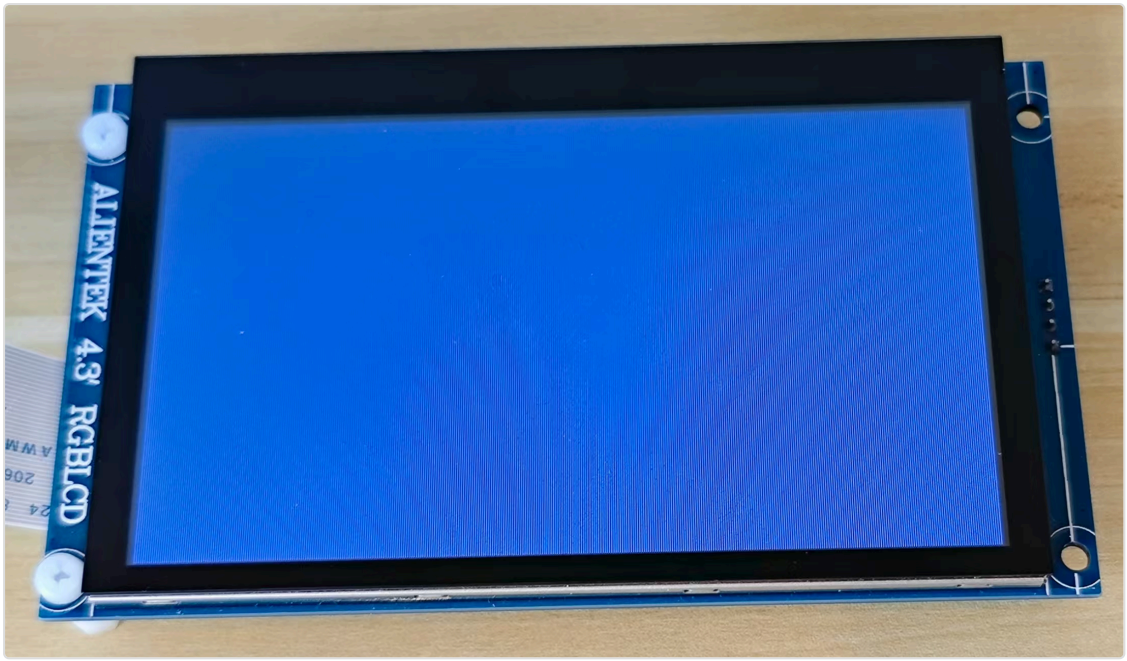
编译完成后，将开发板的 USB-DBG 接口与 PC 机连接，然后将固件下载至开发板。

运行效果

复位开发板后在终端输入 `lcd_test` 命令运行刷屏程序。

```
 \ | /
- RT -     Thread Operating System
 / | \     5.2.0 build Aug 14 2025 16:31:18
 2006 - 2024 Copyright by RT-Thread team
[D/drv_lcd]
lcd framebuffer address:0x70000000

Hello RT-Thread!
================================================
This example project is an rgb lcd routine!
================================================
msh >lcd_test
[D/drv_lcd] red buffer...
[D/drv_lcd] green buffer...
[D/drv_lcd] blue buffer...
[D/drv_lcd] red buffer...
[D/drv_lcd] green buffer...
[D/drv_lcd] blue buffer...
msh >
```


ALIENTEK 4.3' RGBLCD

6.4. USB-PCDC 驱动示例说明

中文 | **English**

简介

本示例演示如何在 **Titan Board** 上使用 **RA8 系列 MCU 的 USB PCDC 模块（r_usb_pcdc）** 将设备枚举为 **USB 虚拟串口（CDC）**，从而在 PC 上以“COM 口/ttyACM” 的形式与板卡进行串行数据通信。

USB PCDC 简介

1. 概述

USB PCDC（USB Peripheral Communications Device Class） 属于 USB 协议中定义的 **CDC（Communications Device Class，通信设备类）** 的一种实现。

它主要用于 **将嵌入式设备通过 USB 接口虚拟成串口（Virtual COM Port, VCOM）**，使主机（PC、嵌入式 Host）能够像访问传统 UART 串口一样访问 USB 设备。

常见应用场景：

- MCU 作为 USB 设备，通过 PCDC 连接到 PC，PC 识别为虚拟串口（COMx）。
- 嵌入式设备与上位机进行数据通信或调试。
- 代替物理 RS-232/RS-485 接口，降低硬件成本。

2. PCDC 的协议特性

1. **CDC 类协议**

- 属于 USB 定义的通信类设备，使用标准类驱动（无需额外安装驱动，在 Windows/Linux/MacOS 中大多数情况自动识别）。

2. **数据传输方式**

- **控制端点（Endpoint 0）**：用于设备枚举、描述符传输。

- **中断端点（Interrupt Endpoint）**：用于状态和控制信号（如 DTR、RTS）。
- **批量传输端点（Bulk Endpoint）**：用于大数据量的串口数据传输（高效可靠）。

3. **虚拟串口功能**

- 在主机端表现为标准串口（COM 口或 ttyUSB 设备）。
- 支持波特率、数据位、校验位、停止位等 UART 常见配置。

3. 典型应用架构

- **USB Device（MCU 端）**：实现 PCDC 协议栈，将 USB 数据与 MCU 内部 UART/应用层对接。
- **USB Host（PC/嵌入式系统）**：通过操作系统自带 CDC 驱动，映射出虚拟串口接口。
- **应用层**：上位机可直接使用串口工具（如 PuTTY、Tera Term、串口助手）进行交互。

4. 优点

- **无需额外驱动**：大多数 OS 自带 CDC 驱动。
- **替代传统串口**：即使 MCU 无物理串口，也能通过 USB 实现调试/通信。
- **速率更高**：USB 全速（12 Mbps）、高速（480 Mbps）带宽远高于传统 UART。
- **通用性强**：支持各种上位机应用（LabVIEW、Python pySerial、C# 串口库等）。

RA8 系列 USB PCDC 模块（r_usb_pcdc）简介

RA8 系列 MCU 内置 **USB PCDC（Peripheral CDC）模块**，专用于将 MCU 枚举为 **USB 虚拟串口（CDC 设备）**，从而实现与 PC 或其他 USB 主机的数据双向通信。该模块与 RA8 的基础 USB 模块（r_usb_basic）结合使用，通过类驱动 r_usb_pcdc 提供完整的 CDC 功能。

1. 模块定位与功能

- **设备模式支持**：实现 MCU 作为 USB 设备连接到主机（Full-Speed / High-Speed，根据具体 MCU 规格）
- **虚拟串口（CDC）功能**：
- 处理类请求：`SET_LINE_CODING`、`GET_LINE_CODING`、`SET_CONTROL_LINE_STATE`、`SEND_BREAK`
- 支持标准 Bulk IN/OUT 端点和 Interrupt IN 端点
- 生成串口状态通知（如 DTR/RTS、线路状态）
- **数据收发管理**：可通过 DMA/DTC 或 FIFO 缓冲实现大数据量高效传输
- **事件回调机制**：连接、断开、配置完成、数据收发完成、类请求处理、错误事件等
- **热插拔与低功耗**：支持 VBUS 侦测、挂起/唤醒、远程唤醒功能
- **应用层接口**：通过类驱动 API 提供读写缓冲、发送完成通知、事件回调机制

2. 支持能力

- **标准 USB 枚举**：支持 Device Descriptor、Configuration Descriptor、Interface Descriptor 等
- **端点管理**：配置 Bulk IN/OUT 和 Interrupt IN 端点
- **类请求解析**：按 CDC 协议解析行编码、控制线状态
- **传输可靠性**：支持 CRC、NAK、STALL 及超时处理
- **多设备与复合设备支持**：可在 Hub 环境下动态识别和管理多个 CDC 设备
- **与 DMA 协同**：优化数据吞吐，减少 CPU 占用

3. 模块架构

```
[ 应用层任务/线程 ]
          ↑            回调/事件通知（连接、配置、收发完成）
[ r_usb_pcdc 类驱动 ]
          ↕            类请求解析、数据收发管理、端点配置
[ r_usb_basic 基础驱动 ]
          ↕            USB 设备枚举、端点控制、SOF 中断、DMA 支持
[ USB 设备控制器/PHY ]
          ↕            USB D+/D- 信号（FS）或 ULPI/HS PHY（HS）
[ USB 线缆 ]  <–> 主机（PC）
```

关键子模块说明：

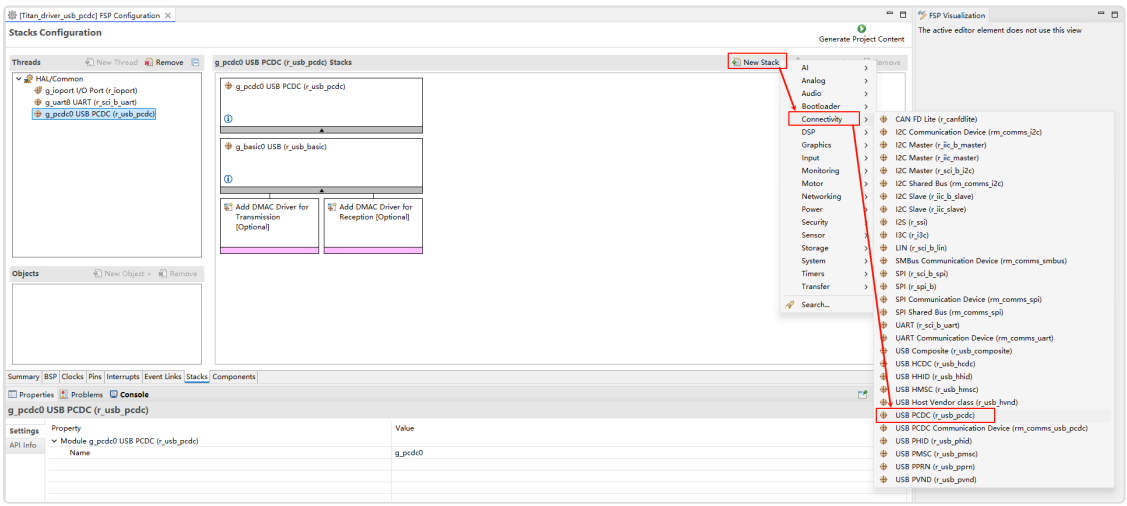
1. **类请求处理器**：解析 CDC 类请求，回调上层应用
2. **端点控制与缓冲**：管理 Bulk/Interrupt 端点传输，支持 DMA/FIFO
3. **事件管理**：连接/断开、配置完成、挂起/恢复、传输完成、错误处理
4. **电源与时钟控制**：USB PHY、VBUS 侦测、挂起唤醒管理

4. 工作流程

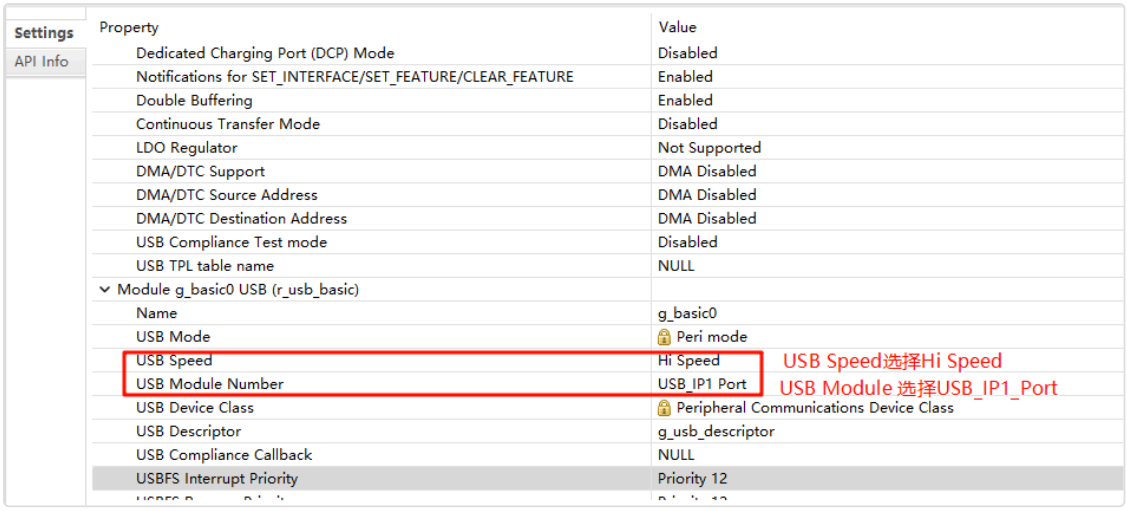
1. **USB 初始化**：USB 控制器、PHY 及类驱动初始化
2. **设备连接检测**：检测 VBUS/ID 信号触发枚举
3. **枚举与配置**：上传描述符、设置接口和端点
4. **端点数据传输**：应用通过 Bulk IN/OUT 与主机进行数据收发
5. **类请求处理**：处理主机发来的 CDC 控制请求
6. **事件回调上报**：将连接、配置、收发完成等事件通知应用
7. **断开处理**：停止端点传输，释放缓冲区，处理拔出事件

FSP 配置

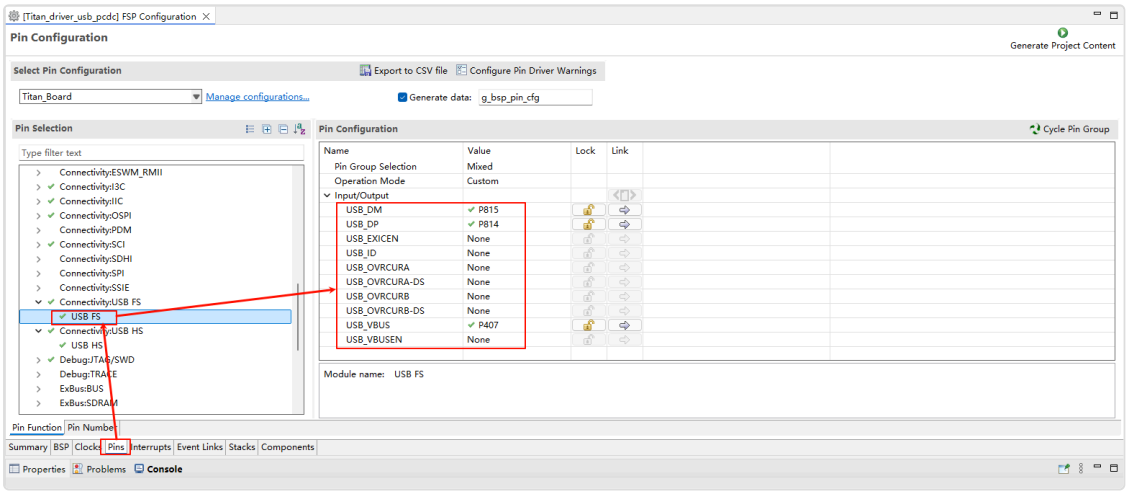
- 新建 r_usb_pcdc stack：



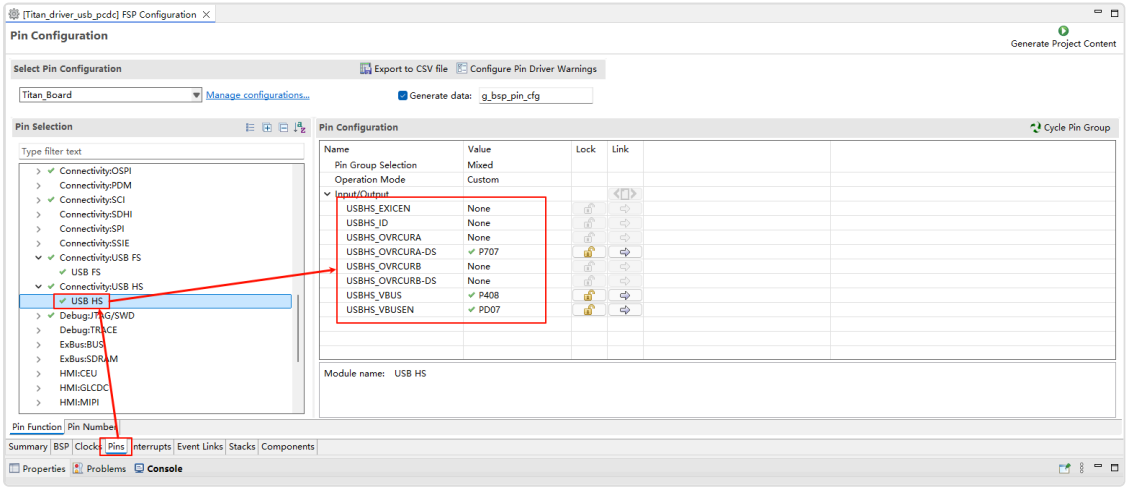
- 配置 r_usb_basic：



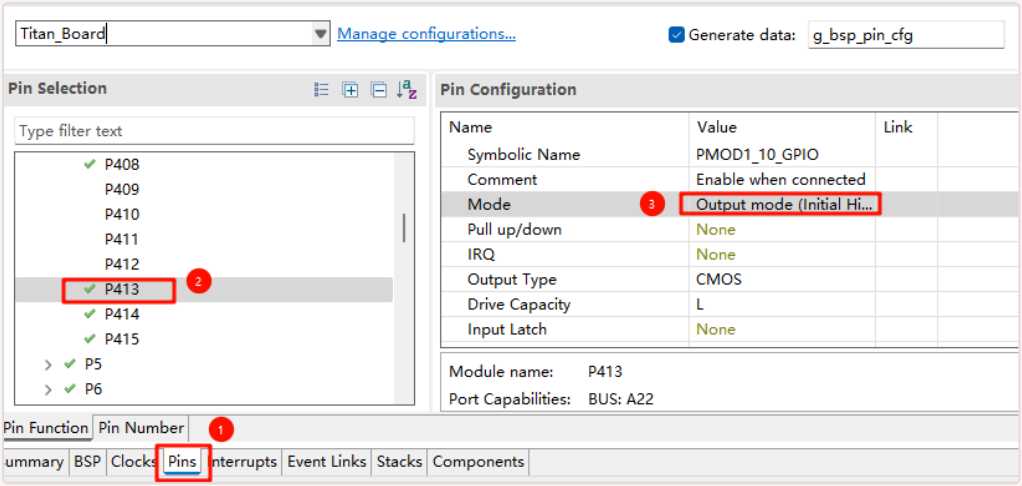
- 配置 USB FS 引脚：



- 配置 USB HS 引脚：

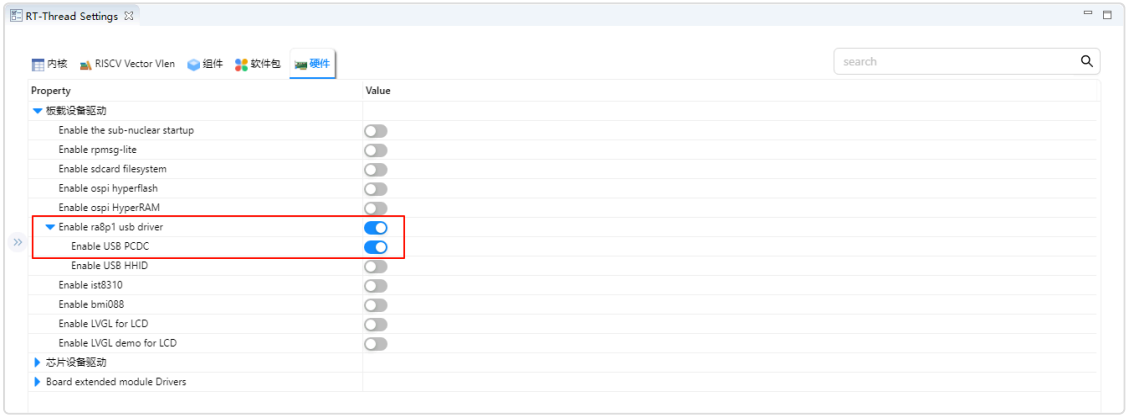


PS：注意，切换HS模式需要拉高P413引脚，如下图所示：



RT-Thread Settings 配置

使能 USB PCDC。



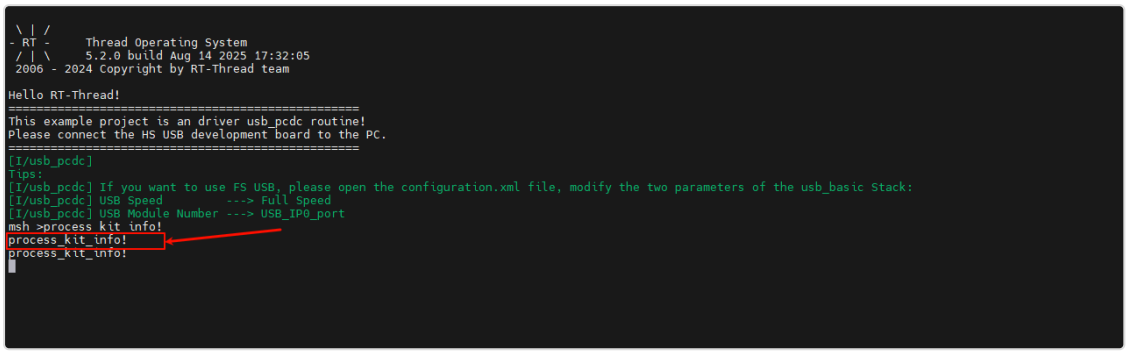
编译&下载

- RT-Thread Studio：在 RT-Thread Studio 的包管理器中下载 Titan Board 资源包，然后创建新工程，执行编译。

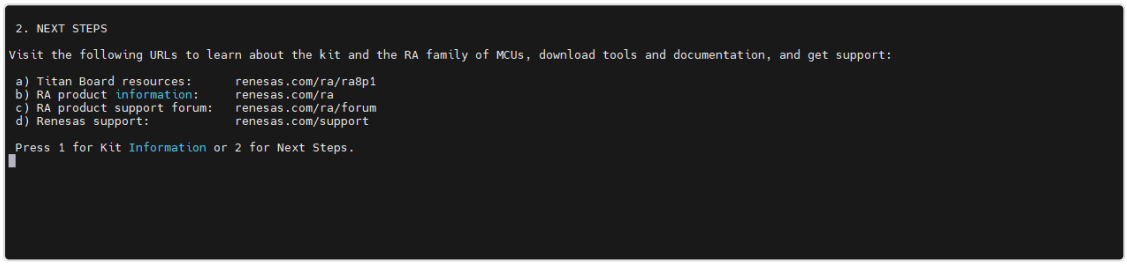
编译完成后，将开发板的 USB-DBG 接口与 PC 机连接，然后将固件下载至开发板。

运行效果

使用 USB 线连接 Titan Board 的 USB-DEV 接口与电脑，使用串口助手打开串口终端，向串口终端中输入 1，能在开发板的调试终端看到输出。



向 USB-PCDC 的串口终端中输入 2，能在 USB-PCDC 的串口终端中看到如下输出。



6.5. WAV 音频播放器示例说明

[中文] | English

简介

本示例展示了如何在 **Titan Board Mini** 上，使用 **ES8156 音频编解码器** 和 **SSI(I2S)** 接口播放 WAV 音频文件，并通过 **RT-Thread Audio Framework** 完成音频输出。

本工程当前仅支持以下音频格式：

- **WAV (RIFF)**
- **PCM 编码**
- **16 种采样率**
- **16-bit 位深**
- **单声道 / 双声道**

不满足以上条件的音频文件将无法正常播放。

主要功能

- 基于 ES8156 的 WAV 音频播放
- 通过 SSI 接口输出 I2S 音频数据
- 支持 `wavplay` 命令播放本地文件
- 支持音量调节、暂停、恢复、停止
- 支持从 SD 卡文件系统读取音频文件

硬件说明

1. ES8156 音频编解码器

Titan Board Mini 板载 **ES8156** 音频编解码器，主要参数如下：

参数	说明
型号	ES8156
类型	立体声 DAC
分辨率	24-bit
采样率范围	8kHz - 192kHz
音频接口	I2S / PCM
控制接口	I2C

2. SSI 音频接口

RA8P1 的 SSI 接口用于输出 I2S 音频数据：

- 支持标准 I2S 协议
- 支持 DMA 传输
- 本工程支持 `16 种采样率 / 16-bit / 单双通道 PCM WAV`

工程结构

```
Titan_Mini_wavplayer/
├── src/
│   └── hal_entry.c
├── libraries/
├── packages/
│   └── wavplayer-latest/
└── README_zh.md
```

使用说明

1. 启动提示

系统启动后会在串口输出当前工程说明，并提示当前支持：

- `16 种采样率`

- `16-bit`
- `mono / stereo PCM WAV`

2. 播放命令

将符合要求的 WAV 文件放入 SD 卡后，使用 msh 命令播放：

```
msh > wavplay -s /sdcard/test.wav
```

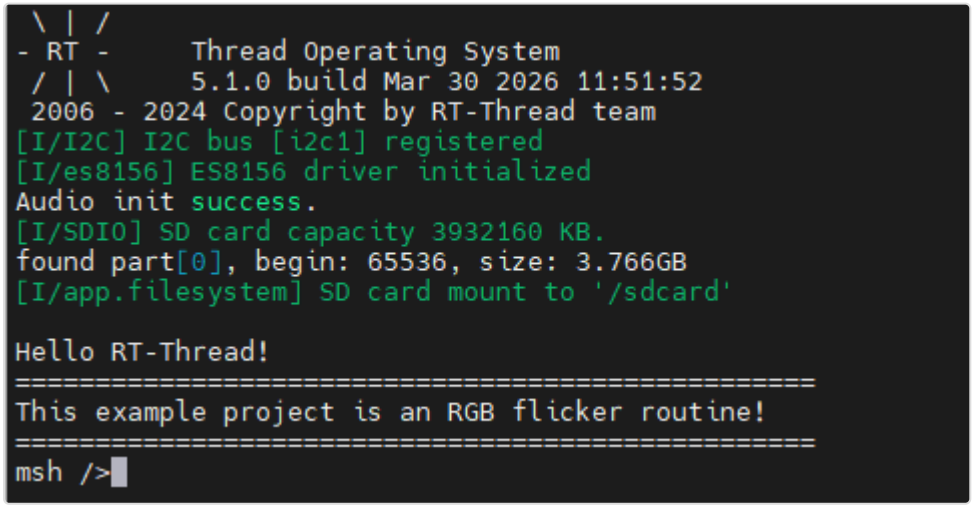
3. 音频文件格式要求

播放文件必须满足以下全部条件：

- **格式**：标准 WAV（RIFF）
- **编码**：PCM
- **采样率**：16 种受支持采样率之一
- **位深**：16-bit
- **声道**：单声道或双声道
- **扩展名**：`.wav`

运行说明

请先将音频文件转换为 **PCM、16-bit、单声道或双声道** 的 WAV 格式，并确保采样率属于支持的 16 种之一，再拷贝到 SD 卡中。开发板上电并挂载 SD 卡后，执行 `wavplay -s 文件名` 即可播放。



相关资料

- RT-Thread 音频框架文档